Mentify

AI Enhanced Digital Learning Platform

Software Design Specification (SDS)

Version 2.0 – As-Implemented Architecture Revision

By
Mohanathas Holins - SE/2021/002
Sivanathan Dilakshan - SE/2021/003
Jeyaruban Jenushan - SE/2021/035
Zaiena Rifkie – SE/2021/040

A report submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science Honors in Software Engineering (B. Sc. SE)

Software Engineering Teaching Unit
Faculty of Science
University of Kelaniya
2026

Version History

Version	Date	Authors	Summary of Changes
1.0	2026.04.19	Project Team	Initial SDS draft
2.0	2026.10.25	Project Team	Revised the system architecture to reflect the implemented fourteen business microservices, Docker Compose-based deployment, PostgreSQL persistence, and JWT-based authentication. Updated architectural diagrams and supporting appendices to align with the final implementation.

Contents

Version History i

List of Figures x

List of Tables xiii

1 Architectural Design 1

1.1 High-Level Architecture . . . 1

1.1.0.1 Microservice Decomposition . . . 1

1.1.0.2 Client Layer . . . 4

1.1.0.3 Request Routing and Infrastructure . . . 4

1.1.0.4 Asynchronous Event-Driven Communication . . . 5

1.1.0.5 Architecture Diagram . . . 5

1.1.0.6 Implementation Status Convention . . . 6

1.2 Container-Based Architecture . . . 7

1.2.0.1 Containerisation and Orchestration . . . 7

1.2.0.2 Data and Storage Services . . . 7

1.2.0.3 CI/CD Pipeline . . . 7

1.2.0.4 External Service Integrations . . . 8

1.3 Justification for Architecture Choice . . . 9

1.3.0.1 Comparison with Alternative Architectures . . . 9

1.3.0.2 Key Justifications . . . 10

1.4 Scalability and Maintainability Considerations . . . 10

1.4.0.1 Scalability . . . 10

1.4.0.2 Maintainability . . . 11

2 Database Design 12

2.1 ER Diagram . . . 12

2.1.0.1 Auth Service . . . 12

2.1.0.2 User Service . . . 12

2.1.0.3 Course Service . . . 13

2.1.0.4 Enrollment Service . . . 13
2.1.0.5 Assignment Service (Assignments) . . . 16
2.1.0.6 Assignment Service (Quizzes) . . . 16
2.1.0.7 Attendance Service . . . 16
2.1.0.8 Exam Service . . . 19
2.1.0.9 Material Service . . . 19
2.1.0.10 Payment Service . . . 21
2.1.0.11 Communication Service . . . 23
2.1.0.12 Notification Service . . . 23
2.1.0.13 Calendar Service . . . 23
2.1.0.14 AI Service . . . 26
2.1.0.15 Analytics Service . . . 26
2.1.0.16 Data Ownership and Cross-Service References . . . 27
2.2 Normalisation Analysis . . . 29
2.2.0.1 Definitions . . . 29
2.2.0.2 Auth and User Services Normalisation . . . 29
2.2.0.3 Course and Enrollment Services Normalisation . . . 29
2.2.0.4 Assignment Service Normalisation . . . 30
2.2.0.5 Attendance and Exam Services Normalisation . . . 31
2.2.0.6 Material and Payment Services Normalisation . . . 31
2.2.0.7 Communication, Notification, Calendar, and AI Services Normalisation . . . 32

3 Class Diagram **33**
3.1 Auth Service . . . 33
3.2 User Service . . . 33
3.3 Course Service . . . 35
3.4 Enrollment Service . . . 35
3.5 Assignment Service . . . 37
3.6 Attendance Service . . . 37
3.7 Exam Service . . . 40
3.8 Material Service and Marketplace Catalogue . . . 40
3.9 Payment Service . . . 40
3.10 Communication Service . . . 40
3.11 Notification Service . . . 40
3.12 Calendar Service . . . 46
3.13 Analytics Service . . . 46
3.14 AI Service . . . 46
3.15 Inter-Service Class Diagram . . . 46

4 Sequence Diagrams **50**
4.1 User Login and Authentication . . . 53
4.1.0.1 Use Case Details . . . 53
4.1.0.2 Actors and System Components . . . 53
4.1.0.3 Pre-conditions . . . 54
4.1.0.4 Main Success Flow . . . 54
4.1.0.5 Alternative and Exception Flows . . . 55
4.1.0.6 Post-conditions . . . 55
4.1.0.7 Sequence Diagram . . . 55
4.2 Administrator Manages Users with RBAC . . . 55
4.2.0.1 Use Case Details . . . 55
4.2.0.2 Actors and System Components . . . 57
4.2.0.3 Pre-conditions . . . 58
4.2.0.4 Main Success Flow . . . 58
4.2.0.5 Alternative and Exception Flows . . . 58
4.2.0.6 Post-conditions . . . 58
4.2.0.7 Sequence Diagram . . . 58
4.3 Teacher Uploads and Organises Learning Materials . . . 60
4.3.0.1 Use Case Details . . . 60
4.3.0.2 Actors and System Components . . . 60
4.3.0.3 Pre-conditions . . . 61
4.3.0.4 Main Success Flow . . . 61
4.3.0.5 Alternative and Exception Flows . . . 61
4.3.0.6 Post-conditions . . . 62
4.3.0.7 Sequence Diagram . . . 62
4.4 AI-Assisted Quiz Generation . . . 62
4.4.0.1 Use Case Details . . . 63
4.4.0.2 Actors and System Components . . . 63
4.4.0.3 Pre-conditions . . . 64
4.4.0.4 Main Success Flow . . . 64
4.4.0.5 Alternative and Exception Flows . . . 64
4.4.0.6 Post-conditions . . . 64
4.4.0.7 Sequence Diagram . . . 64
4.5 Student Attempts Online Quiz with Auto-Grading . . . 66
4.5.0.1 Use Case Details . . . 66
4.5.0.2 Actors and System Components . . . 66
4.5.0.3 Pre-conditions . . . 67
4.5.0.4 Main Success Flow . . . 67
4.5.0.5 Alternative and Exception Flows . . . 67

4.5.0.6 Post-conditions 67
4.5.0.7 Sequence Diagram 67
4.6 Teacher Grades Assignment 69
4.6.0.1 Use Case Details 69
4.6.0.2 Actors and System Components 69
4.6.0.3 Pre-conditions 70
4.6.0.4 Main Success Flow 70
4.6.0.5 Alternative and Exception Flows 70
4.6.0.6 Post-conditions 70
4.6.0.7 Sequence Diagram 70
4.7 Real-time Messaging between Student and Teacher 72
4.7.0.1 Use Case Details 72
4.7.0.2 Actors and System Components 72
4.7.0.3 Pre-conditions 73
4.7.0.4 Main Success Flow 73
4.7.0.5 Alternative and Exception Flows 73
4.7.0.6 Post-conditions 73
4.7.0.7 Sequence Diagram 73
4.8 Teacher Posts Class-wide Announcement 75
4.8.0.1 Use Case Details 75
4.8.0.2 Actors and System Components 75
4.8.0.3 Pre-conditions 76
4.8.0.4 Main Success Flow 76
4.8.0.5 Alternative and Exception Flows 76
4.8.0.6 Post-conditions 76
4.8.0.7 Sequence Diagram 76
4.9 Teacher or Administrator Interacts with the AI Assistant 77
4.9.0.1 Use Case Details 77
4.9.0.2 Actors and System Components 78
4.9.0.3 Pre-conditions 78
4.9.0.4 Main Success Flow 79
4.9.0.5 Alternative and Exception Flows 79
4.9.0.6 Post-conditions 79
4.9.0.7 Sequence Diagram 79
4.10 Student Views Performance Dashboard with Analytics 79
4.10.0.1 Use Case Details 79
4.10.0.2 Actors and System Components 81
4.10.0.3 Pre-conditions 81
4.10.0.4 Main Success Flow 82

4.10.0.5 Alternative and Exception Flows . . . 82
4.10.0.6 Post-conditions . . . 82
4.10.0.7 Sequence Diagram . . . 82
4.11 Student Pays a Course Fee through Stripe . . . 82
4.11.0.1 Use Case Details . . . 82
4.11.0.2 Actors and System Components . . . 83
4.11.0.3 Pre-conditions . . . 83
4.11.0.4 Main Success Flow . . . 84
4.11.0.5 Alternative and Exception Flows . . . 84
4.11.0.6 Post-conditions . . . 85
4.11.0.7 Sequence Diagram . . . 85
4.12 Student Enrolment and Approval . . . 85
4.12.0.1 Use Case Details . . . 86
4.12.0.2 Actors and System Components . . . 86
4.12.0.3 Pre-conditions . . . 87
4.12.0.4 Main Success Flow . . . 87
4.12.0.5 Alternative and Exception Flows . . . 87
4.12.0.6 Post-conditions . . . 87
4.12.0.7 Sequence Diagram . . . 87
4.13 Teacher Records Attendance . . . 89
4.13.0.1 Use Case Details . . . 89
4.13.0.2 Actors and System Components . . . 89
4.13.0.3 Pre-conditions . . . 90
4.13.0.4 Main Success Flow . . . 90
4.13.0.5 Alternative and Exception Flows . . . 90
4.13.0.6 Post-conditions . . . 90
4.13.0.7 Sequence Diagram . . . 90
4.14 Examination Management and Results . . . 90
4.14.0.1 Use Case Details . . . 92
4.14.0.2 Actors and System Components . . . 92
4.14.0.3 Pre-conditions . . . 93
4.14.0.4 Main Success Flow . . . 93
4.14.0.5 Alternative and Exception Flows . . . 93
4.14.0.6 Post-conditions . . . 93
4.14.0.7 Sequence Diagram . . . 93
4.15 Administrator Publishes a Marketplace Product . . . 93
4.15.0.1 Use Case Details . . . 95
4.15.0.2 Actors and System Components . . . 95
4.15.0.3 Pre-conditions . . . 96

4.15.0.4 Main Success Flow 96
4.15.0.5 Alternative and Exception Flows 96
4.15.0.6 Post-conditions 96
4.15.0.7 Sequence Diagram 96
4.16 Authorised Material Download 96
4.16.0.1 Use Case Details 96
4.16.0.2 Actors and System Components 98
4.16.0.3 Pre-conditions 99
4.16.0.4 Main Success Flow 99
4.16.0.5 Alternative and Exception Flows 99
4.16.0.6 Post-conditions 99
4.16.0.7 Sequence Diagram 99
4.17 Password Recovery and First Login 99
4.17.0.1 Use Case Details 99
4.17.0.2 Actors and System Components 101
4.17.0.3 Pre-conditions 102
4.17.0.4 Main Success Flow 102
4.17.0.5 Alternative and Exception Flows 102
4.17.0.6 Post-conditions 102
4.17.0.7 Sequence Diagram 102
4.18 Calendar Projection and Reconciliation 102
4.18.0.1 Use Case Details 104
4.18.0.2 Actors and System Components 104
4.18.0.3 Pre-conditions 104
4.18.0.4 Main Success Flow 105
4.18.0.5 Alternative and Exception Flows 105
4.18.0.6 Post-conditions 105
4.18.0.7 Sequence Diagram 105
4.19 Event-Driven Notification Delivery 105
4.19.0.1 Use Case Details 105
4.19.0.2 Actors and System Components 106
4.19.0.3 Pre-conditions 106
4.19.0.4 Main Success Flow 106
4.19.0.5 Alternative and Exception Flows 108
4.19.0.6 Post-conditions 108
4.19.0.7 Sequence Diagram 108

5 UI/UX Design **109**
5.1 Design Overview . . . 109
5.2 Screen Wireframes for Key User Interfaces . . . 111
5.2.0.1 Administration Dashboard . . . 111
5.2.0.2 User Management – Administration . . . 111
5.2.0.3 Enrollment Management – Administration . . . 111
5.2.0.4 Curriculum and Lesson Builder – Teacher . . . 113
5.2.0.5 Quiz Generator – Teacher . . . 113
5.2.0.6 Attendance Workspace – Teacher . . . 113
5.2.0.7 Student Dashboard . . . 113
5.2.0.8 Course Workspace and Materials – Student . . . 115
5.2.0.9 Quiz Attempt – Student . . . 115
5.2.0.10 Public Website – Pre-registration and Resources . . . 115
5.3 User Flow and Navigation Diagrams . . . 118
5.3.0.1 User Login Flow . . . 119
5.3.0.2 Course Enrolment Flow . . . 120
5.3.0.3 Course Management Flow . . . 120
5.3.0.4 Group Messaging Access Flow . . . 120
5.3.0.5 Public Pre-registration Flow . . . 120
5.4 Accessibility Considerations . . . 120
5.4.0.1 Perceivable . . . 125
5.4.0.2 Operable . . . 125
5.4.0.3 Understandable . . . 125
5.4.0.4 Robust . . . 125

6 Security Design **128**
6.1 Authentication Mechanism . . . 128
6.1.0.1 JSON Web Token (JWT) Authentication . . . 128
6.1.0.2 Credential Lifecycle: First Login, Recovery, and Logout . 130
6.2 Authorisation Levels . . . 131
6.2.0.1 Role Definitions . . . 131
6.2.0.2 Authorisation Enforcement . . . 132
6.3 Data Protection Approach . . . 133
6.3.0.1 Password and Credential Management . . . 133
6.3.0.2 Session Management . . . 134
6.3.0.3 Sensitive Data Classification . . . 134
6.4 Input Validation and Sanitisation Strategy . . . 134
6.4.0.1 Validation Layer . . . 134
6.4.0.2 Sanitisation Layer . . . 135

6.5 Encryption Standards for Data at Rest and in Transit . . . 136
6.5.0.1 Encryption in Transit . . . 136
6.5.0.2 Encryption at Rest . . . 136
6.5.0.3 Key Management . . . 136
6.6 Security Flows for Payments and Files . . . 137
6.6.0.1 Payment Webhook Verification . . . 137
6.6.0.2 Authorised File Download . . . 137

7 Deployment Design 140
7.1 Hosting Plan . . . 140
7.2 Hardware Requirements . . . 142
7.3 Network Diagram . . . 143
7.3.0.1 Network Layer Description . . . 143
7.4 Environment Specification . . . 144
7.5 CI/CD Pipeline Overview . . . 145
7.5.0.1 Source Control and Branching Strategy . . . 145
7.5.0.2 Pipeline Stages . . . 146
7.5.0.3 External Integrations in Deployment . . . 147
7.5.0.4 Future Production Upgrade Path . . . 148

Bibliography 149

A Interface Design and Distributed Workflows 154
A.0.1 A.1 Interface Documentation Template . . . 154
A.0.2 A.2 Interface Catalogue . . . 155
A.0.3 A.3 Distributed Workflow Rules . . . 157

B Software Testing And Quality Evidence 158
B.0.1 B.1 Testing Architecture . . . 158
B.0.2 B.2 Test Execution Model . . . 158
B.0.3 B.3 Testing Limitations . . . 159
B.0.4 B.4 Design-to-Test Traceability Matrix . . . 159

C Implementation Status And Known Deviations 161

D Architectural Decision Records 164

E Sds Version 1.0 To 2.0 Change Register 166

List of Figures

1.1 High-Level System Architecture Diagram (As Implemented) 5
1.2 System Context Diagram 6

2.1 Entity-Relationship Diagram – Auth Service 13
2.2 Entity-Relationship Diagram – User Service 14
2.3 Entity-Relationship Diagram – Course Service 15
2.4 Entity-Relationship Diagram – Enrollment Service 16
2.5 Entity-Relationship Diagram – Assignment Service (Assignments) 17
2.6 Entity-Relationship Diagram – Assignment Service (Quizzes) 18
2.7 Entity-Relationship Diagram – Attendance Service 19
2.8 Entity-Relationship Diagram – Exam Service 20
2.9 Entity-Relationship Diagram – Material Service 21
2.10 Entity-Relationship Diagram – Payment Service 22
2.11 Entity-Relationship Diagram – Communication Service 23
2.12 Entity-Relationship Diagram – Notification Service 24
2.13 Entity-Relationship Diagram – Calendar Service 25
2.14 Entity-Relationship Diagram – AI Service 26
2.15 Analytics Service Data Flow (Derived Data) 27

3.1 Class Diagram – Auth Service 34
3.2 Class Diagram – User Service 35
3.3 Class Diagram – Course Service 36
3.4 Class Diagram – Enrollment Service 37
3.5 Class Diagram – Assignment Service 38
3.6 Class Diagram – Attendance Service 39
3.7 Class Diagram – Exam Service 41
3.8 Class Diagram – Material Service and Marketplace Catalogue 42
3.9 Class Diagram – Payment Service 43
3.10 Class Diagram – Communication Service 44
3.11 Class Diagram – Notification Service 45
3.12 Class Diagram – Calendar Service 47

3.13 Class Diagram – Analytics Service 48
3.14 Class Diagram – AI Service 49
3.15 Inter-Service Class Diagram 49

4.1 Sequence Diagram – User Login and Authentication 56
4.2 Sequence Diagram – Administrator Manages Users with RBAC 59
4.3 Sequence Diagram – Teacher Uploads and Organises Learning Materials . . 62
4.4 Sequence Diagram – AI-Assisted Quiz Generation 65
4.5 Sequence Diagram – Student Attempts Online Quiz with Auto-Grading . . 68
4.6 Sequence Diagram – Teacher Grades Assignment 71
4.7 Sequence Diagram – Real-time Messaging between Student and Teacher . . 74
4.8 Sequence Diagram – Teacher Posts Class-wide Announcement 77
4.9 Sequence Diagram – Teacher or Administrator Interacts with the AI Assistant 80
4.10 Sequence Diagram – Student Views Performance Dashboard with Analytics 84
4.11 Sequence Diagram – Student Pays a Course Fee through Stripe 85
4.12 Sequence Diagram – Student Enrolment and Approval 88
4.13 Sequence Diagram – Teacher Records Attendance 91
4.14 Sequence Diagram – Examination Management and Results 94
4.15 Sequence Diagram – Administrator Publishes a Marketplace Product 97
4.16 Sequence Diagram – Authorised Material Download 100
4.17 Sequence Diagram – Password Recovery and First Login 103
4.18 Sequence Diagram – Calendar Projection and Reconciliation 107
4.19 Sequence Diagram – Event-Driven Notification Delivery 108

5.1 Administration Dashboard – Wireframe 112
5.2 User Management – Wireframe 112
5.3 Enrollment Management – Wireframe 113
5.4 Curriculum and Lesson Builder – Wireframe 114
5.5 Quiz Generator (AI Draft) – Wireframe 114
5.6 Attendance Workspace – Wireframe 115
5.7 Student Dashboard – Wireframe 116
5.8 Course Workspace and Materials – Wireframe 116
5.9 Quiz Attempt – Wireframe 117
5.10 Public Website – Wireframe 117
5.11 User Login Flow Diagram 119
5.12 Course Enrolment Flow Diagram 121
5.13 Course Management Flow Diagram 122
5.14 Group Messaging Access Flow Diagram 123
5.15 Public Pre-registration Flow Diagram 124

6.1 Security Architecture Diagram 129
6.2 Payment Webhook Verification Flow 138
6.3 Authorised File Download Flow 139

7.1 System Network and Deployment Architecture Diagram 143
7.2 CI/CD and Test Architecture Diagram 145

List of Tables

1.1 Microservice Decomposition and Data Store Mapping 2
1.2 Implementation Status Labels 6
1.3 Container-Based Infrastructure Components 8
1.4 Architectural Style Comparison 9

2.1 Data Ownership Matrix 27
2.2 Auth and User Services – Normalisation Summary 29
2.3 Course and Enrollment Services – Normalisation Summary 30
2.4 Assignment Service – Normalisation Summary 30
2.5 Attendance and Exam Services – Normalisation Summary 31
2.6 Material and Payment Services – Normalisation Summary 31
2.7 Communication, Notification, Calendar, and AI Services – Normalisation Summary 32

4.1 Summary of Major Use Cases and Corresponding Sequence Diagrams . . . 50
4.2 Use Case Details – User Login and Authentication 53
4.3 Actors and Components – User Login and Authentication 53
4.4 Use Case Details – Administrator Manages Users with RBAC 57
4.5 Actors and Components – Administrator Manages Users with RBAC 57
4.6 Use Case Details – Teacher Uploads and Organises Learning Materials . . . 60
4.7 Actors and Components – Teacher Uploads and Organises Learning Materials 60
4.8 Use Case Details – AI-Assisted Quiz Generation 63
4.9 Actors and Components – AI-Assisted Quiz Generation 63
4.10 Use Case Details – Student Attempts Online Quiz with Auto-Grading . . . 66
4.11 Actors and Components – Student Attempts Online Quiz with Auto-Grading 66
4.12 Use Case Details – Teacher Grades Assignment 69
4.13 Actors and Components – Teacher Grades Assignment 69
4.14 Use Case Details – Real-time Messaging between Student and Teacher . . . 72
4.15 Actors and Components – Real-time Messaging between Student and Teacher 72
4.16 Use Case Details – Teacher Posts Class-wide Announcement 75
4.17 Actors and Components – Teacher Posts Class-wide Announcement 75

4.18 Use Case Details – Teacher or Administrator Interacts with the AI Assistant 77
4.19 Actors and Components – Teacher or Administrator Interacts with the AI Assistant . . . 78
4.20 Use Case Details – Student Views Performance Dashboard with Analytics . 81
4.21 Actors and Components – Student Views Performance Dashboard with Analytics . . . 81
4.22 Use Case Details – Student Pays a Course Fee through Stripe . . . 82
4.23 Actors and Components – Student Pays a Course Fee through Stripe . . . 83
4.24 Use Case Details – Student Enrolment and Approval . . . 86
4.25 Actors and Components – Student Enrolment and Approval . . . 86
4.26 Use Case Details – Teacher Records Attendance . . . 89
4.27 Actors and Components – Teacher Records Attendance . . . 89
4.28 Use Case Details – Examination Management and Results . . . 92
4.29 Actors and Components – Examination Management and Results . . . 92
4.30 Use Case Details – Administrator Publishes a Marketplace Product . . . 95
4.31 Actors and Components – Administrator Publishes a Marketplace Product . 95
4.32 Use Case Details – Authorised Material Download . . . 98
4.33 Actors and Components – Authorised Material Download . . . 98
4.34 Use Case Details – Password Recovery and First Login . . . 101
4.35 Actors and Components – Password Recovery and First Login . . . 101
4.36 Use Case Details – Calendar Projection and Reconciliation . . . 104
4.37 Actors and Components – Calendar Projection and Reconciliation . . . 104
4.38 Use Case Details – Event-Driven Notification Delivery . . . 105
4.39 Actors and Components – Event-Driven Notification Delivery . . . 106

5.1 Summary of Key User Interface Screens . . . 110
5.2 Summary of User Flow Diagrams . . . 118
5.3 WCAG 2.1 Accessibility Design Intent Summary . . . 125

6.1 Authentication Token Summary . . . 130
6.2 Role-Based Access Control Definitions . . . 131
6.3 Representative Authorisation Matrix . . . 132
6.4 Data Sensitivity Classification and Controls . . . 134
6.5 Input Validation and Sanitisation Mechanisms . . . 135
6.6 Encryption Standards Summary . . . 136

7.1 Hosting Plan Summary . . . 140
7.2 Deployment Resource Planning Profile . . . 142
7.3 Environment Specification Matrix . . . 144
7.4 CI/CD Pipeline Stages . . . 146

7.5 External Integration Summary 147

A.1 Interface Documentation Template 154
A.2 Interface Catalogue 155

B.1 Test Execution Model 158
B.2 Design-to-Test Traceability Matrix 159

C.1 Known Deviations and Implementation Status 161

D.1 Architectural Decision Register 164

E.1 SDS Version 1.0 to 2.0 Change Register 166

Chapter 1

Architectural Design

This chapter presents the architectural design of the system as implemented, describing the high-level structure, the container-based deployment model, the rationale behind key architectural decisions, and the mechanisms employed to support scalability and maintainability. The system is built upon a microservices architecture. The backend is packaged as Docker containers and defined for deployment with Docker Compose on a single Docker host (planned on Amazon Web Services EC2); Kubernetes-based orchestration on AWS is retained only as a future upgrade option. Version 2.0 of this document replaces the six-service, EKS-based design of Version 1.0 with the as-implemented architecture of the development branches of the frontend, backend, and test repositories.

1.1 High-Level Architecture

The system adopts a Microservices Architecture pattern, wherein the application is decomposed into a collection of small, independently deployable services, each responsible for a distinct business capability. This architectural style was selected over monolithic or layered alternatives due to its alignment with the system's scalability, maintainability, and continuous delivery requirements. Its costs, including distributed data consistency, authentication propagation, and operational complexity, are discussed in Section 1.3.

1.1.0.1 Microservice Decomposition

The backend is decomposed into fourteen business microservices, each developed using Java and the Spring Boot framework, together with three infrastructure services and one shared library. Repository module names are given exactly as they appear in the source tree (for example entrollment-service and calender-service); in explanatory text and diagram labels the standard spellings Enrollment Service and Calendar Service are used.

Table 1.1: Microservice Decomposition and Data Store Mapping

Microservice	Responsibility	Data Store
auth-service	Authentication, password recovery (OTP), first-login password change, JWT access and refresh tokens, and public key (JWKS) publication.	PostgreSQL (Auth DB)
user-service	User profiles for students, teachers, administrators, and super administrators; registration, invitation, account status, and deletion.	PostgreSQL (User DB)
course-service	Grades, courses, modules, lessons, and the curriculum hierarchy.	PostgreSQL (Course DB)
entrollment-service	Enrollment requests, approval and rejection, student-course relationships, payment verification, and group membership synchronisation.	PostgreSQL (Enrollment DB)
assignment-service	Assignments, rubrics, submissions, grading, quizzes, questions, attempts, and results.	PostgreSQL (Assignment DB)
attendance-service	Attendance sessions, attendance records, and summaries.	PostgreSQL (Attendance DB)
exam-service	Examinations, scheduling, attachments, marks, result sheets, and student results.	PostgreSQL (Exam DB)
material-service	Learning-resource metadata, uploads, access authorisation, presigned downloads, and the marketplace product catalogue.	PostgreSQL (Material DB) + S3-compatible storage

payment-service	Grade-based fees, course payments through Stripe, webhook handling, payment history, and refunds.	PostgreSQL (Payment DB)
communication-service	Communication groups, membership, messages, announcements, and real-time WebSocket delivery.	PostgreSQL (Communication DB)
notification-service	Event-driven in-app notifications and supported e-mail delivery.	PostgreSQL (Notification DB) + Kafka consumer
calender-service	Calendar events, venues, role-based event queries, and projections from academic workflows.	PostgreSQL (Calendar DB)
analytics-service	Cross-domain aggregation and reporting for administrator, teacher, and student dashboards.	PostgreSQL (Analytics DB); data assembled from source services
ai-service	AI conversations, content generation, summarisation, document extraction, contextual tools, guards, and AI audit data.	PostgreSQL (AI DB)
api-gateway	Single entry point: routing, CORS, and the first JWT security boundary.	Stateless
service-registry	Eureka-based service discovery.	Infrastructure
config-server	Externalised Spring application configuration.	Infrastructure
common-lib	Shared security, contracts, and cross-cutting functionality (shared library, not a microservice).	Library

Each business microservice owns its own logical database, adhering to the Database-per-Service pattern. References between services are held as identifiers and are not enforced as

database-level foreign keys, which preserves loose coupling. Logical separation does not imply separate database servers: the reviewed production configuration hosts the service databases on a single PostgreSQL 14 container.

1.1.0.2 Client Layer

The frontend repository provides four identifiable web experiences. All of them communicate with the backend through the API Gateway, which serves as the single entry point into the backend infrastructure.

- Administration portal: user management, grade and course management, enrollment oversight, attendance, examinations, quizzes, materials, marketplace, analytics, reports, communication, calendar, notifications, and the AI assistant.
- Teacher portal: courses, curriculum and lessons, assignment review, quizzes, attendance, gradebook, examinations, student directory, messaging, calendar, and AI assistance.
- Student portal: course exploration, enrollments, course workspace, materials, assignments, quiz taking, examinations, grades, calendar, messages, notifications, profile, and settings.
- Public institute website: public information pages (Home, About, Services, Contact), pre-registration, resources, privacy policy, terms of use, and English, Sinhala, and Tamil translation assets.

The portals are built with React and Next.js and are separate applications rather than a single page hierarchy. The original description of a generic mobile and web client pair is superseded; no native mobile application is part of the current implementation.

1.1.0.3 Request Routing and Infrastructure

The request path follows the sequence below. Components marked as future are not part of the reviewed deployment.

- A portal or the public website sends an HTTP request.
- The deployed Caddy reverse proxy (listening on port 80 in the reviewed Caddyfile) forwards supported requests to api-gateway:8080. The gateway port is exposed on the loopback interface of the host.
- The Spring Cloud Gateway applies routing and the first security checks (CORS and JWT handling).
- Backend services register with Eureka for service discovery and obtain configuration from the Config Server.
- Services call each other synchronously through HTTP and OpenFeign where an immediate response is required.
- Kafka carries supported asynchronous events, and Redis supports configured caching and coordination.

An AWS Application Load Balancer, an EKS ingress controller, and managed TLS termination are not part of the reviewed deployment. They appear only in the proposed future upgrade path in Section 7.5.4.

1.1.0.4 Asynchronous Event-Driven Communication

Inter-service communication that does not require an immediate response is facilitated through Apache Kafka. Services publish success events after completing significant operations, using shared components from the common library, and the notification-service consumes them to create notifications and deliver supported e-mails. Event payloads are designed to avoid sensitive data, and duplicate-event handling is applied where implemented. Failure and retry behaviour is limited to what the Kafka and consumer configuration provides.

Not all inter-service workflows are event-based. Enrollment, payment verification, material authorisation, group membership synchronisation, and analytics aggregation use direct HTTP/OpenFeign calls. The reviewed production configuration runs a single Kafka 3.9.1 broker and therefore does not provide broker-level fault tolerance.

1.1.0.5 Architecture Diagram

The overall system architecture is illustrated in Figure 1 below. The diagram is a simplified conceptual grouping; the service-to-service dependencies are shown in Figure 32 and the deployed network in Figure 70.

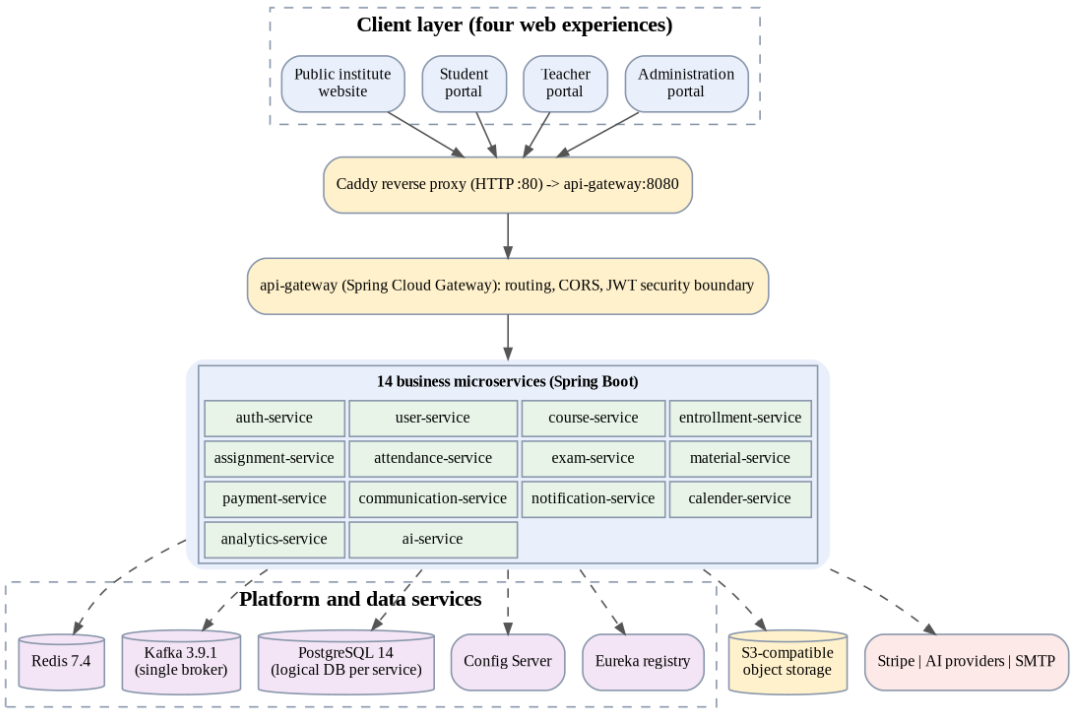


Figure 1.1: High-Level System Architecture Diagram (As Implemented)

The system context of Mentify, showing its human actors and external providers, is given in Figure 2.

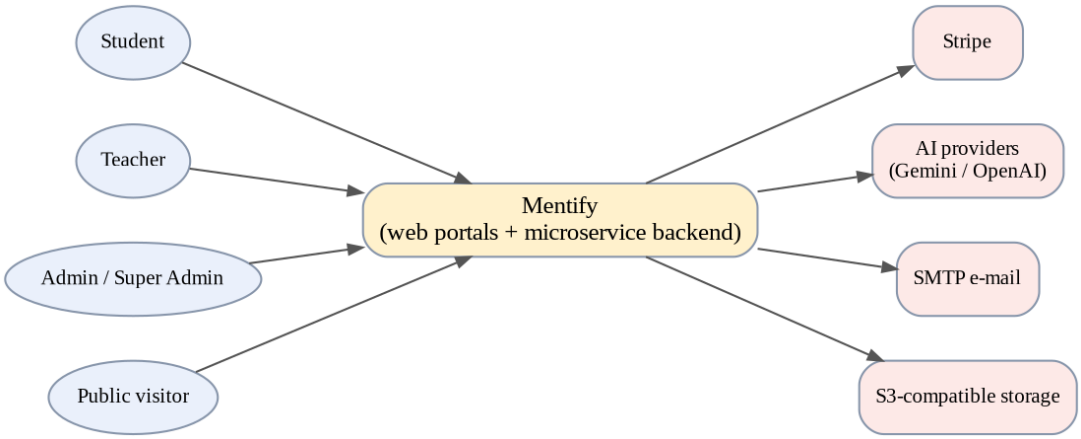


Figure 1.2: System Context Diagram

1.1.0.6 Implementation Status Convention

Because a screen, an API client method, a controller, an automated test, and a verified production workflow are different levels of evidence, design elements in this document are labelled with the following status values where status is relevant.

Table 1.2: Implementation Status Labels

Status Label	Meaning
Implemented in Source	The element exists in the development branch of the repository.
Integrated and Tested	The element is implemented and covered by an executed test with recorded evidence.
Configured, Not Deployment-Verified	Configuration exists (for example a Compose definition or workflow) but a successful deployment or execution has not been evidenced.
Partially Implemented	Part of the element is present, or the frontend and backend contracts differ.
Future Design	Planned or proposed; not part of the reviewed implementation.

1.2 Container-Based Architecture

The system is packaged as containers and defined for deployment with Docker Compose. The principles of containerisation, externalised configuration, and automation guide the infrastructure, while the managed-cloud services described in Version 1.0 are retained only as future options.

1.2.0.1 Containerisation and Orchestration

Each microservice and portal is packaged as a Docker container image. The backend production Compose file defines the infrastructure services (service registry, configuration server, API Gateway), the business microservices, PostgreSQL 14, Redis 7.4, Apache Kafka 3.9.1, and a Caddy reverse proxy, with named persistent volumes, container health checks, and an internal Docker network. Image names are configurable through the DOCKERHUB_IMAGE variable. The frontend production Compose file defines three portal containers (Administration, Teacher, and Student) that attach to the backend's external Docker network. The public website is present in the repository but is not included in that Compose file and is deployed separately.

Kubernetes, with self-healing, horizontal scaling, and rolling updates, is not demonstrated by the reviewed deployment and is therefore a future option rather than a current capability.

1.2.0.2 Data and Storage Services

Relational data for all business services is persisted in PostgreSQL, using Spring Data JPA and Flyway migrations. File and media content is stored in S3-compatible object storage, with application-level metadata and authorisation held by the material-service. Redis supports caching and coordination. The MongoDB Atlas and Amazon RDS components described in Version 1.0 are not part of the current design: the AI and Analytics services use PostgreSQL, and the reviewed deployment hosts PostgreSQL in a container.

1.2.0.3 CI/CD Pipeline

Automation is implemented with GitHub Actions workflows in each repository. The frontend repository provides continuous-integration and frontend deployment workflows. The backend repository provides build and test workflows, container image creation, coverage artefacts, and deployment automation where configured. The test repository provides static Maven verification, separately enabled live integration testing, performance workflows, and manual staging smoke checks. A configured workflow evidences an automation design; a

successful run and deployment record is required to claim that a particular release was built, tested, and deployed. Details are given in Section 7.5.

1.2.0.4 External Service Integrations

The platform integrates with the following external services:
• Stripe processes course payments through Payment Intents, signed webhooks, and refunds.
• AI providers (configured Gemini and OpenAI adapters; the provider enumeration also names Claude) are consumed by the AI Service for summarisation, quiz draft generation, and role-authorised conversation. Provider availability depends on implemented adapters and runtime configuration.
• SMTP e-mail delivers invitations, one-time passwords for password recovery, and supported notifications.
• S3-compatible object storage stores uploaded learning resources, accessed through authorised presigned URLs.

Google OAuth, used for authentication in Version 1.0, is not part of the implemented authentication architecture; identity is provided by the first-party auth-service.

Table 1.3: Container-Based Infrastructure Components

Component	Technology (As Configured)	Purpose
Container Packaging	Docker images; Docker Compose	Packaging and orchestration of services on a single Docker host
Reverse Proxy	Caddy	Receives external HTTP requests and forwards to the API Gateway
API Entry Point	Spring Cloud Gateway	Routing, CORS, and first-level JWT security
Service Discovery	Netflix Eureka	Dynamic service registration and discovery
Config Management	Spring Cloud Config Server	Externalised configuration management
Inter-service Calls	OpenFeign (HTTP)	Synchronous service-to-service integration
Relational Databases	PostgreSQL 14 (container, named volume)	Structured data for all business services
Schema Migration	Flyway	Versioned database migrations

Caching Layer	Redis 7.4	Configured caching and coordination
Message Broker	Apache Kafka 3.9.1 (single broker)	Asynchronous event-driven communication
Object Storage	S3-compatible storage	File and media asset storage
Real-time Messaging	WebSocket / STOMP	Group messaging and AI chat
Payments	Stripe	Course payment processing
CI/CD Automation	GitHub Actions	Build, test, image, and verification workflows

1.3 Justification for Architecture Choice

The selection of a microservices architecture was driven by the functional and non-functional requirements of the system. The following rationale is presented in comparison with the principal architectural alternatives, and includes the costs that the choice introduces.

1.3.0.1 Comparison with Alternative Architectures

Table 1.4: Architectural Style Comparison

Criterion	Monolithic	Layered (MVC)	Microservices (Selected)
Independent Deployability	Not supported	Partially supported	Supported per service
Technology Heterogeneity	Not supported	Limited	Supported
Fault Isolation	Poor	Moderate	Strong where dependencies are asynchronous; limited by synchronous calls
Horizontal Scalability	Coarse-grained	Coarse-grained	Fine-grained per service (not exercised in the Compose deployment)
Team Autonomy	Low	Moderate	High

Data Consistency	Simple (single transaction)	Simple	Complex (eventual consistency, no cross-service transactions)
Operational Complexity	Low	Moderate	High (discovery, configuration, many containers)

1.3.0.2 Key Justifications

The following factors informed the final architectural decision:

• Independent Deployability: Each service can be built, tested, and deployed in isolation, enabling faster release cycles and reducing the blast radius of defects.

• Domain Alignment: Dedicated services for enrollment, attendance, examinations, materials, payments, notifications, calendar, and analytics keep each domain's data and rules in one place.

• Fault Isolation (qualified): Separating services limits the impact of a failure in an independent domain, for example an AI provider outage does not prevent course browsing. However, synchronous HTTP dependencies, such as enrollment calling payment-service, can still propagate failures unless timeouts and fallbacks are applied.

• Spring Boot Ecosystem: Spring Cloud Gateway, Config Server, Eureka, OpenFeign, Spring Security, and Actuator align directly with the architectural requirements.

The accepted costs of this choice are distributed data consistency without cross-service transactions, authentication propagation between services, deployment complexity, service discovery, network failure handling, service ownership, and test-environment complexity. These are addressed through logical references between databases, JWT validation in every service, shared library contracts, health checks, and the layered test strategy described in Appendix B.

1.4 Scalability and Maintainability Considerations

Scalability and maintainability are design concerns embedded in the architecture. This section distinguishes mechanisms that are implemented or configured from options reserved for future production growth.

1.4.0.1 Scalability

Implemented or configured mechanisms:

• Service separation allows each domain service to be built and deployed independently.

• Redis is configured for caching and coordination, reducing repeated database and computation work.
• Kafka decouples producers from consumers for supported workflows such as notifications.
• S3-compatible object storage offloads large file content from the application tier.
• Container health checks and Eureka discovery support service availability management.

Future design options (not demonstrated by the Compose deployment):

• Kubernetes orchestration with Horizontal Pod Autoscaling.
• Load balancing across multiple hosts with a managed AWS Application Load Balancer.
• A multi-broker Kafka cluster and a replicated or managed PostgreSQL deployment with automated failover.

The production Compose file currently shows one Kafka broker and one PostgreSQL service. The deployment must therefore not be described as fault-tolerant or multi-zone. Automatic container restarts alone do not constitute high availability.

1.4.0.2 Maintainability

• Service Isolation: Each microservice has a defined API boundary and owns its data. Internal changes do not require changes in other services provided the contract is preserved.
• Externalised Configuration: The Spring Cloud Config Server stores environment-specific configuration outside the application code.
• Versioned Schema Changes: Flyway migrations record database schema evolution per service.
• Automated Verification: GitHub Actions workflows run static verification, and a separate test repository provides integration, end-to-end, contract, security, resilience, and performance tests, with live execution enabled explicitly.
• Containerisation: Docker containers provide reproducible runtime environments across local, integration, and deployment settings.
• Observability: Spring Boot Actuator health endpoints are configured where applicable and are used by container health checks.

Together, these mechanisms support scaling the system to growing demand while keeping it straightforward to operate, modify, and extend. Evidence of successful runs is required before any of the automated mechanisms is described as verified for a release.

Chapter 2

Database Design

This chapter presents the logical database design of the system. Version 1.0 described six logical data models; the implementation uses one logical database per business service. The models below are derived from the entities, DTOs, enumerations, and service behaviour of the development branch. Column-level details must be reconciled with the Flyway migration scripts of each service before final submission.

2.1 ER Diagram

The following subsections present the ER diagram for each microservice. Entities depicted with a dashed border are owned by a different service and are shown for contextual clarity only. Relationships to such entities are logical references (identifiers) and are not database foreign keys. Relationships are annotated with their cardinality (1, N, 0..1, or ranges).

2.1.0.1 Auth Service

The Auth Service owns authentication credentials and session artefacts, kept separate from profile data. AuthUser holds the unique email, the password hash, the role (STUDENT, TEACHER, ADMIN, or SUPER_ADMIN), the account status, and a flag that forces a password change at first login. RefreshToken records issued refresh sessions with expiry and revocation. PasswordResetOtp records one-time passwords for recovery with expiry and single-use state. JWT signing keys are managed through configuration and published as a JWKS document, and are therefore not modelled as an application table.

2.1.0.2 User Service

The User Service owns profile data and is linked to the authentication identity through authUserId. The model is a profile base with role-specific extensions: StudentProfile (grade and guardian details), TeacherProfile (specialisation and hire date), and AdminProfile (access level). Version 1.0 used an inheritance hierarchy with passwordHash on the User entity

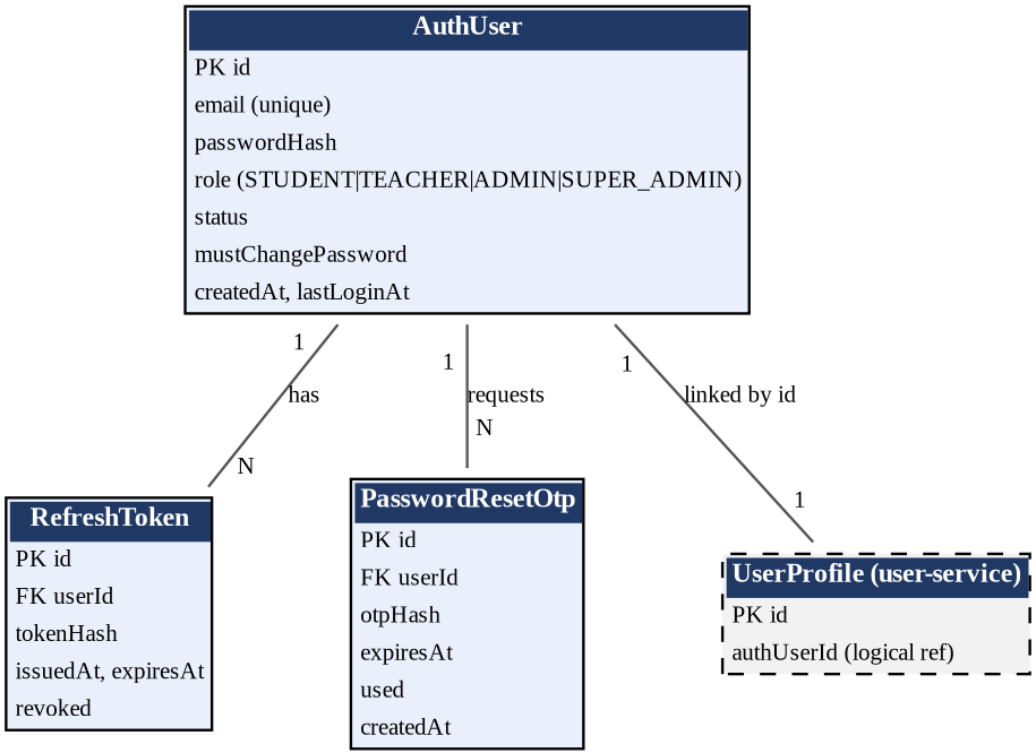


Figure 2.1: Entity-Relationship Diagram – Auth Service

and a SupportAgent subtype; both are removed. Super Admin is a role value rather than a separate table.

2.1.0.3 Course Service

The Course Service governs the academic hierarchy: Grade, Course, Module, and Lesson. A Course belongs to a Grade, is assigned to a teacher by identifier, and carries the attributes supported by CourseRequest, including course code, subject, academic year, semester, level, capacity, credits, schedule, grading weights, teaching assistants, and the online, visible, and published flags. Enrollment and learning materials, which Version 1.0 placed in this service, are now owned by the Enrollment Service and the Material Service respectively.

2.1.0.4 Enrollment Service

The Enrollment Service owns the student-course relationship. Enrollment carries a status (PENDING, APPROVED, or REJECTED), the source of the enrollment (student self-enrollment or staff-created), a logical reference to the verifying payment for paid courses, and decision metadata. A unique constraint on the student and course prevents duplicate enrollment. Student self-enrollment checks course existence, duplicates, and (for paid courses) a successful payment before saving with status PENDING; staff-created enrollments follow a separate path with different authorisation checks.

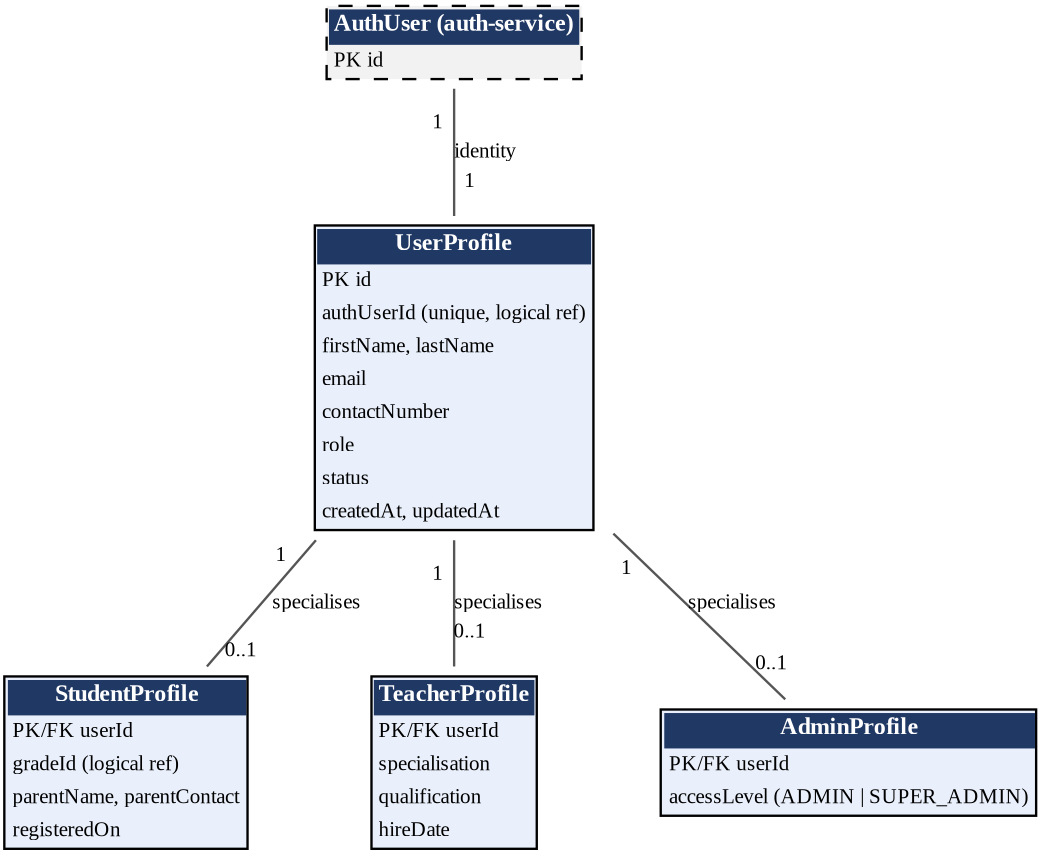


Figure 2.2: Entity-Relationship Diagram – User Service

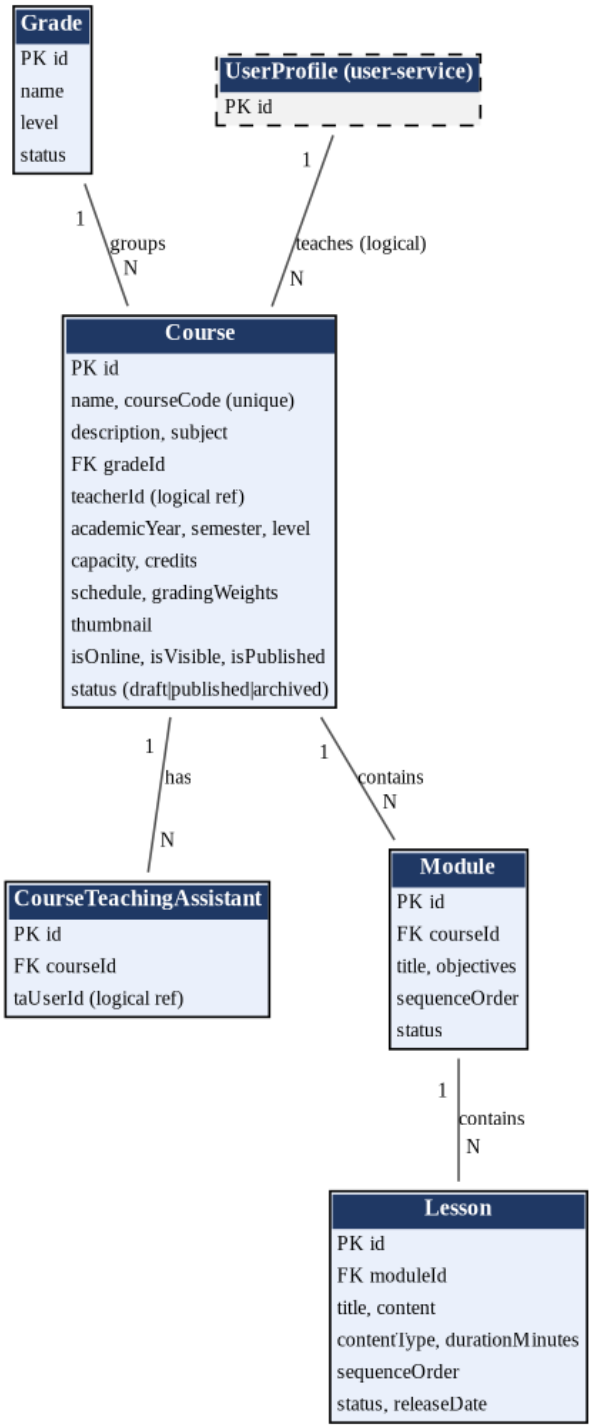


Figure 2.3: Entity-Relationship Diagram – Course Service

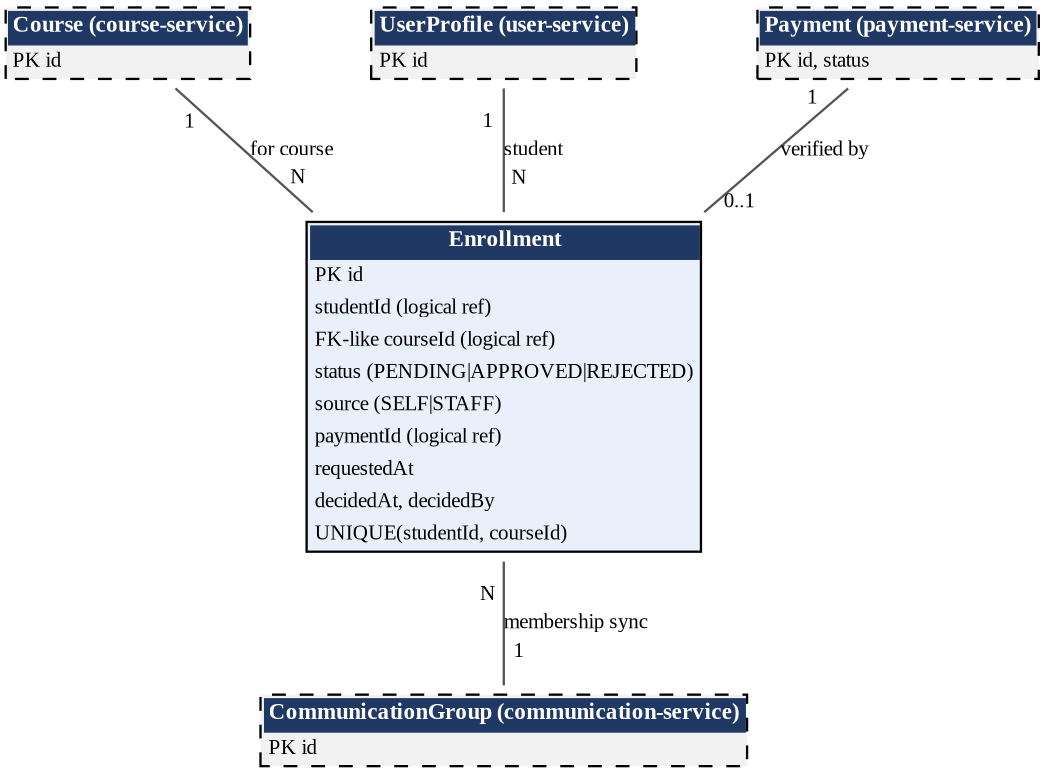


Figure 2.4: Entity-Relationship Diagram – Enrollment Service

2.1.0.5 Assignment Service (Assignments)

The Assignment Service governs assignments, rubrics, submissions, and grading. Assignment carries the course and module references (lesson optional), a title of at most 150 characters, start and due dates, maximum marks of at least 1, allowed attempts of at least 1, a late-submission flag, and a late-penalty percentage between 0 and 100. An assignment may hold up to 50 rubric items. A submission records its attempt number, lateness, and status, and is evaluated by a teacher who records marks and feedback.

2.1.0.6 Assignment Service (Quizzes)

The quiz model covers Quiz, QuizQuestion, QuizOption, QuizAttempt, and QuizAnswer. A quiz defines duration, pass mark, maximum attempts, optional start and end times, a result-visibility setting, a creation method (manual or AI draft), and a publication status. The only implemented question type is MULTIPLE_CHOICE_SINGLE_ANSWER; essay, short-answer, matching, and multiple-correct formats are not modelled. A manual creation request is limited to 100 questions.

2.1.0.7 Attendance Service

Attendance is owned by a dedicated service rather than by Analytics. AttendanceSession represents a taught session for a course; AttendanceRecord records each student's status for

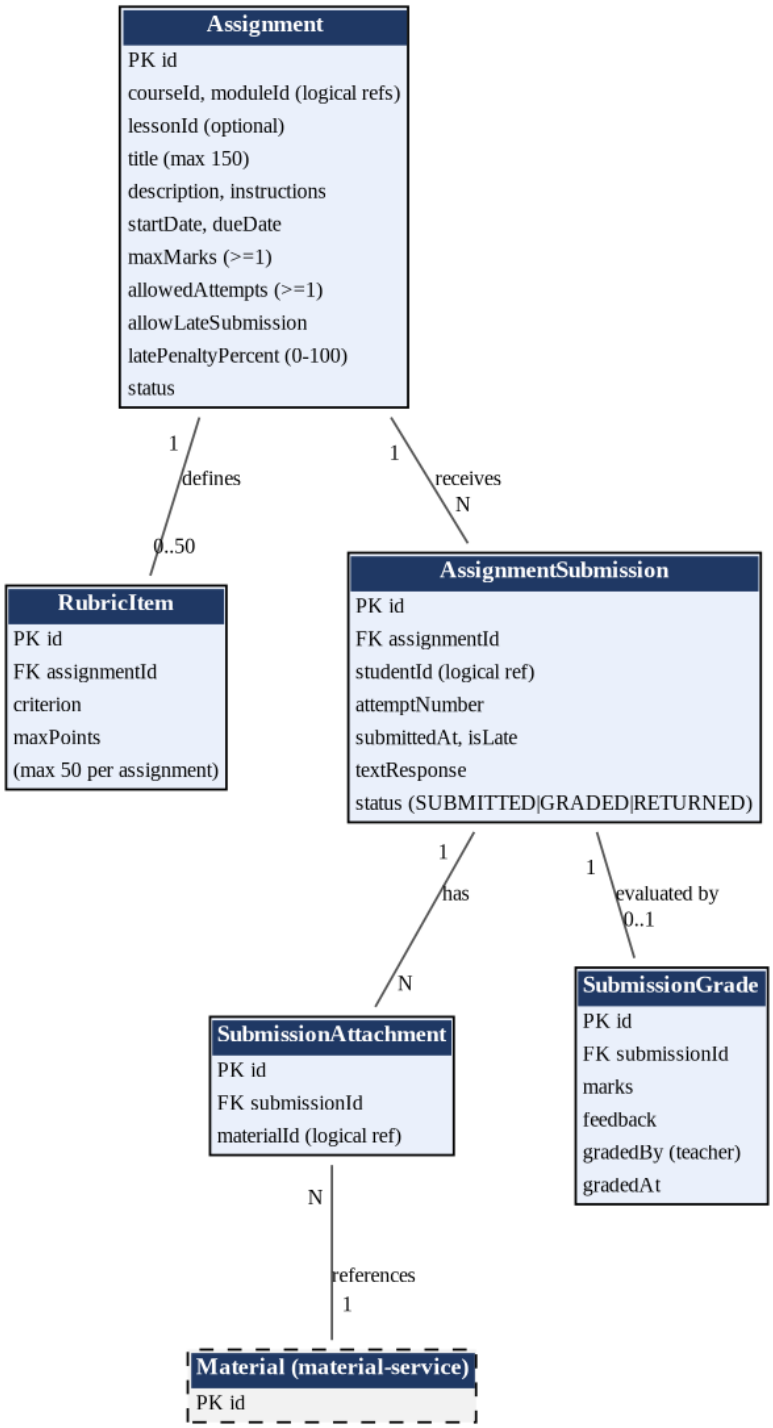


Figure 2.5: Entity-Relationship Diagram – Assignment Service (Assignments)

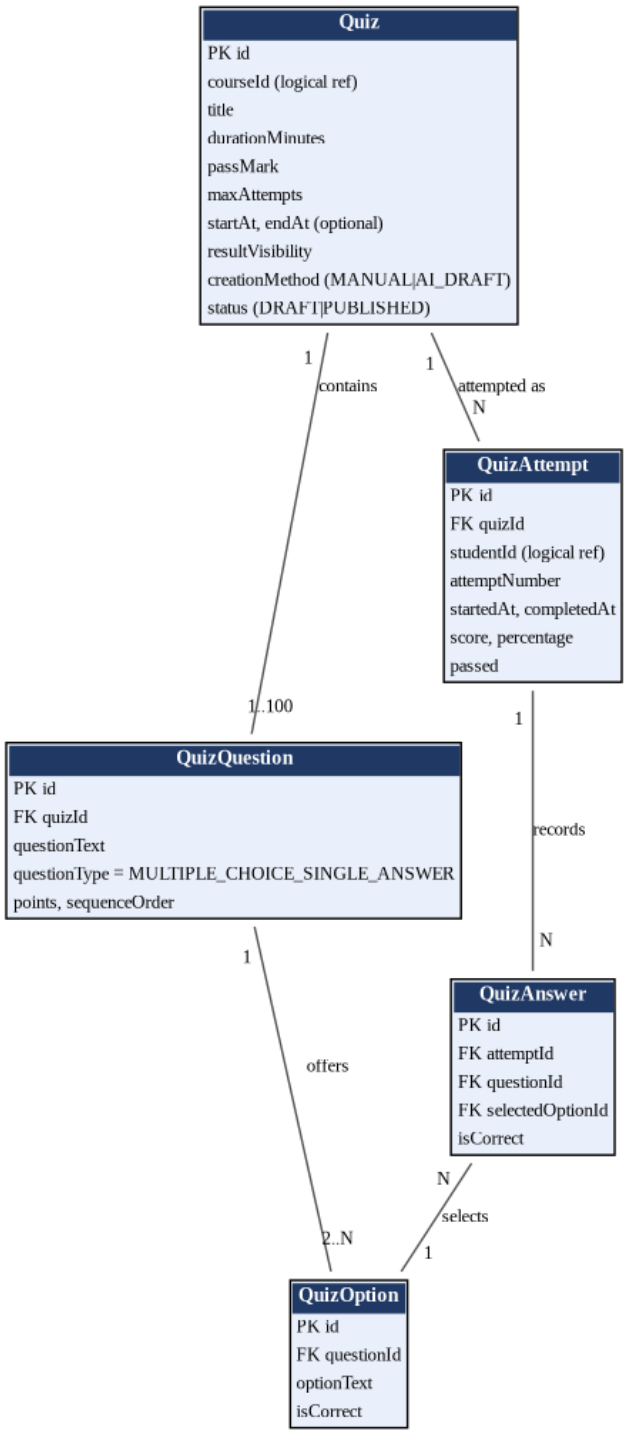


Figure 2.6: Entity-Relationship Diagram – Assignment Service (Quizzes)

a session and is unique per session and student. Attendance summaries are derived from the records, and sessions are projected to the calendar service.

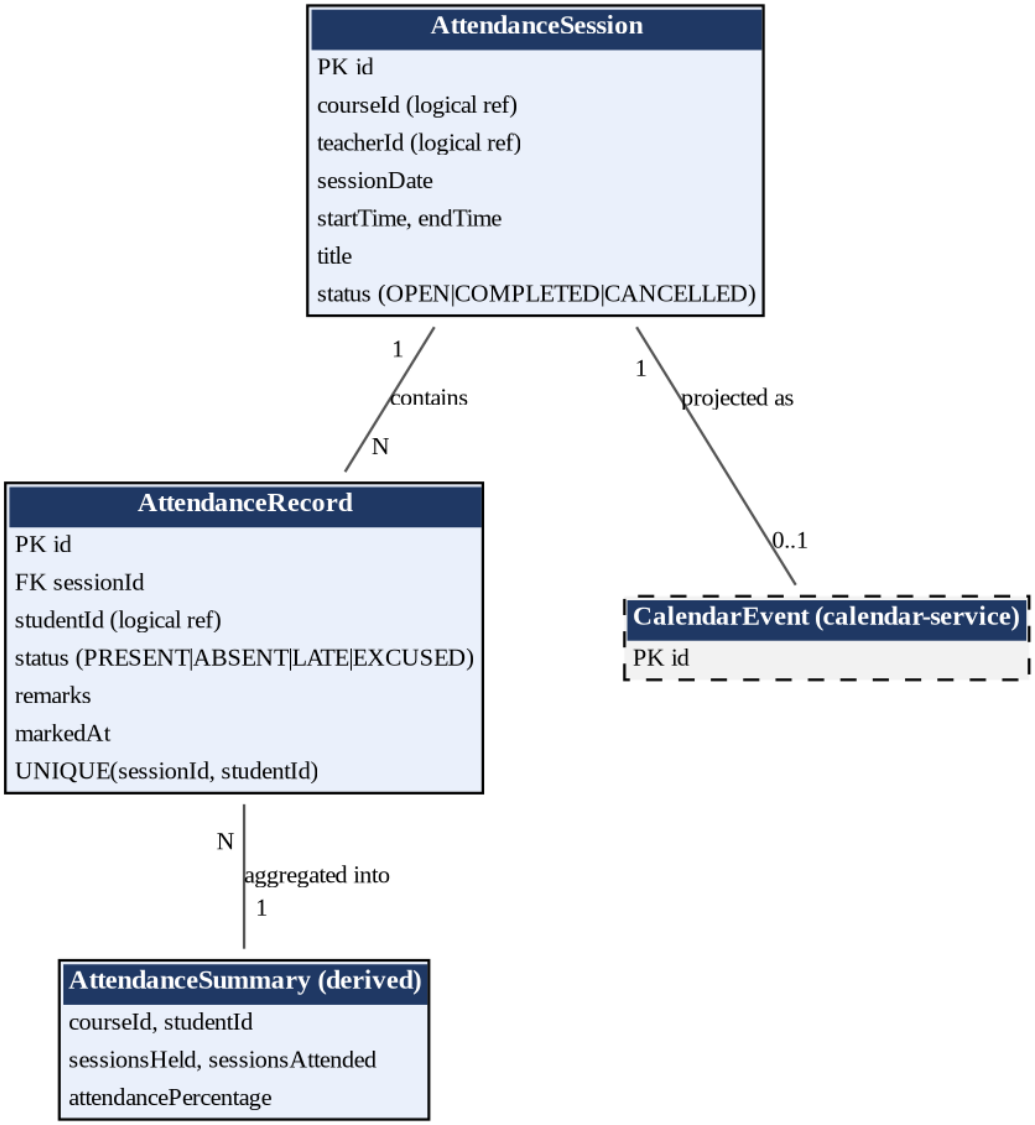


Figure 2.7: Entity-Relationship Diagram – Attendance Service

2.1.0.8 Exam Service

The Exam Service owns examinations of type MIDTERM, FINAL, PRACTICAL, or OTHER, their attachments, and per-student results. Results become visible to students according to a publication flag, and examination schedules are synchronised to the calendar service.

2.1.0.9 Material Service

The Material Service owns resource metadata, associations to courses, modules, and lessons, audit events, and the marketplace product catalogue. File content resides in S3-compatible

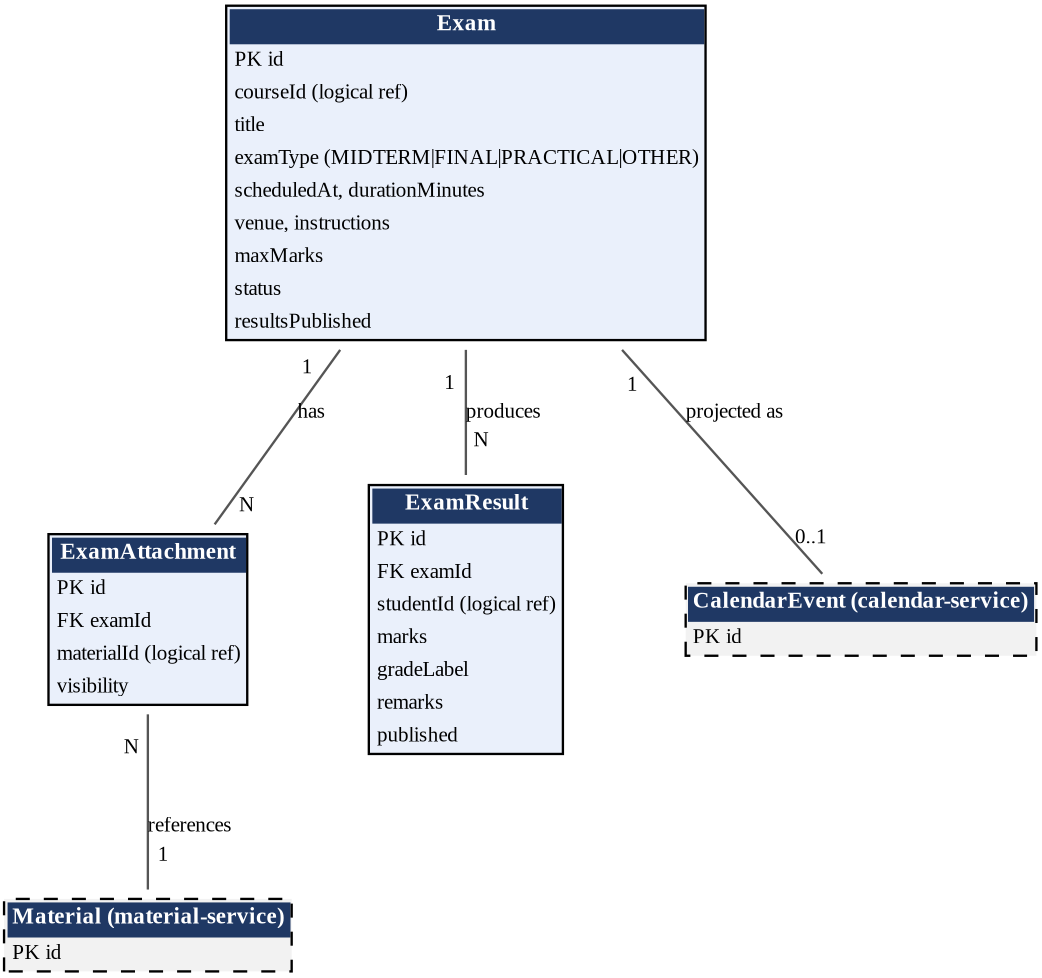


Figure 2.8: Entity-Relationship Diagram – Exam Service

object storage and is referenced through an object key. Material purposes include learning materials, assignment attachments, submission attachments, examination attachments, AI sources, announcement attachments, communication attachments, marketplace products, and other resources. MarketplaceProduct records title and descriptions, category, subject, grade association, price, free flag, discount percentage, currency, publication status, and thumbnail.

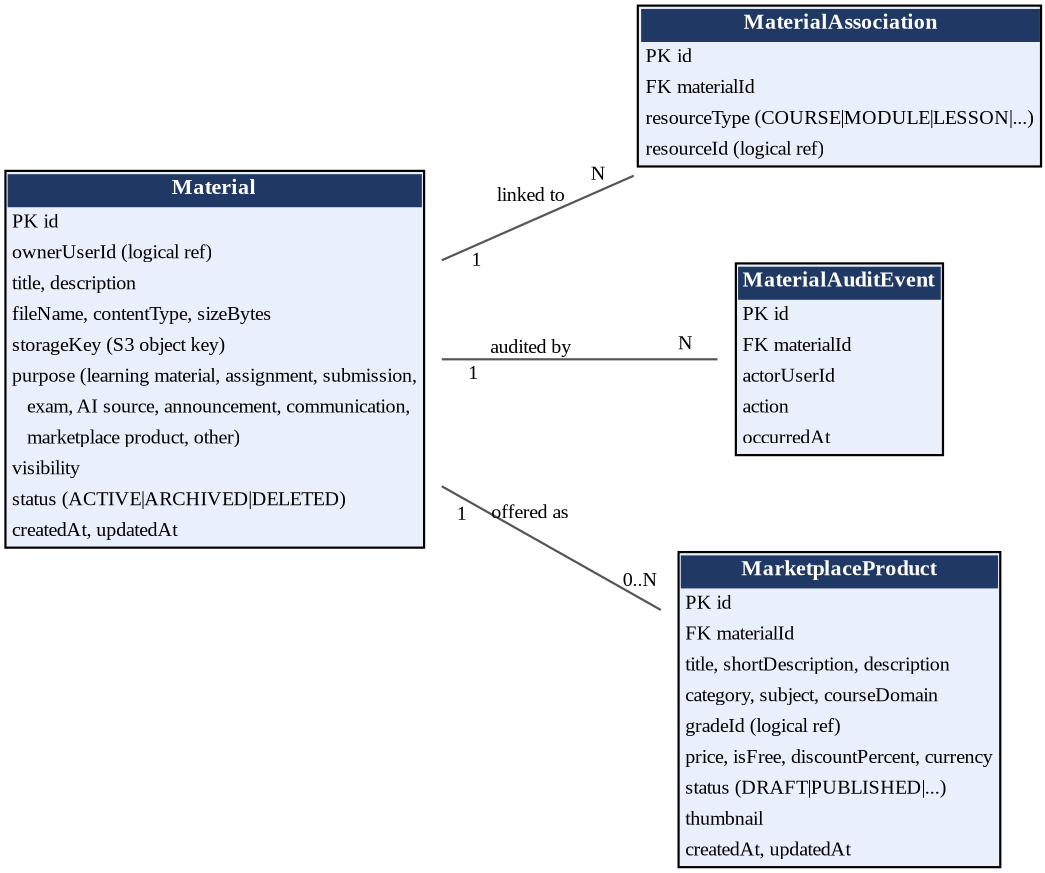


Figure 2.9: Entity-Relationship Diagram – Material Service

2.1.0.10 Payment Service

The Payment Service owns fee configuration and the financial lifecycle. GradeFee holds the monthly fee configured per grade. Payment records a Stripe Payment Intent for a student and course with a status of pending, success, failed, or refunded. Refund records refunds and WebhookEvent stores the identifiers of processed Stripe events to prevent duplicate processing. A monthly fee value is configuration data and does not itself establish recurring billing.

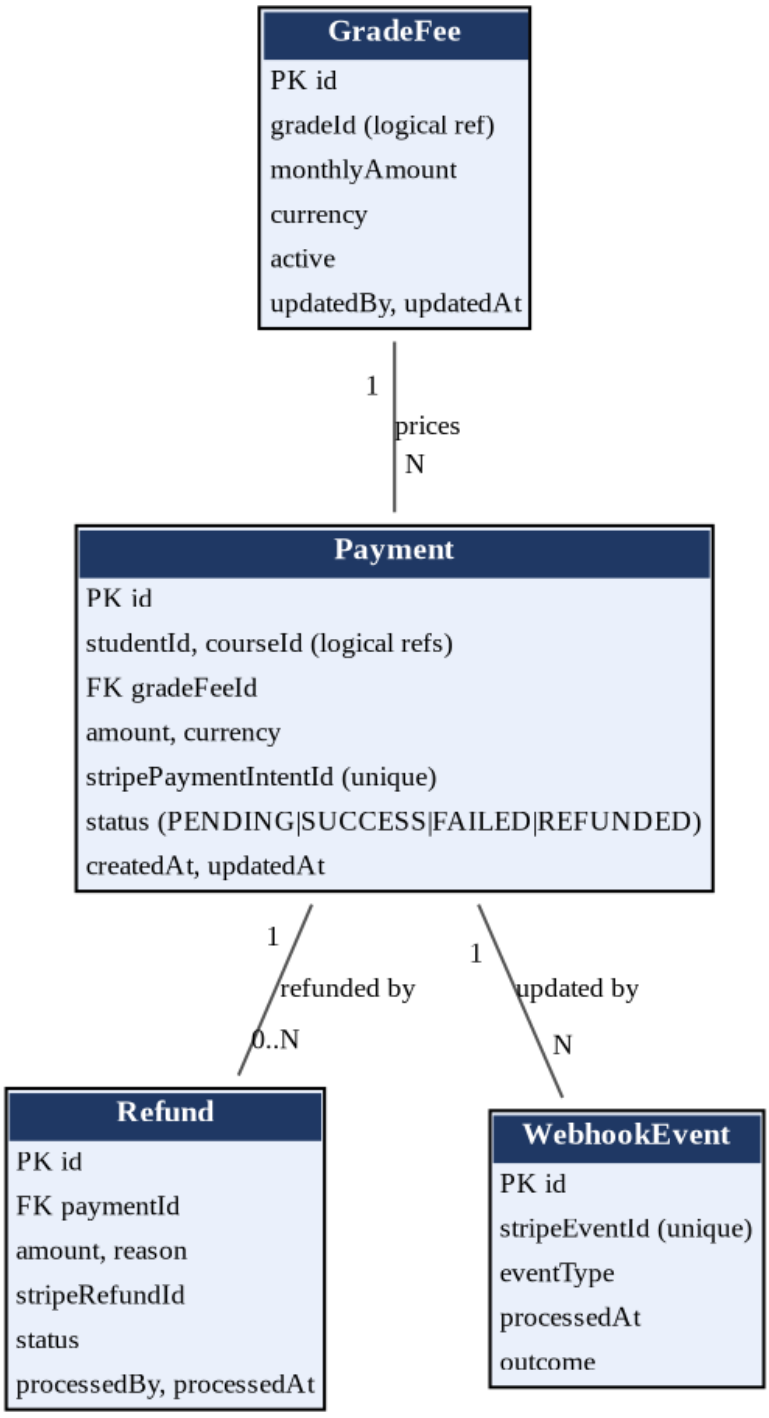


Figure 2.10: Entity-Relationship Diagram – Payment Service

2.1.0.11 Communication Service

The Communication Service is separate from the Notification Service. It holds communication groups, group membership, messages with attachments, announcements, and communication settings. Course group membership follows enrollment status, with strict and non-strict synchronisation paths in the enrollment workflow.

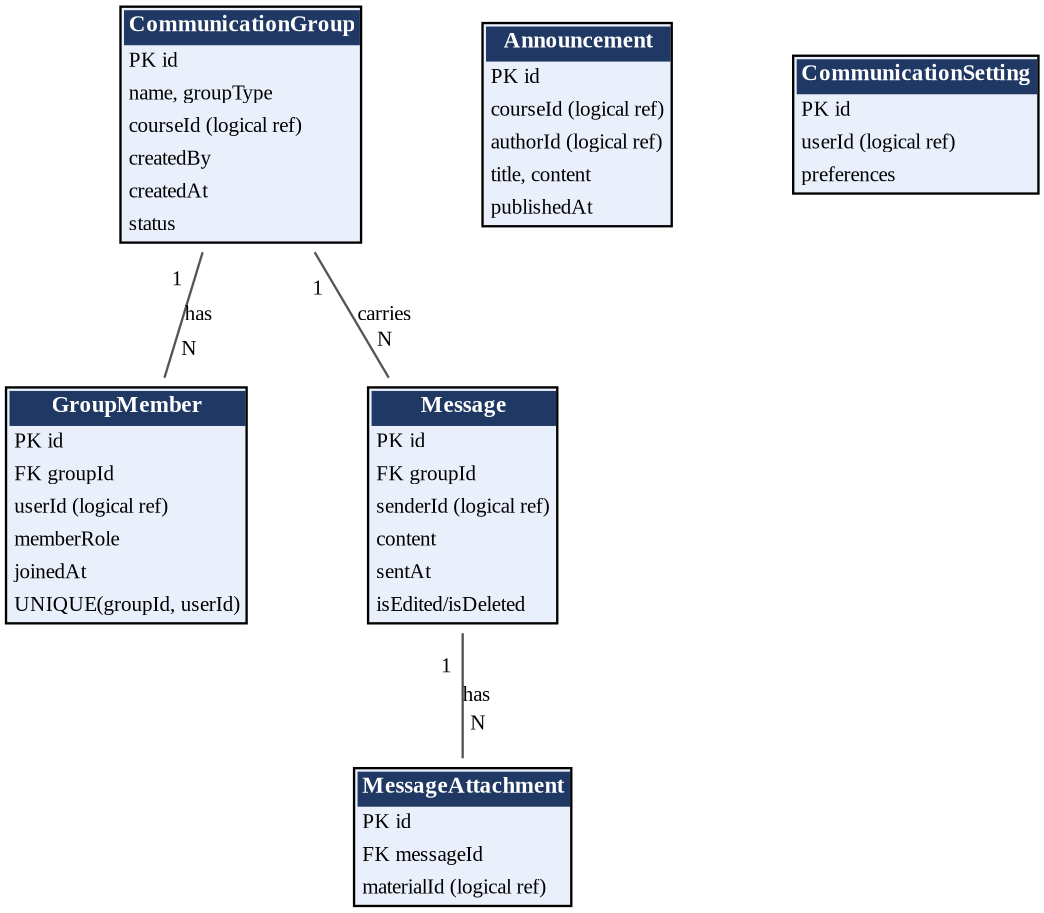


Figure 2.11: Entity-Relationship Diagram – Communication Service

2.1.0.12 Notification Service

The Notification Service stores notifications created from consumed Kafka events and the delivery attempts made through supported channels (in-app and e-mail).

2.1.0.13 Calendar Service

The Calendar Service stores calendar events and venues. A manually created event is distinguished from a projected event, which carries the type and identifier of its source (exam, assignment, quiz, or attendance session) so that projections can be upserted and reconciled.

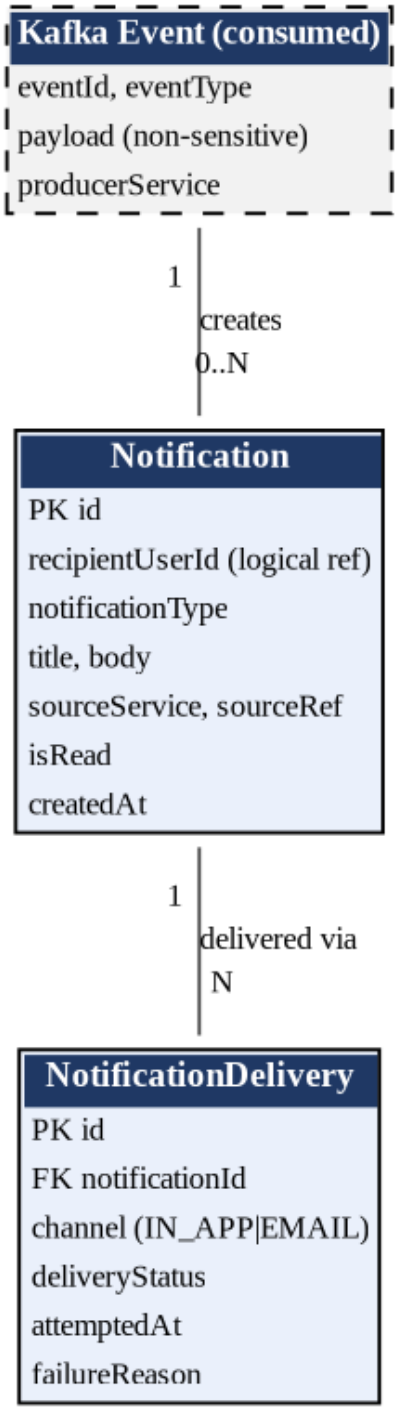


Figure 2.12: Entity-Relationship Diagram – Notification Service

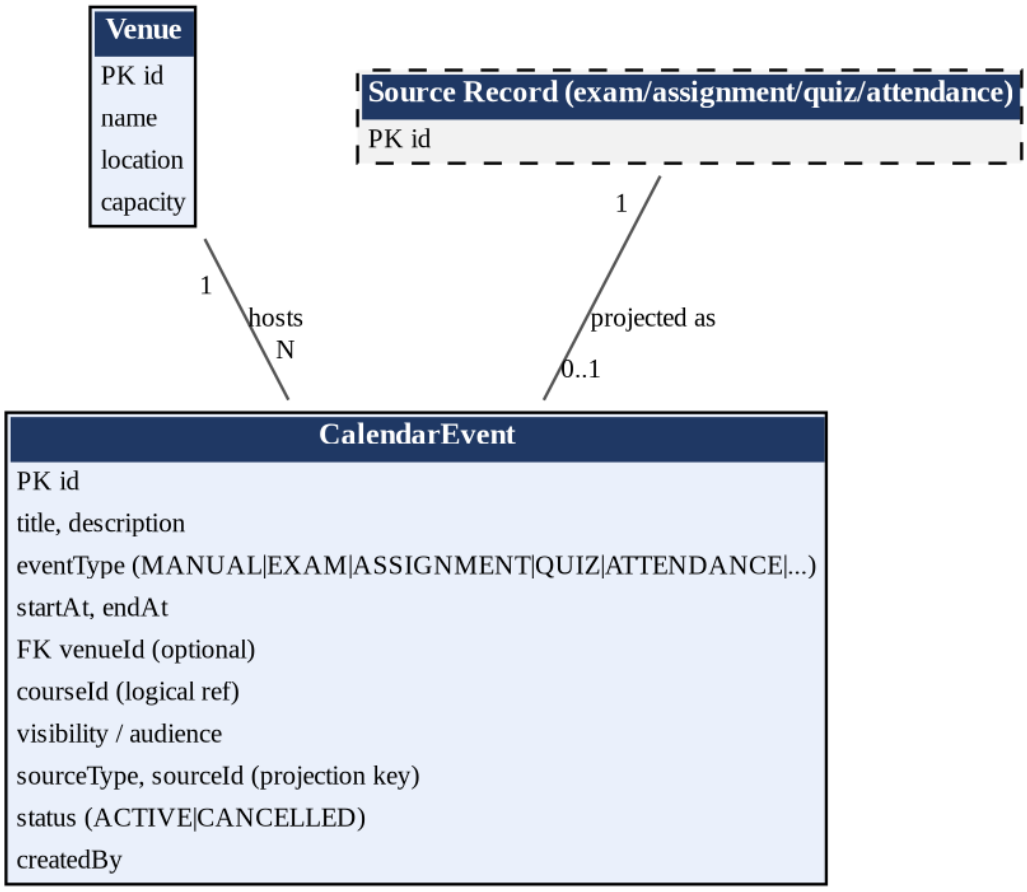


Figure 2.13: Entity-Relationship Diagram – Calendar Service

2.1.0.14 AI Service

The AI Service persists AiConversation, AiMessage, and AiAuditEvent. Version 1.0 entities AIRecommendation, AIInsightReport, AIGeneratedContent, AIChatSession, and AIChatMessage are not implemented as physical tables and are removed. Generated answers and quiz drafts are returned to the caller and, for quizzes, persisted by the Assignment Service only after teacher review.

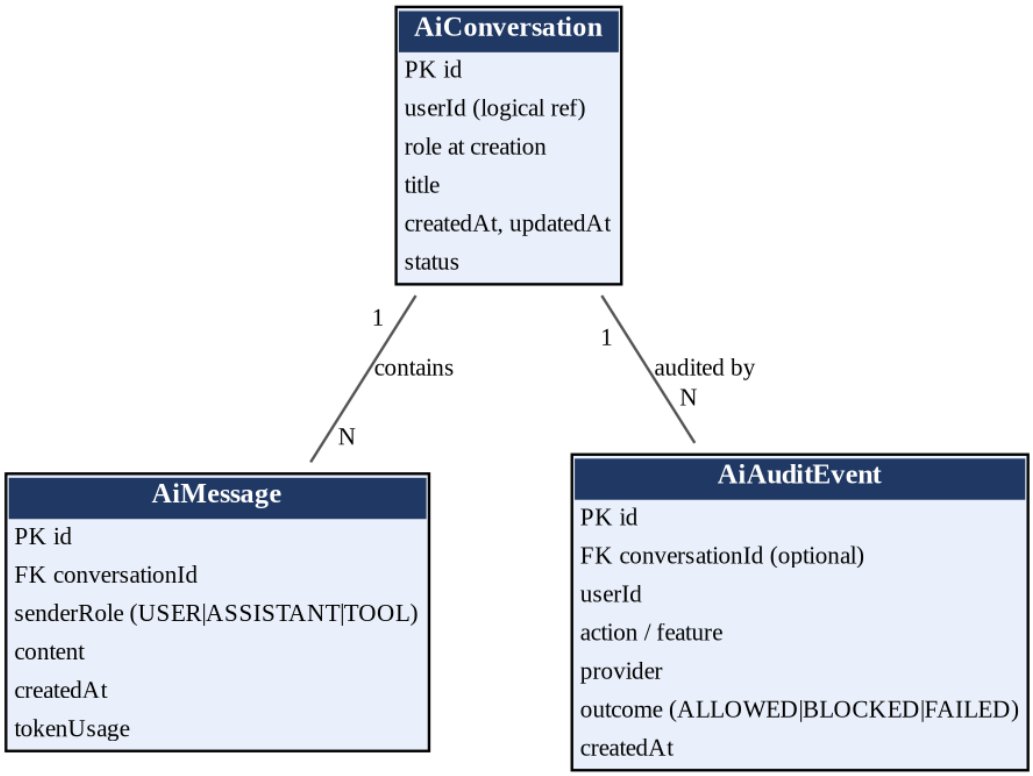


Figure 2.14: Entity-Relationship Diagram – AI Service

2.1.0.15 Analytics Service

The Analytics Service does not own attendance, progress, or engagement records, as Version 1.0 assumed. It aggregates data from the source services through Feign clients for users, courses, enrollment, assessments, attendance, and payments, and may cache results temporarily. The persisted schema of the Analytics Service is limited to what its own migrations define; its main design is the data flow shown in Figure 17. The Version 1.0 entities AttendanceRecord, ProgressReport, PerformanceAnalytics, and EngagementMetric are therefore removed from the physical design.

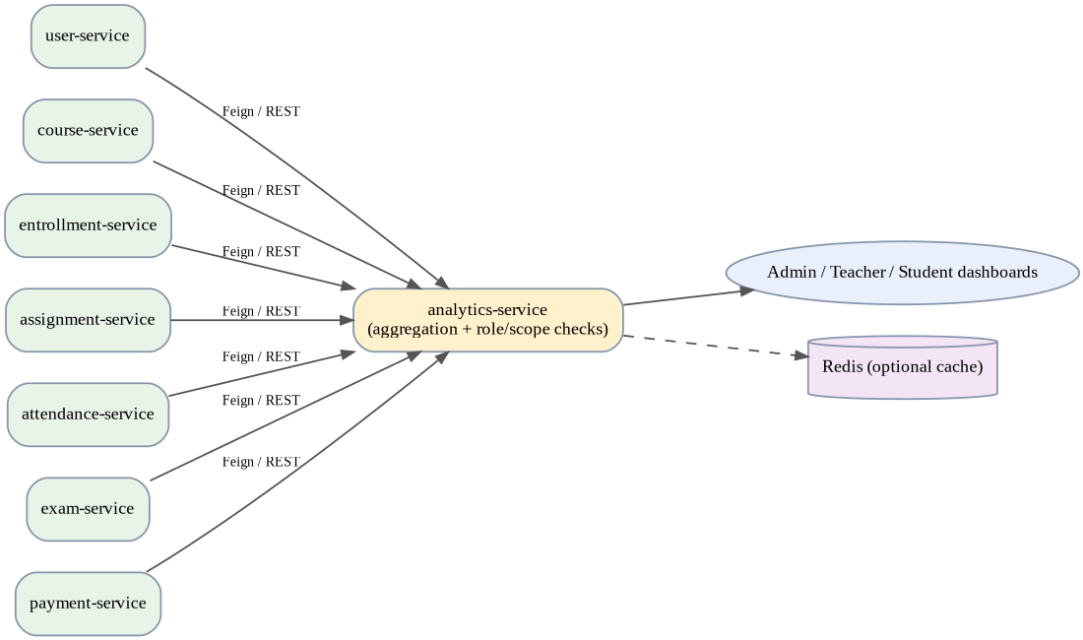


Figure 2.15: Analytics Service Data Flow (Derived Data)

2.1.0.16 Data Ownership and Cross-Service References

The ownership matrix below identifies the responsible service for each data domain and the logical references through which services relate to each other.

Table 2.1: Data Ownership Matrix

Service	Principal Tables / Entities	Logical References to Other Services
auth-service	AuthUser, RefreshToken, PasswordResetOtp	UserProfile (authUserId)
user-service	UserProfile, StudentProfile, TeacherProfile, AdminProfile	AuthUser, Grade
course-service	Grade, Course, CourseTeachingAssistant, Module, Lesson	Teacher (user-service)
entrollment-service	Enrollment	Student, Course, Payment, CommunicationGroup

assignment-service	Assignment, RubricItem, AssignmentSubmission, SubmissionAttachment, SubmissionGrade, Quiz, QuizQuestion, QuizOption, QuizAttempt, QuizAnswer	Course, Module, Lesson, Student, Material
attendance-service	AttendanceSession, AttendanceRecord	Course, Teacher, Student, CalendarEvent
exam-service	Exam, ExamAttachment, ExamResult	Course, Student, Material, CalendarEvent
material-service	Material, MaterialAssociation, MaterialAuditEvent, MarketplaceProduct	Owner user, Course/Module/Lesson, Grade
payment-service	GradeFee, Payment, Refund, WebhookEvent	Grade, Student, Course
communication-service	CommunicationGroup, GroupMember, Message, MessageAttachment, Announcement, CommunicationSetting	Course, User, Material
notification-service	Notification, NotificationDelivery	Recipient user, source service reference
calender-service	CalendarEvent, Venue	Course, source records
analytics-service	Derived aggregates (no ownership of source data)	All source services
ai-service	AiConversation, AiMessage, AiAuditEvent	User

Three distributed-data rules follow from this ownership. First, enrollment status controls access to course materials, quizzes, and communication groups, and each consuming service verifies it through the Enrollment Service. Second, a successful payment recorded by the Payment Service is required before the Enrollment Service accepts a self-enrollment in a paid course. Third, academic actions in the exam, attendance, assignment, and quiz services create calendar projections and notifications through the calendar and notification services.

2.2 Normalisation Analysis

The relational schemas derived from the logical models in Section 2.1 are analysed table by table. Version 1.0 asserted that every relation was in Third Normal Form (3NF); that blanket claim is replaced by the analysis below, which must be confirmed against the Flyway migrations and which records intentional denormalisation explicitly.

2.2.0.1 Definitions

First Normal Form (1NF) requires that all attribute values are atomic (i.e., indivisible) and that each column contains values of a single type. Second Normal Form (2NF) requires that 1NF is satisfied and that every non-key attribute is fully functionally dependent on the entire primary key (applicable only to relations with composite keys). Third Normal Form (3NF) requires that 2NF is satisfied and that no non-key attribute is transitively dependent on the primary key through another non-key attribute.

2.2.0.2 Auth and User Services Normalisation

All tables use a single-attribute surrogate key. AuthUser stores credentials only; profile attributes are held in UserProfile, so no profile attribute depends on the credential record. Role-specific profile tables use the profile identifier as their primary key.

Table 2.2: Auth and User Services – Normalisation Summary

Table	Key / Constraints	Normalisation Note
AuthUser	PK id; UNIQUE email	Attributes depend on id only; role and status are single-valued.
RefreshToken	PK id; userId reference	Token facts depend on the token id; 3NF.
PasswordResetOtp	PK id; userId reference	OTP facts depend on the record; 3NF.
UserProfile	PK id; UNIQUE authUserId	Name, contact, role, and status depend on id; 3NF.
StudentProfile / TeacherProfile / AdminProfile	PK = userId	Only role-specific attributes; no transitive dependencies; 3NF.

2.2.0.3 Course and Enrollment Services Normalisation

Course attributes depend on the course identifier. The grade is referenced by gradeId rather than copying grade name and level into Course. The enrollment relation uses a surrogate

key with UNIQUE(studentId, courseId); status and timestamps depend on the enrollment record only.

Table 2.3: Course and Enrollment Services – Normalisation Summary

Table	Key / Constraints	Normalisation Note
Grade	PK id	name, level, status depend on id; 3NF.
Course	PK id; UNIQUE courseCode; gradeId	No transitive dependency via gradeId or teacherId; 3NF.
Module	PK id; courseId	title, objectives, order depend on id; 3NF.
Lesson	PK id; moduleId	Content and status depend on id; 3NF.
Enrollment	PK id; UNIQUE(studentId, courseId)	status, source, paymentId, decision data depend on id; 3NF.

2.2.0.4 Assignment Service Normalisation

Assignments, submissions, grades, rubric items, quizzes, questions, options, attempts, and answers each use a surrogate key. Child tables hold the parent key and describe only their own facts. The stored attemptNumber is a recorded fact per submission or attempt, not a derived count.

Table 2.4: Assignment Service – Normalisation Summary

Table	Key / Constraints	Normalisation Note
Assignment	PK id	Policy attributes (marks, attempts, late rules) depend on id; 3NF.
RubricItem	PK id; assignmentId	criterion, maxPoints depend on id; 3NF.
AssignmentSubmission	PK id; assignmentId, studentId	Attempt, lateness, status depend on id; isLate is stored at submission time.
SubmissionGrade	PK id; submissionId	marks, feedback, gradedBy depend on id; 3NF.
Quiz / QuizQuestion / QuizOption	PK id each	Descriptive attributes depend on their own key; 3NF.

QuizAttempt / QuizAnswer	PK id each	score, percentage, passed are stored results; isCorrect recorded per answer (intentional snapshot).

2.2.0.5 Attendance and Exam Services Normalisation

Sessions, records, exams, attachments, and results are separate relations keyed by surrogate identifiers. Attendance summaries are derived at query time and are not persisted, so no derived value is stored redundantly.

Table 2.5: Attendance and Exam Services – Normalisation Summary

Table	Key / Constraints	Normalisation Note
AttendanceSession	PK id; courseId	Date, time, status depend on id; 3NF.
AttendanceRecord	PK id; UNIQUE(sessionId, studentId)	status, remarks, markedAt depend on id; 3NF.
Exam	PK id; courseId	Type, schedule, maxMarks, publication flag depend on id; 3NF.
ExamAttachment	PK id; examId	Attachment reference only; 3NF.
ExamResult	PK id; examId, studentId	marks and gradeLabel depend on id; gradeLabel is a stored result.

2.2.0.6 Material and Payment Services Normalisation

Material metadata is separated from associations so that one resource can serve several purposes. MarketplaceProduct references Material by identifier. Payment stores the amount and currency charged as a point-in-time snapshot, which is intentionally denormalised relative to GradeFee so that later fee changes do not alter historical payments.

Table 2.6: Material and Payment Services – Normalisation Summary

Table	Key / Constraints	Normalisation Note
Material	PK id; storageKey	File facts and purpose depend on id; 3NF.

MaterialAssociation	PK id; materialId	resourceType and resourceId describe the link; 3NF.
MarketplaceProduct	PK id; materialId	Pricing and status depend on id; finalPrice is calculated, not stored.
GradeFee	PK id; gradeId	monthlyAmount, currency depend on id; 3NF.
Payment	PK id; UNIQUE stripePaymentIntentId	amount, currency are intentional snapshots (documented denormalisation).
Refund / WebhookEvent	PK id; UNIQUE stripeEventId	Facts depend on id; 3NF.

2.2.0.7 Communication, Notification, Calendar, and AI Services Normalisation

Group membership uses a surrogate key with UNIQUE(groupId, userId). Calendar projections copy the title and time of the source record by design, so a projection is an intentional, reconcilable denormalised copy identified by (sourceType, sourceId). AI conversation data is split into conversations, messages, and audit events.

Table 2.7: Communication, Notification, Calendar, and AI Services – Normalisation Summary

Table	Key / Constraints	Normalisation Note
CommunicationGroup / GroupMember	PK id; UNIQUE(groupId, userId)	Membership facts depend on the record; 3NF.
Message / MessageAttachment	PK id; groupId / messageId	Content and sent time depend on id; 3NF.
Announcement	PK id; courseId, authorId	Title, content, publishedAt depend on id; 3NF.
Notification / NotificationDelivery	PK id; notificationId	Delivery status is separated from the notification; 3NF.
CalendarEvent / Venue	PK id; (sourceType, sourceId)	Projected title and time are intentional copies of source data.
AiConversation / AiMessage / AiAuditEvent	PK id; conversationId	Each depends on its own key; 3NF.

Cached aggregates held by the Analytics Service, where present, are derived and are not part of the normalised source schemas.

Chapter 3

Class Diagram

This chapter presents the class diagrams of the system. Version 1.0 contained seven diagrams for six services; the diagrams are redrawn per business domain because the backend now has fourteen services. Each diagram shows the layers used throughout the codebase: the REST controller, the service interface and its implementation, the Spring Data repositories, the JPA entities and enumerations, the request and response DTOs, and the Feign clients used to reach other services. Cross-service entities are referenced by identifier only and are therefore not drawn as associations. Class names that are evidenced in the repository (for example CourseRequest, EntrollmentServiceImpl, GradingServiceImpl, QuizGradingServiceImpl, PaymentServiceImpl, AiChatServiceImpl, MarketplaceProductServiceImpl, Role, and QuestionType) are used as they appear; other names follow the same Controller, Service, ServiceImpl, Repository naming convention and must be reconciled with the source before final submission.

3.1 Auth Service

The Auth Service implements authentication and session management. AuthController exposes login, refresh, logout, password recovery, password change, and the public JWKS endpoint. AuthServiceImpl verifies credentials, issues signed access and refresh tokens through a token provider, manages one-time passwords, and calls the User Service client to link identities with profiles. The Role enumeration defines STUDENT, TEACHER, ADMIN, and SUPER_ADMIN; the SupportAgent role of Version 1.0 is removed.

3.2 User Service

The User Service manages profiles, registration, invitations, account status, and deletion. Registration of administrator accounts is restricted to the SUPER_ADMIN role. The abstract User base class and four-subtype inheritance of Version 1.0 are replaced by a User-

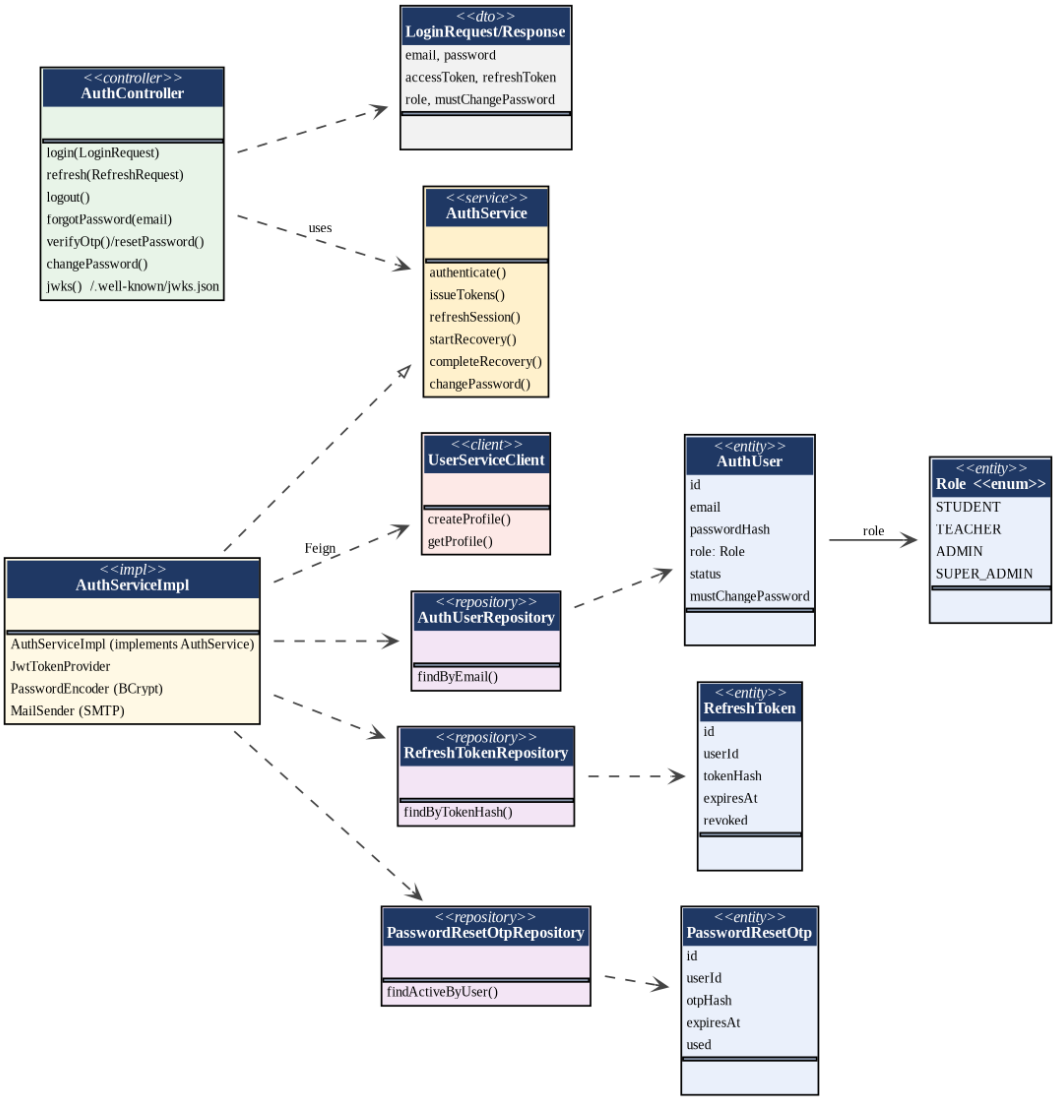


Figure 3.1: Class Diagram – Auth Service

Profile with role-specific profile entities linked by the authentication identifier.

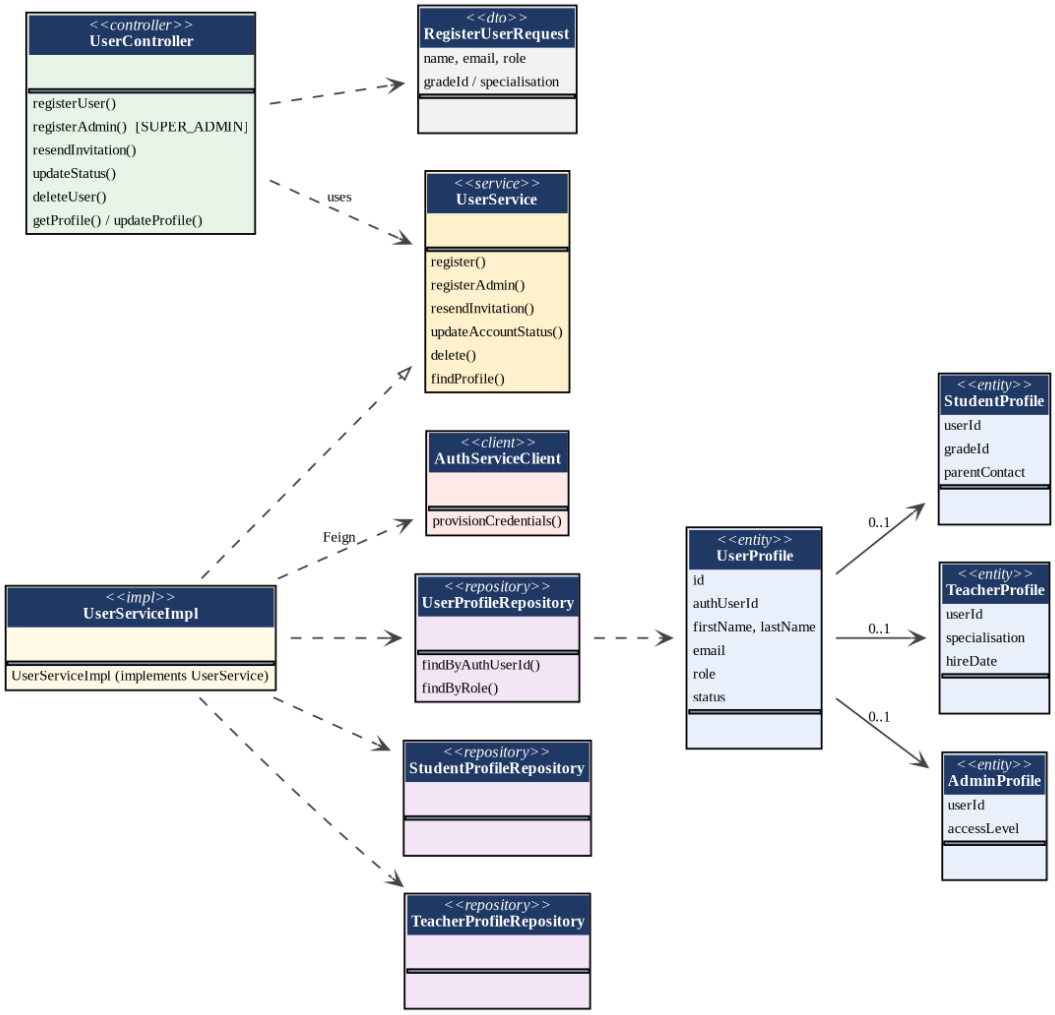


Figure 3.2: Class Diagram – User Service

3.3 Course Service

The Course Service governs grades, courses, modules, and lessons. CourseRequest validates a required name, description, grade, assigned teacher, and subject, and carries optional academic metadata. Course deletion is restricted to the Admin and Super Admin roles, whereas an assigned teacher may manage course content.

3.4 Enrollment Service

The Enrollment Service separates student self-enrollment from staff-created enrollment. EntrollmentServiceImpl validates the course and duplicates, verifies successful payment for paid courses through the Payment client, saves the enrollment with status PENDING,

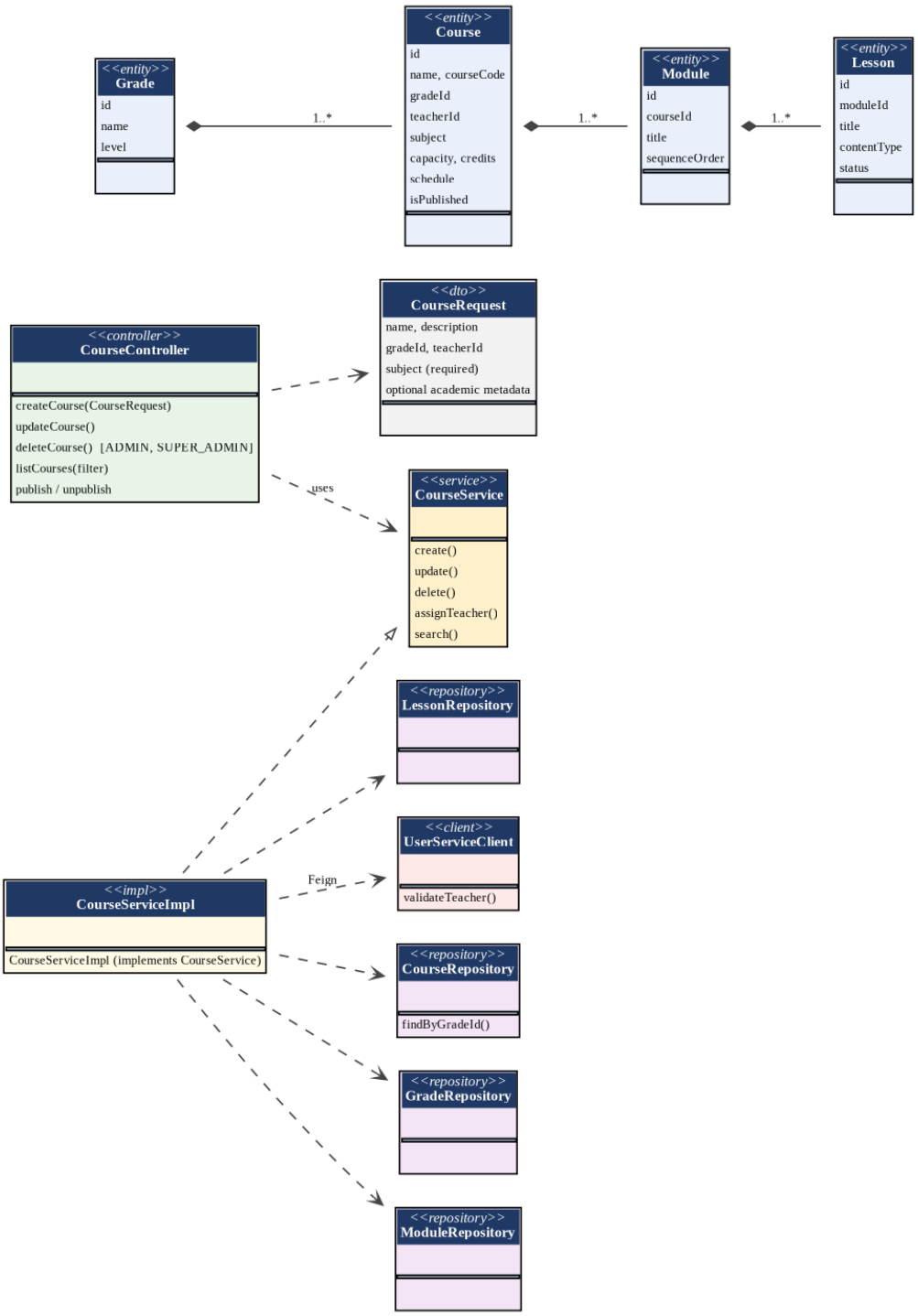


Figure 3.3: Class Diagram – Course Service

and applies the EnrollmentStatus transitions (PENDING, APPROVED, REJECTED) under teacher or administrator authorisation. Communication-group synchronisation uses strict and non-strict handling paths.

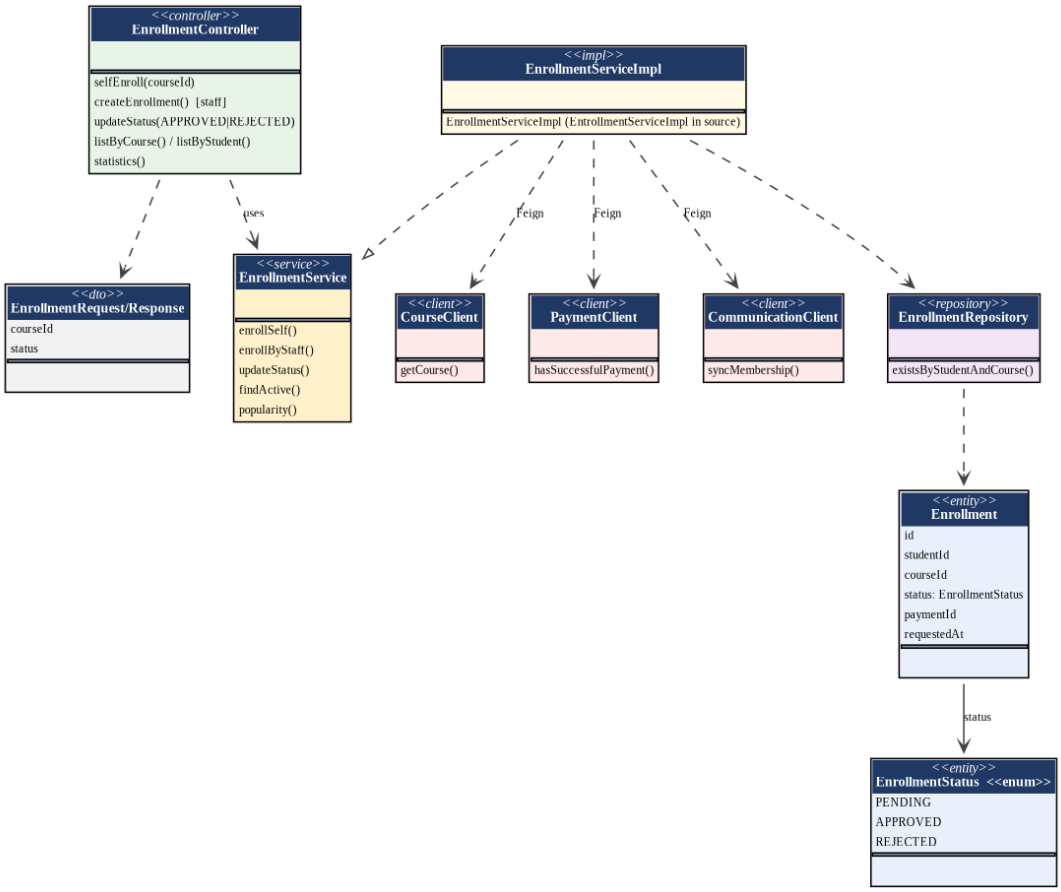


Figure 3.4: Class Diagram – Enrollment Service

3.5 Assignment Service

The Assignment Service covers both assignments and quizzes. GradingServiceImpl checks teacher authorisation, records marks and feedback, and updates the submission status; it does not perform AI grading. QuizGradingServiceImpl scores single-answer multiple-choice attempts by comparing selected options with stored correct options. Quiz draft generation calls the AI Service through a client, and the teacher reviews the draft before publication.

3.6 Attendance Service

The Attendance Service manages attendance sessions, records, completion, summaries, and analytics, and notifies the Calendar Service so that sessions appear as projected events.

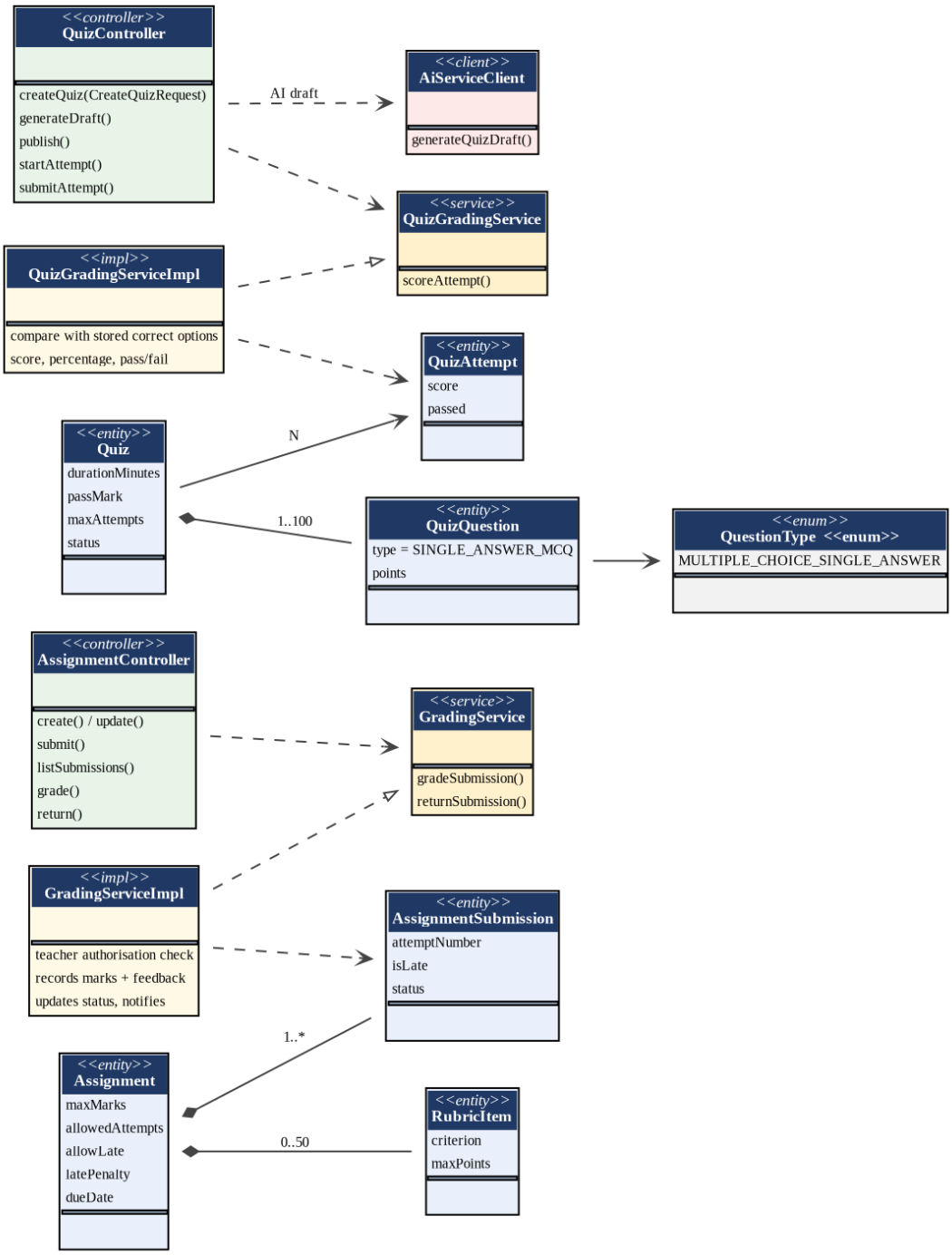


Figure 3.5: Class Diagram – Assignment Service

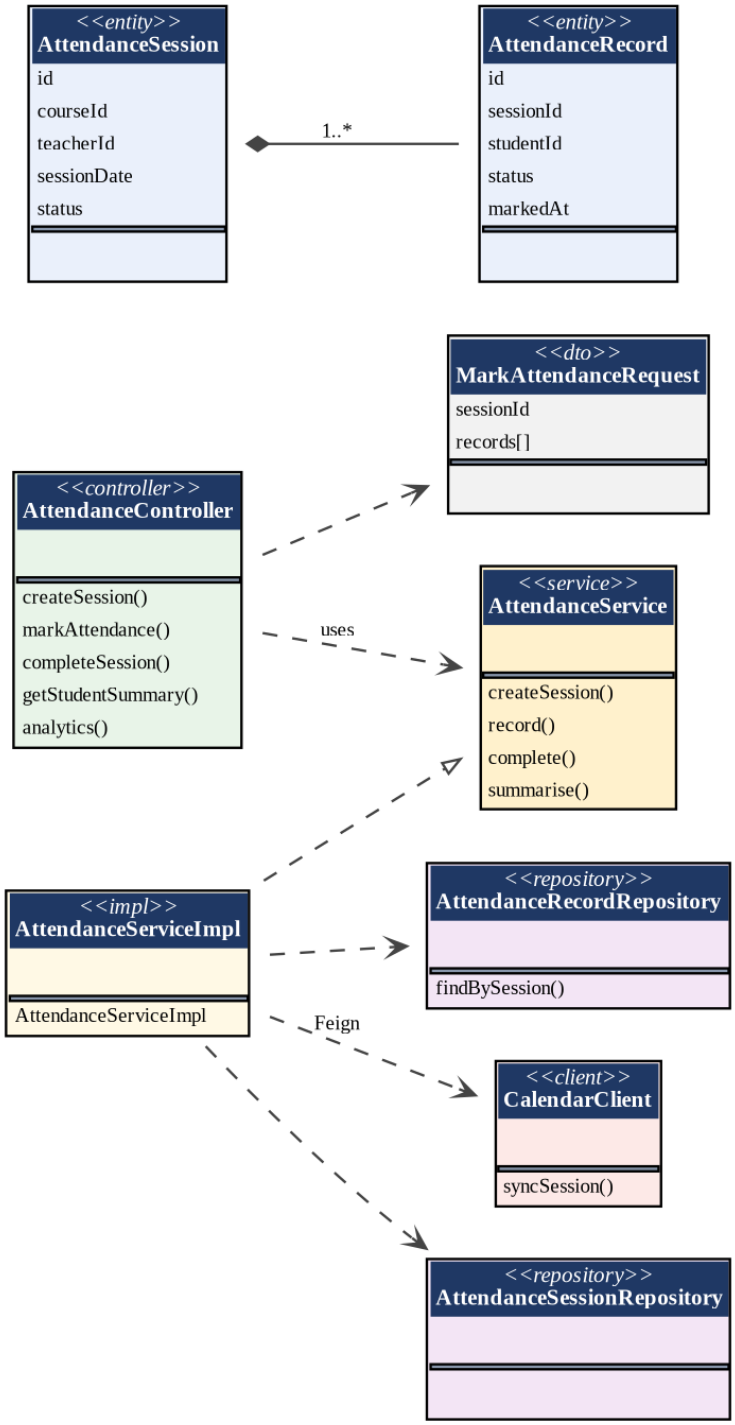


Figure 3.6: Class Diagram – Attendance Service

3.7 Exam Service

The Exam Service manages the examination lifecycle: creation and scheduling, attachments authorised through the Material Service, marks entry, result publication, student views, and calendar synchronisation. ExamType defines MIDTERM, FINAL, PRACTICAL, and OTHER.

3.8 Material Service and Marketplace Catalogue

The Material Service validates and stores uploads, associates materials with resources, authorises access, and issues time-limited presigned download URLs through the object-storage client. MarketplaceProductController provides administrator-only product management and a public listing of published products; MarketplaceProductServiceImpl applies the pricing and status rules. No checkout or purchase-entitlement classes exist.

3.9 Payment Service

PaymentController initiates course payments, returns payment status and history, supports administrator search and refunds, and receives Stripe webhooks. PaymentServiceImpl resolves the course fee, creates Stripe Payment Intents, verifies webhook signatures, records processed event identifiers to prevent duplicate processing, and manages refunds. PaymentStatus defines PENDING, SUCCESS, FAILED, and REFUNDED.

3.10 Communication Service

The Communication Service provides groups, membership, messages, announcements, and WebSocket/STOMP delivery. It is independent of the Notification Service: communication is user or group interaction, whereas a notification is a system-generated alert. The Version 1.0 PrivateMessage and DiscussionForum classes are not part of the reviewed implementation and are removed.

3.11 Notification Service

The Notification Service consumes Kafka events, creates notifications, records delivery attempts, and sends e-mail through the SMTP integration where configured. Duplicate-event handling is applied where implemented.

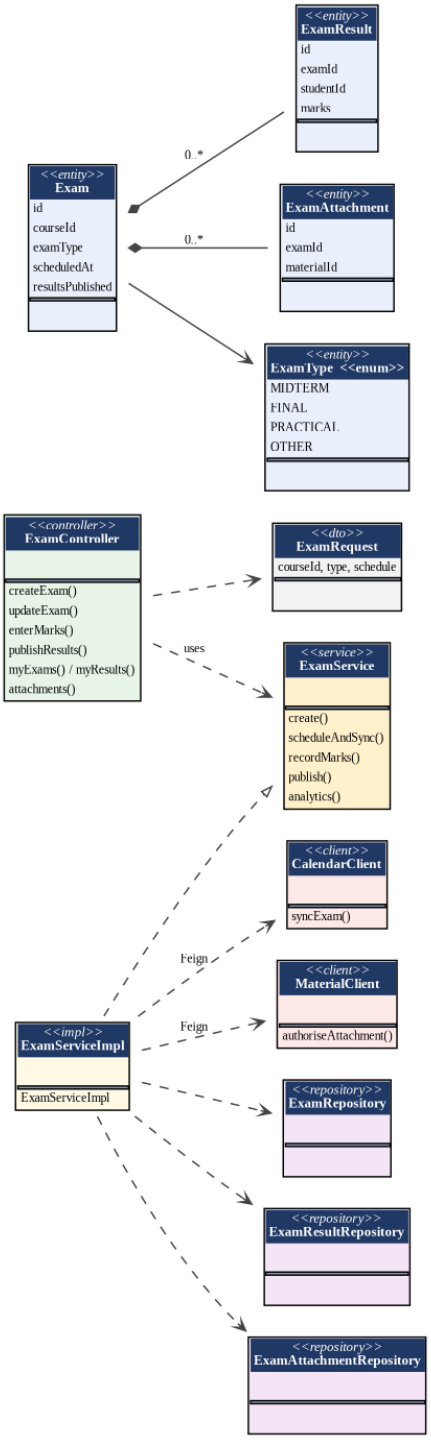


Figure 3.7: Class Diagram – Exam Service

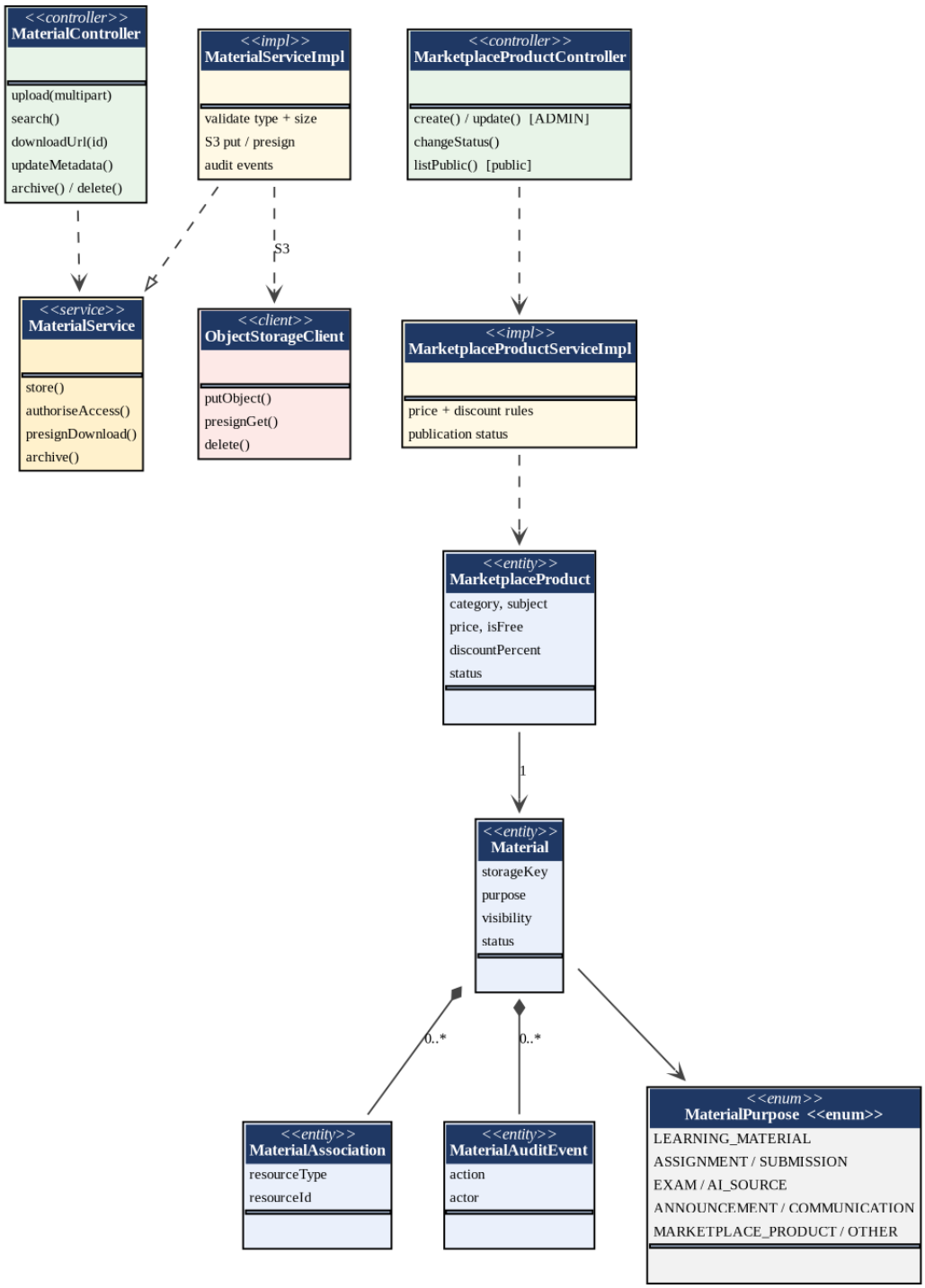


Figure 3.8: Class Diagram – Material Service and Marketplace Catalogue

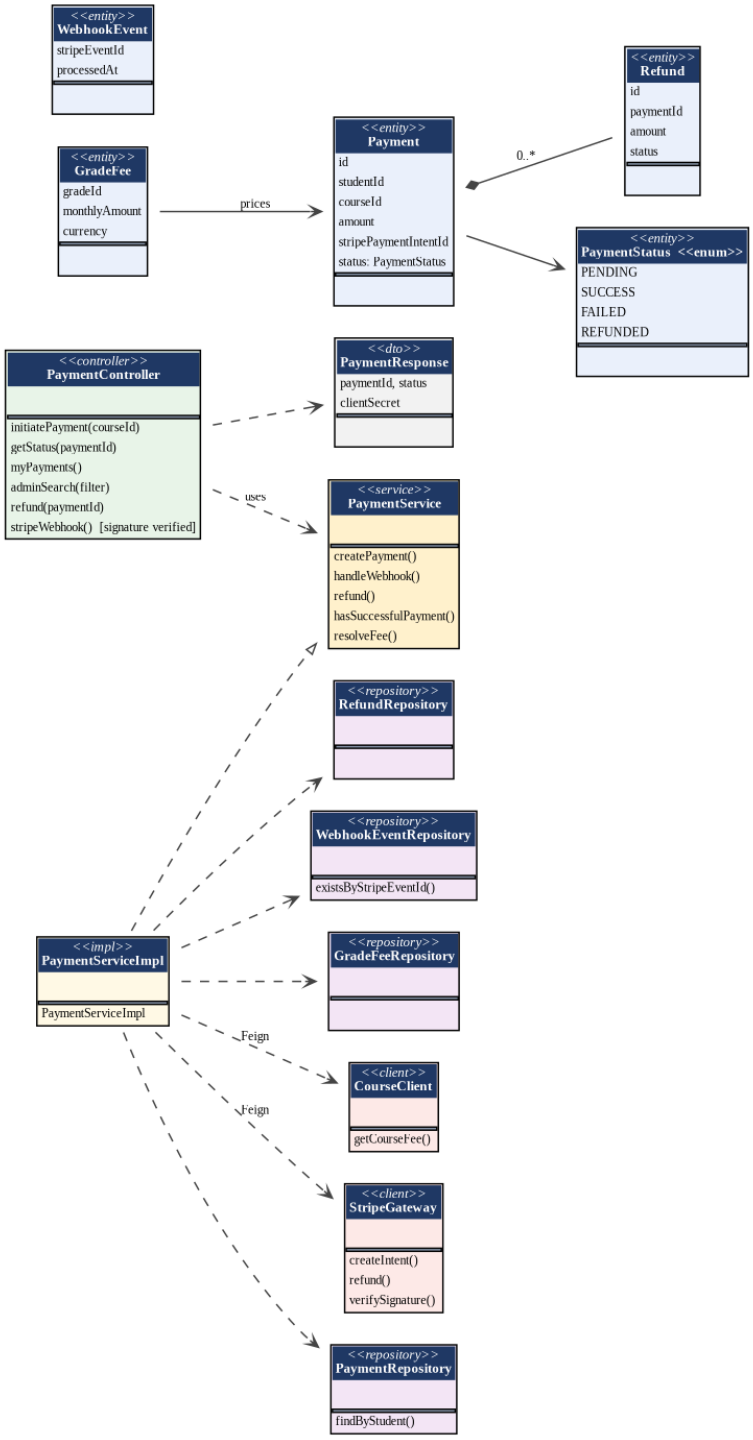


Figure 3.9: Class Diagram – Payment Service

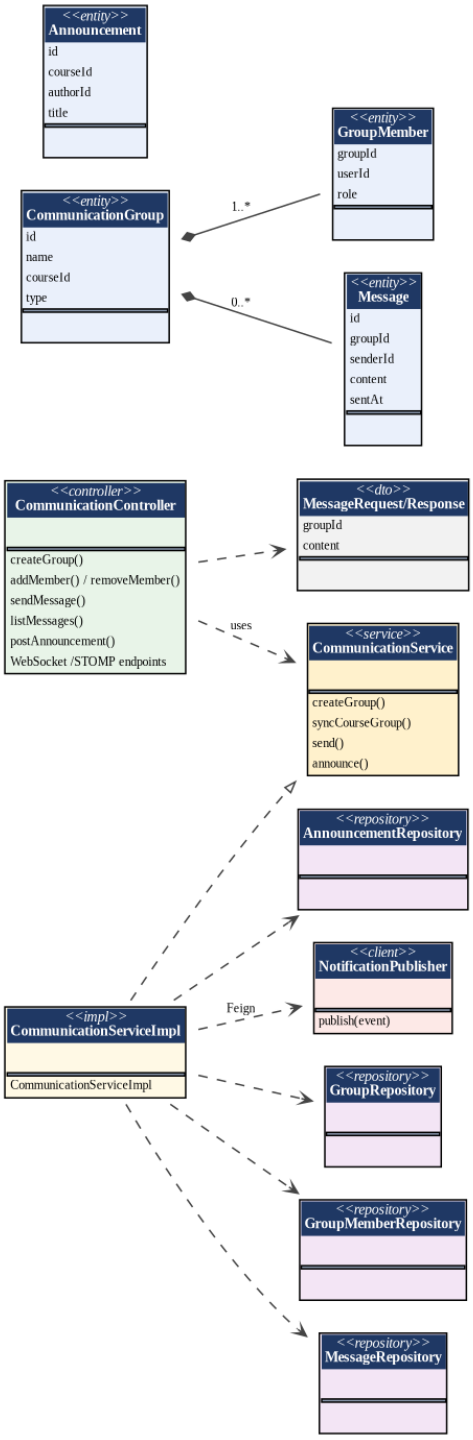


Figure 3.10: Class Diagram – Communication Service

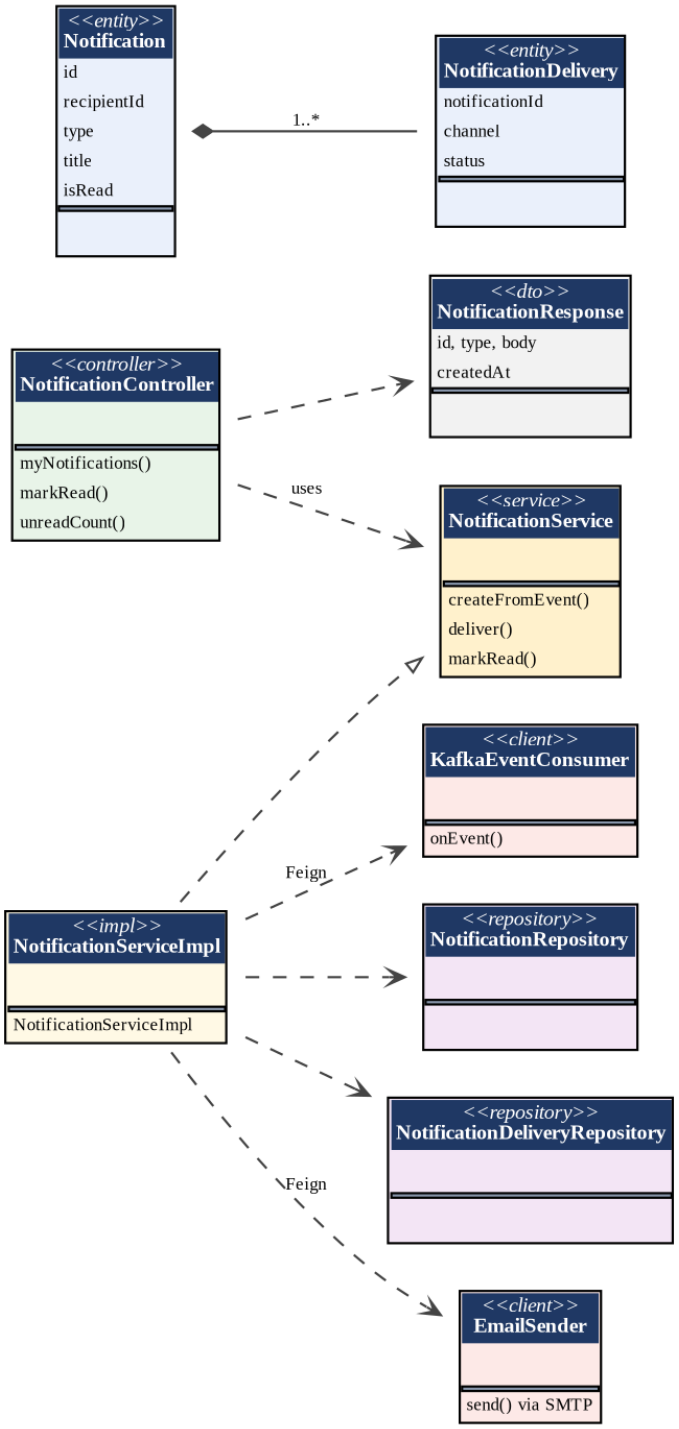


Figure 3.11: Class Diagram – Notification Service

3.12 Calendar Service

The Calendar Service manages manually created events and venues and maintains projections from exams, assignments, quizzes, and attendance sessions, keyed by source type and identifier, with reconciliation of failed projections and role-based visibility on queries.

3.13 Analytics Service

The Analytics Service owns no source data. AnalyticsServiceImpl composes dashboard DTOs from clients for the user, course, enrollment, assessment, attendance, and payment services and applies role and academic-scope checks. This replaces the Version 1.0 design in which Analytics owned attendance records.

3.14 AI Service

The AI Service exposes REST and WebSocket chat controllers restricted to the Teacher, Admin, and Super Admin roles, summarisation endpoints open to Student, Teacher, Admin, and Super Admin, and a quiz-draft capability for teachers. AiChatServiceImpl applies usage guards and content filtering, calls authorised domain tools (quiz, attendance, and assignment performance, student progress, course resolution, registration overview, and upcoming calendar events), invokes a configured provider adapter, and persists the conversation, messages, and an audit event.

The student frontend contains an AI client (student/src/api/ai.ts) that sends messages to /api/ai/chat. That backend route is authorised only for Teacher, Admin, and Super Admin, so the student conversational feature is a documented deviation (Appendix C) and is not depicted as a working student capability.

3.15 Inter-Service Class Diagram

The inter-service diagram replaces the Version 1.0 composition diagram. It shows the gateway routing to each service, the synchronous Feign dependencies between services, and the asynchronous publication of events to Kafka for consumption by the notification-service. It is a component-level dependency model, not a class listing.

Figure 32 should be read together with the data ownership matrix in Chapter 2: a dependency arrow indicates that the caller needs data or an authorisation decision owned by the callee.

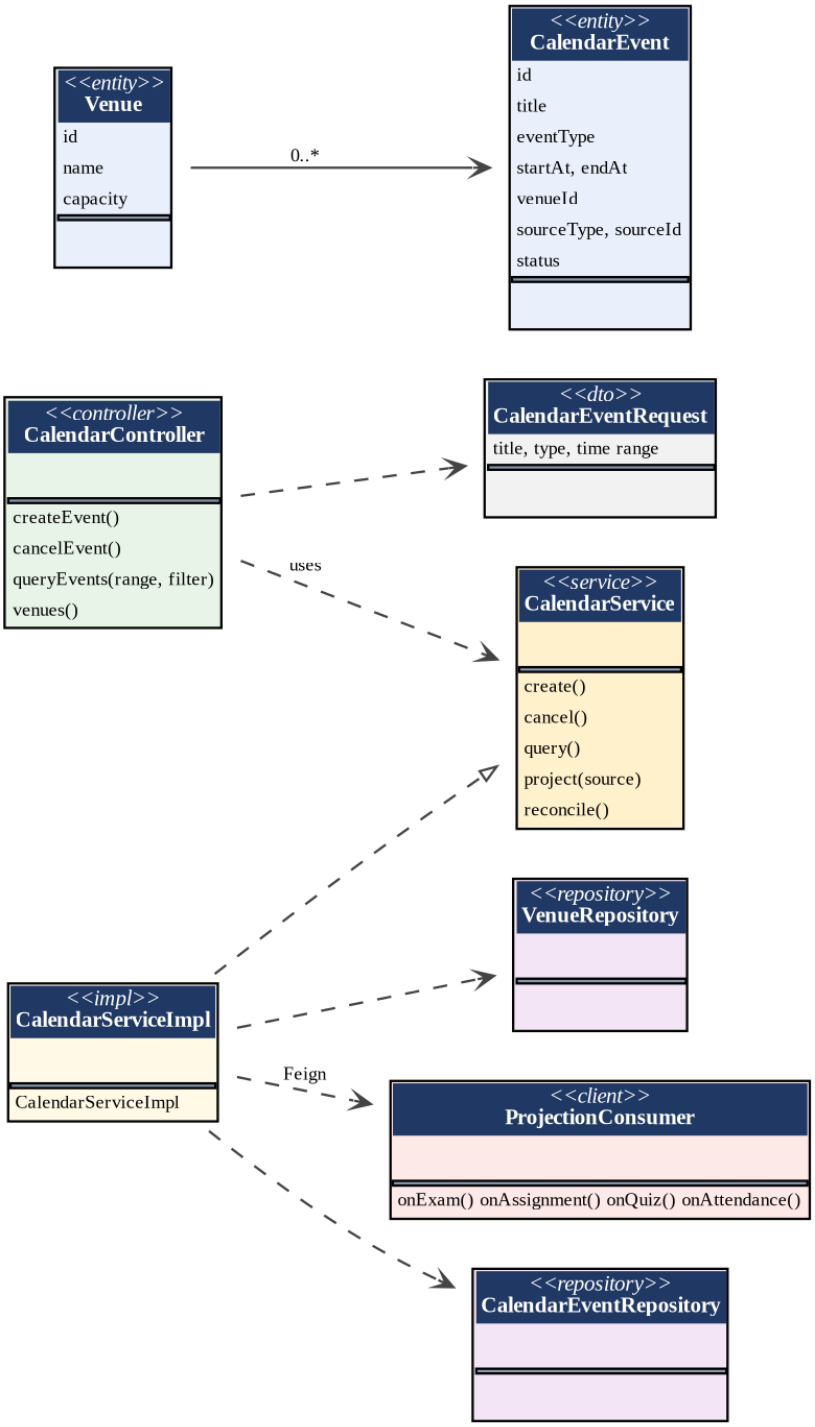


Figure 3.12: Class Diagram – Calendar Service

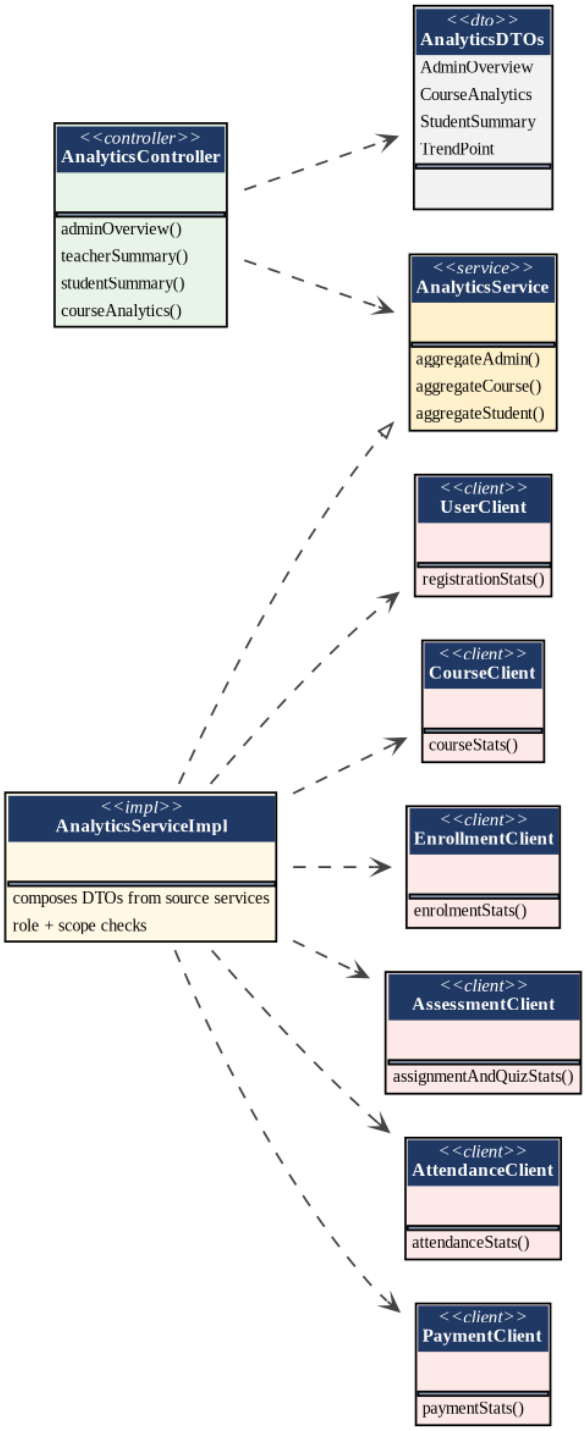


Figure 3.13: Class Diagram – Analytics Service

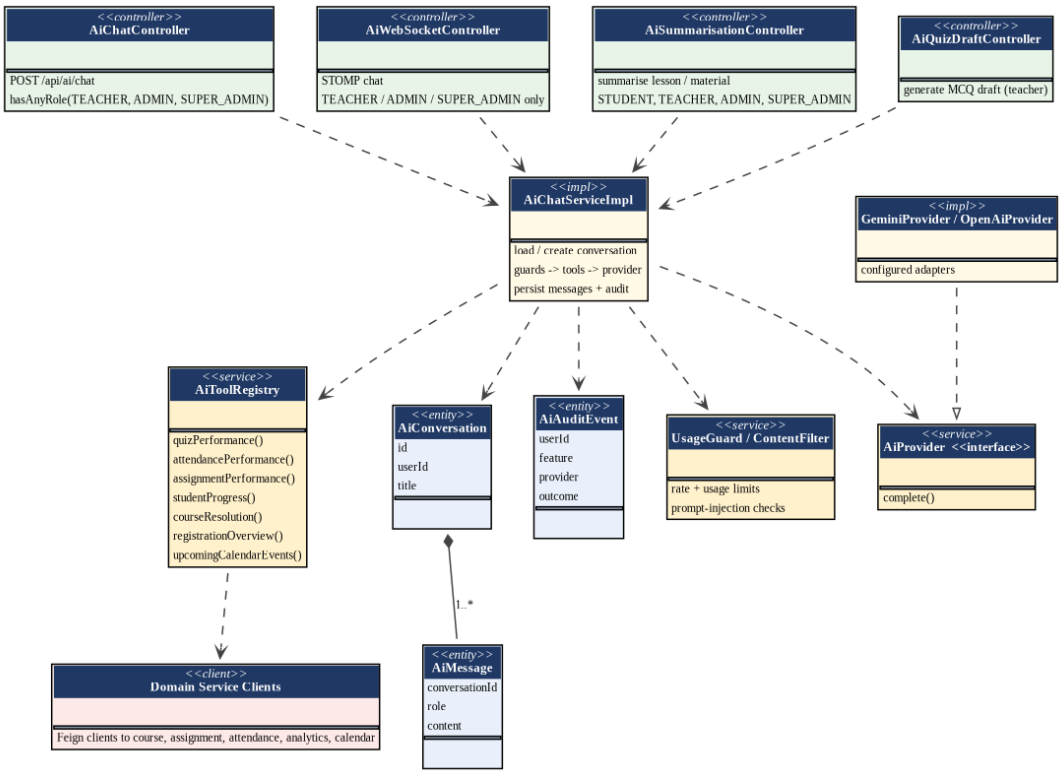


Figure 3.14: Class Diagram – AI Service

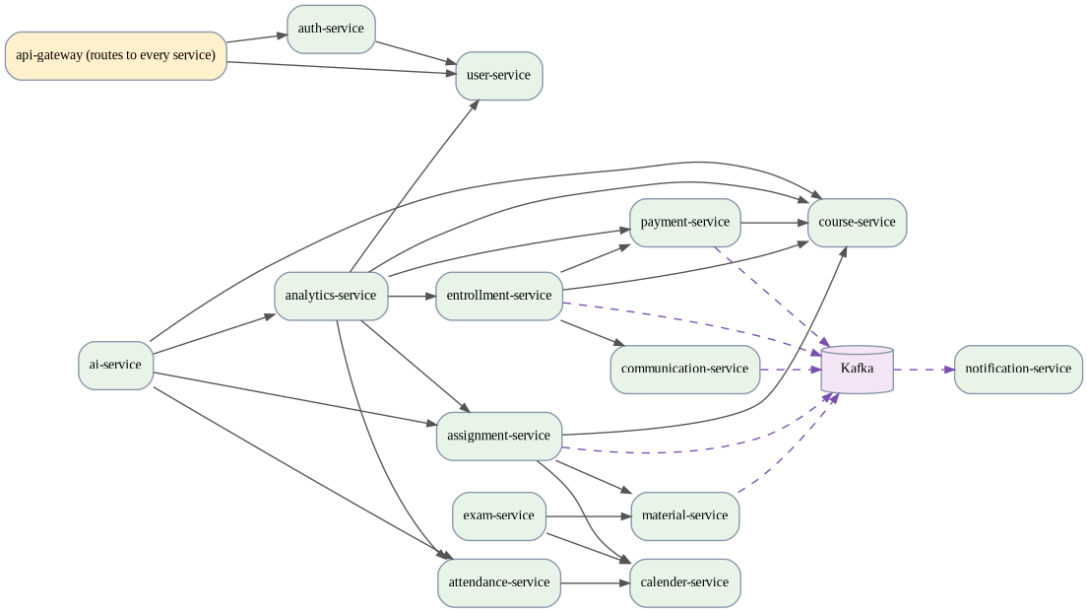


Figure 3.15: Inter-Service Class Diagram

Chapter 4

Sequence Diagrams

This chapter presents the Unified Modelling Language (UML) sequence diagrams for the major use cases of the system as implemented. Each sequence diagram models the chronological flow of messages between end users, frontend portals, the API Gateway, backend microservices, databases, object storage, message infrastructure, and external service integrations. The diagrams follow the UML 2.5 sequence diagram notation and cover the workflows of the Student, Teacher, Administrator, and Super Administrator roles.

Each use case section is structured to include: a descriptive overview, a formal use case details table, a tabular summary of actors and system components, pre-conditions and post-conditions, a numbered main success flow with alternative and exception flows, and the corresponding UML sequence diagram. Version 1.0 documented eleven use cases. Ten are retained and updated; the AI help chatbot use case is replaced by the role-authorised AI conversation, the AI-assisted grading flow is replaced by teacher-driven grading, the personal notes use case is withdrawn because personal-note persistence is not established in the implementation, and nine new use cases are added for payment, enrollment, attendance, examinations, marketplace, material access, account recovery, calendar projection, and notifications.

The consolidated reference table of all use cases in this chapter follows, with the unique identifier, primary actor, and the principal system components involved in each interaction.

Table 4.1: Summary of Major Use Cases and Corresponding Sequence Diagrams

Ref.	Use Case Name	Primary Actor	Key System Components
4.1	User Login and Authentication	Student / Teacher / Administrator / Super Administrator	Portal, API Gateway, Auth Service, User Service, PostgreSQL

4.2	Administrator Manages Users with RBAC	Administrator / Super Administrator	Admin Portal, API Gateway, User Service, Auth Service, SMTP
4.3	Teacher Uploads and Organises Learning Materials	Teacher	Portal, API Gateway, Course Service, Material Service, S3-compatible storage
4.4	AI-Assisted Quiz Generation	Teacher	Portal, API Gateway, AI Service, AI Provider, Assignment Service
4.5	Student Attempts Online Quiz with Auto-Grading	Student	Portal, API Gateway, Assignment Service, Enrollment Service, Kafka
4.6	Teacher Grades Assignment	Teacher	Portal, API Gateway, Assignment Service, Material Service, Kafka
4.7	Real-time Messaging between Student and Teacher	Student and Teacher	WebSocket/STOMP, Communication Service, Enrollment Service, PostgreSQL
4.8	Teacher Posts Classwide Announcement	Teacher	Portal, Communication Service, Kafka, Notification Service, SMTP
4.9	Teacher or Administrator Interacts with the AI Assistant	Teacher / Administrator / Super Administrator	Portal, AI Service, domain services, AI Provider, PostgreSQL
4.10	Student Views Performance Dashboard with Analytics	Student	Portal, Analytics Service, source services, Enrollment Service

4.11	Student Pays a Course Fee through Stripe	Student	Portal, Payment Service, Stripe, Course Service, PostgreSQL
4.12	Student Enrolment and Approval	Student	Portal, Enrollment Service, Payment Service, Course Service, Communication Service
4.13	Teacher Records Attendance	Teacher	Portal, Attendance Service, Calendar Service, PostgreSQL
4.14	Examination Management and Results	Teacher / Administrator	Portal, Exam Service, Material Service, Calendar Service, PostgreSQL
4.15	Administrator Publishes a Marketplace Product	Administrator	Admin Portal, Material Service, S3-compatible storage, Public Website
4.16	Authorised Material Download	Student / Teacher / Administrator	Portal, Material Service, Enrollment Service, S3-compatible storage
4.17	Password Recovery and First Login	Any registered user	Portal, Auth Service, SMTP, Auth DB
4.18	Calendar Projection and Reconciliation	System (Calendar Service); users view the calendar	Source services, Kafka / API, Calendar Service, PostgreSQL
4.19	Event-Driven Notification Delivery	System (Notification Service); users read notifications	Producer service, Kafka, Notification Service, PostgreSQL, SMTP

4.1 User Login and Authentication

This use case describes the credential-based authentication workflow of the first-party Auth Service. The portal submits credentials through the API Gateway; the Auth Service verifies them against its own credential records and issues a signed JWT access token and a refresh token. If the account must change its password, the portal forces the change before granting access. The use case also covers the validation of the token on a protected request and the renewal of an expired access token. It is a prerequisite for all other use cases. Social login through Google, described in Version 1.0, is not part of the implementation.

4.1.0.1 Use Case Details

Table 4.2: Use Case Details – User Login and Authentication

Use Case ID	UC-01
Use Case Name	User Login and Authentication
Primary Actor	Student / Teacher / Administrator / Super Administrator
Supporting Actors	Portal (Next.js), API Gateway, Auth Service, User Service, PostgreSQL
Trigger	A registered user opens a portal login page and submits an email address and password.

4.1.0.2 Actors and System Components

Table 4.3: Actors and Components – User Login and Authentication

Participant	Type	Responsibility
User	Primary Actor	Submits credentials and uses the role portal.
Portal (Next.js)	Frontend Component	Renders the role-specific screens, calls the REST or WebSocket APIs with the Bearer token, and presents validation and error feedback.

API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Auth Service	Application Server	Verifies credentials, issues and refreshes JWT access tokens, manages one-time passwords, and publishes the signing keys through JWKS.
User Service	Application Server	Owns user profiles, registration, invitations, and account status; enforces role checks on user operations.
PostgreSQL	Relational Database	Persists the owning service's data in its logical database.

4.1.0.3 Pre-conditions

● The user has an active account created by an administrator (or through registration) and holds one of the roles STUDENT, TEACHER, ADMIN, or SUPER_ADMIN.
● The portal can reach the platform through the reverse proxy and API Gateway.
● The Auth Service, User Service, and PostgreSQL are operational and the signing keys are available.

4.1.0.4 Main Success Flow

● The user enters the email address and password in the login form of the portal.
● The portal sends a login request over the network to the API Gateway, which routes the public authentication endpoint to the Auth Service.
● The Auth Service loads the credential record by email and verifies the password against the stored hash and the account status.
● On success it issues a signed access token and a refresh token and returns them with the role and a mustChangePassword indicator.
● The portal stores the session and redirects the user to the role-appropriate portal, or to a forced change-password screen when required.
● For each protected request the portal sends the access token as a Bearer token; the API Gateway validates signature (using the published JWKS) and expiry and forwards the re-

quest.
• The target service validates the token again and applies role and ownership checks before returning the data.
• When the access token expires, the portal requests a new one with the refresh token and the Auth Service returns a new access token.

4.1.0.5 Alternative and Exception Flows

If the email does not exist or the password is wrong, a generic 401 Unauthorized response is returned so that account existence is not disclosed. A deactivated account is rejected. If the access token has expired or is invalid, protected endpoints return 401 and the portal attempts a refresh; if the refresh token is expired or revoked the user must log in again. Account locking after repeated failures, which Version 1.0 specified, must be confirmed in the auth-service configuration before it is described as implemented.

4.1.0.6 Post-conditions

• The client holds a valid access token and a refresh token for the session.
• The user is presented with the portal that matches the role, or the mandatory password change.
• Subsequent requests are authorised by token validation at the gateway and at each service.

4.1.0.7 Sequence Diagram

Figure 33 below illustrates the complete interaction sequence for the use case described above.

4.2 Administrator Manages Users with RBAC

This use case models administrative user management under a fixed four-role model (STUDENT, TEACHER, ADMIN, SUPER_ADMIN). Administrators register students and teachers, resend invitations with temporary credentials, update account status, and delete users. Only a Super Administrator can register administrator accounts. Version 1.0 described editable role-permission matrices with a Redis cache and arbitrary role creation; the implementation uses fixed roles enforced through method-level authorisation, so those sub-workflows are removed.

4.2.0.1 Use Case Details

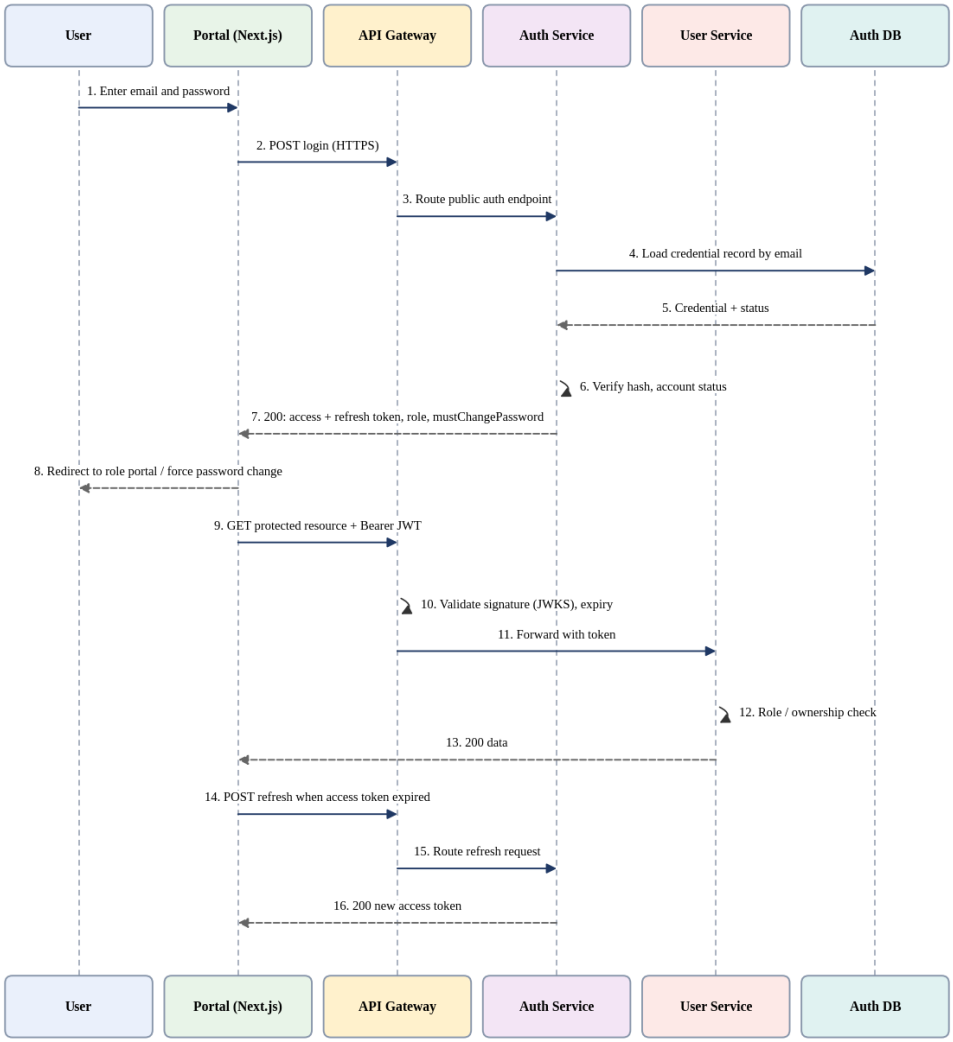


Figure 4.1: Sequence Diagram – User Login and Authentication

Table 4.4: Use Case Details – Administrator Manages Users with RBAC

Use Case ID	UC-02
Use Case Name	Administrator Manages Users with RBAC
Primary Actor	Administrator / Super Administrator
Supporting Actors	Administration Portal, API Gateway, User Service, Auth Service, Mail (SMTP)
Trigger	The administrator opens user management to register accounts, resend invitations, change account status, or delete a user.

4.2.0.2 Actors and System Components

Table 4.5: Actors and Components – Administrator Manages Users with RBAC

Participant	Type	Responsibility
Administrator / Super Administrator	Primary Actor	Performs user-management operations in the Administration Portal.
Admin Portal	Frontend Component	Renders user tables and forms and submits operations with the Bearer token.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
User Service	Application Server	Owns user profiles, registration, invitations, and account status; enforces role checks on user operations.
Auth Service	Application Server	Verifies credentials, issues and refreshes JWT access tokens, manages one-time passwords, and publishes the signing keys through JWKS.

Mail (SMTP)	External Service	Delivers invitations, one-time passwords, and notification e-mails.

4.2.0.3 Pre-conditions

- The actor is authenticated and holds the ADMIN or SUPER_ADMIN role.
- The User Service, Auth Service, and SMTP integration are operational.

4.2.0.4 Main Success Flow

- The administrator opens user management; the portal requests the user list through the API Gateway.
- The User Service verifies the role and returns the users.
- The administrator submits a new user (name, email, role, and grade or specialisation).
- The User Service validates the request; creation of an administrator account requires the SUPER_ADMIN role.
- The User Service asks the Auth Service to provision credentials with a temporary password and creates the profile.
- An invitation e-mail with the temporary credentials is sent through SMTP.
- The administrator can resend an invitation, update the account status, or delete the user; each request is role-checked.

4.2.0.5 Alternative and Exception Flows

A duplicate email address returns 409 Conflict and nothing is persisted. An Admin attempting to register an administrator receives 403 Forbidden. If SMTP is unavailable the account is created and the invitation can be resent. Version 1.0 behaviours such as blacklisting sessions in Redis on deactivation must be confirmed before being claimed.

4.2.0.6 Post-conditions

- The user and credentials are created and the invitation is dispatched, or the request is rejected without change.
- Account status changes and deletions are persisted by the User Service.

4.2.0.7 Sequence Diagram

Figure 34 below illustrates the complete interaction sequence for the use case described above.

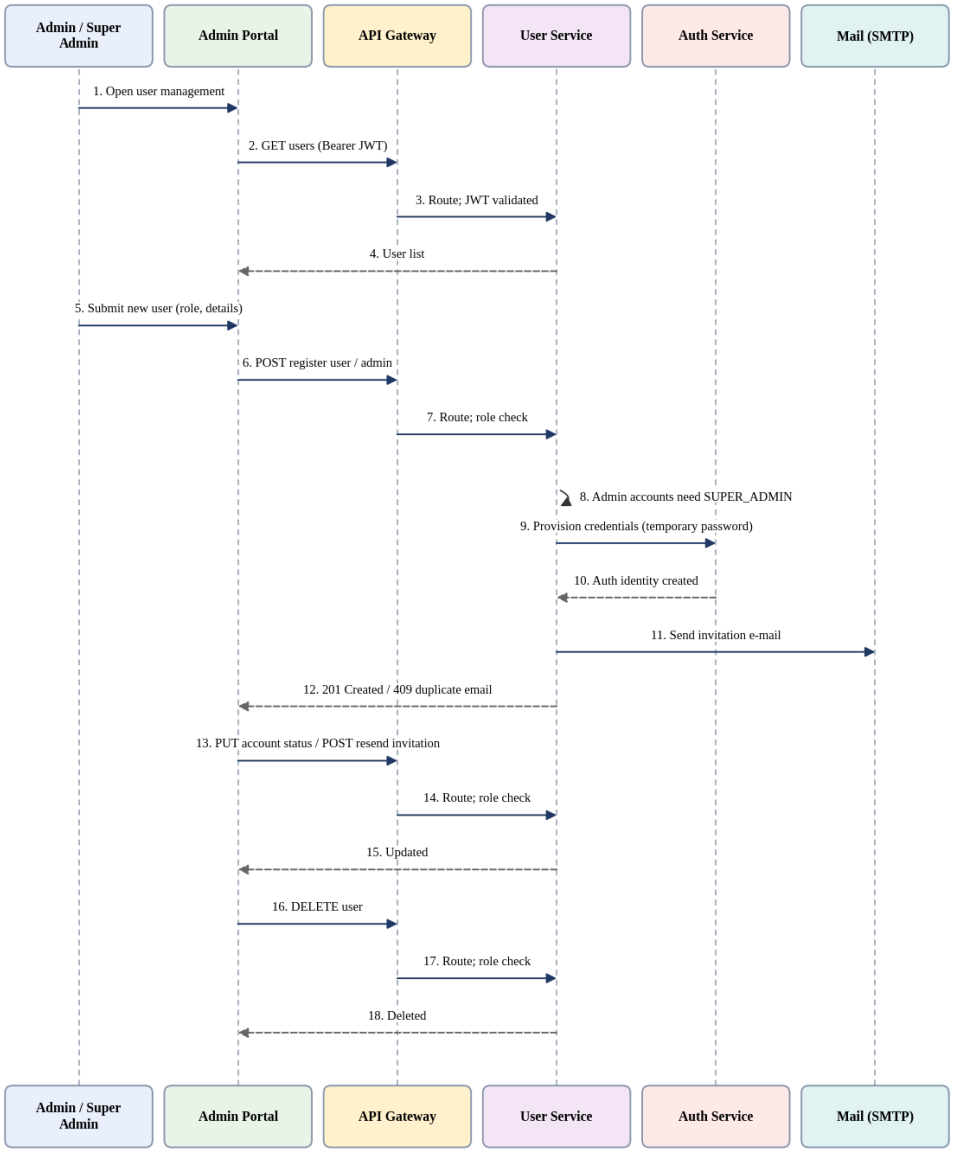


Figure 4.2: Sequence Diagram – Administrator Manages Users with RBAC

4.3 Teacher Uploads and Organises Learning Materials

This use case describes how a teacher uploads learning materials, associates them with modules and lessons, and manages the curriculum. File content is stored in S3-compatible storage, and the Material Service keeps the metadata, purpose, resource associations, and audit information. CloudFront delivery, inline virus scanning, and Quartz-scheduled publication, described in Version 1.0, are not established in the implementation and are removed; downloads use authorised presigned URLs (Section 4.16).

4.3.0.1 Use Case Details

Table 4.6: Use Case Details – Teacher Uploads and Organises Learning Materials

Use Case ID	UC-03
Use Case Name	Teacher Uploads and Organises Learning Materials
Primary Actor	Teacher
Supporting Actors	Teacher Portal, API Gateway, Course Service, Material Service, S3 Storage
Trigger	The teacher opens the curriculum builder to upload materials or organise modules and lessons.

4.3.0.2 Actors and System Components

Table 4.7: Actors and Components – Teacher Uploads and Organises Learning Materials

Participant	Type	Responsibility
Teacher	Primary Actor	Uploads files and organises the curriculum.
Teacher Portal	Frontend Component	Renders the curriculum builder and the multipart upload interface.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.

Course Service	Application Server	Owns grades, courses, modules, and lessons and enforces teacher ownership of course content.
Material Service	Application Server	Owns resource metadata, access authorisation, pre-signed download URLs, and the marketplace catalogue.
S3 Storage	Object Storage	Stores file content and serves time-limited pre-signed downloads.

4.3.0.3 Pre-conditions

- The teacher is authenticated with the TEACHER role and is assigned to the course.
- The course exists and the Material Service and object storage are operational.

4.3.0.4 Main Success Flow

- The teacher opens the curriculum builder; the portal fetches the course structure from the Course Service.
- The Course Service checks that the teacher owns the course and returns modules and lessons.
- The teacher selects a file, the purpose, and the target lesson.
- The portal submits a multipart upload to the Material Service through the gateway.
- The Material Service validates the file type and size against configured rules and authorises the teacher.
- The file is written to object storage under a generated storage key.
- The Material Service saves the metadata, the resource association, and an audit event and returns the material identifier.
- The teacher links the material to a lesson, and the lesson is saved with its publication status.

4.3.0.5 Alternative and Exception Flows

Unsupported types or oversized files are rejected with a validation error before storage. If the object-storage write fails the metadata is not persisted and the portal prompts a retry. A teacher who is not assigned to the course receives 403 Forbidden.

4.3.0.6 Post-conditions

- The file is stored in object storage and its metadata and associations are persisted.
- The lesson references the material and carries its publication state.
- An audit event records the operation.

4.3.0.7 Sequence Diagram

Figure 35 below illustrates the complete interaction sequence for the use case described above.

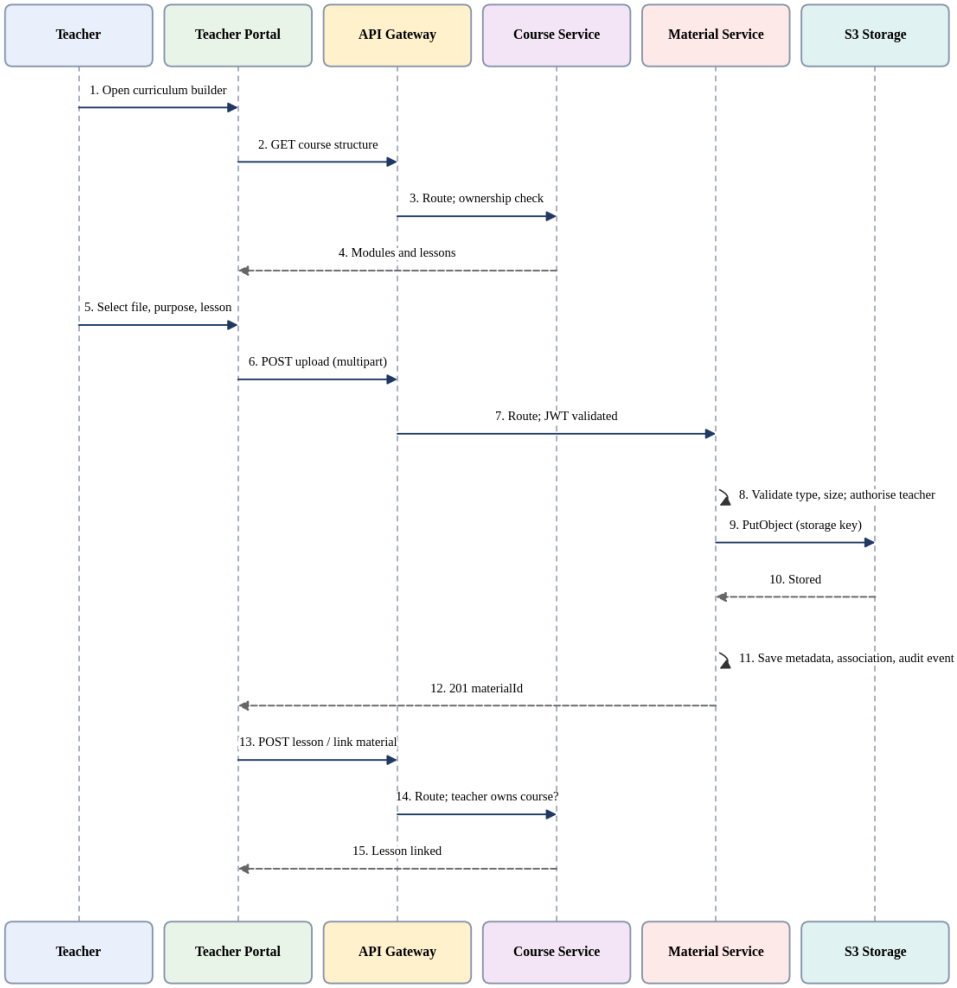


Figure 4.3: Sequence Diagram – Teacher Uploads and Organises Learning Materials

4.4 AI-Assisted Quiz Generation

This use case models AI-assisted creation of a multiple-choice quiz. The AI Service applies usage and content guards, loads permitted source content, prompts the configured provider, and validates the returned questions as single-answer multiple-choice items. The result is

returned as a draft only; the teacher reviews and edits it before the Assignment Service saves it as a draft quiz and later publishes it. AI generation of questions is separate from automatic scoring of attempts.

4.4.0.1 Use Case Details

Table 4.8: Use Case Details – AI-Assisted Quiz Generation

Use Case ID	UC-04
Use Case Name	AI-Assisted Quiz Generation
Primary Actor	Teacher
Supporting Actors	Teacher Portal, API Gateway, AI Service, AI Provider, Assignment Service
Trigger	The teacher selects a course and source material and requests an AI draft quiz.

4.4.0.2 Actors and System Components

Table 4.9: Actors and Components – AI-Assisted Quiz Generation

Participant	Type	Responsibility
Teacher	Primary Actor	Requests a draft, reviews and edits it, and publishes the quiz.
Portal (Next.js)	Frontend Component	Renders the role-specific screens, calls the REST or WebSocket APIs with the Bearer token, and presents validation and error feedback.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
AI Service	Application Server	Applies role, usage, and content guards; calls authorised tools and the AI provider; persists conversations and audit events.

AI Provider	External Service	Generates completions or draft content for the configured provider adapter.
Assignment Service	Application Server	Owns assignments, submissions, grading, quizzes, attempts, and results.

4.4.0.3 Pre-conditions

- The teacher is authenticated with the TEACHER role.
- Source material is available and the AI provider is configured and reachable.

4.4.0.4 Main Success Flow

- The teacher selects the course, source material, and number of questions.
- The portal requests a draft through the gateway; the AI Service verifies the role.
- The usage guard and content filter validate the request.
- The AI Service loads the permitted source content and prompts the AI provider for MCQ questions.
- The response is parsed and validated; only single-answer multiple-choice questions are accepted.
- An audit event is recorded and the draft is returned to the teacher.
- The teacher edits or removes questions and submits a create-quiz request (maximum 100 questions per request).
- The Assignment Service saves the quiz as DRAFT; the teacher publishes it when ready.

4.4.0.5 Alternative and Exception Flows

If the provider times out or fails, the AI Service returns an error and the teacher may create the quiz manually. If usage limits are exceeded or content is blocked, the request is rejected and audited. Invalid generated output is discarded.

4.4.0.6 Post-conditions

- A draft quiz exists only after the teacher saves it; AI output alone is never published.
- The AI request and its outcome are recorded in the audit data.

4.4.0.7 Sequence Diagram

Figure 36 below illustrates the complete interaction sequence for the use case described above.

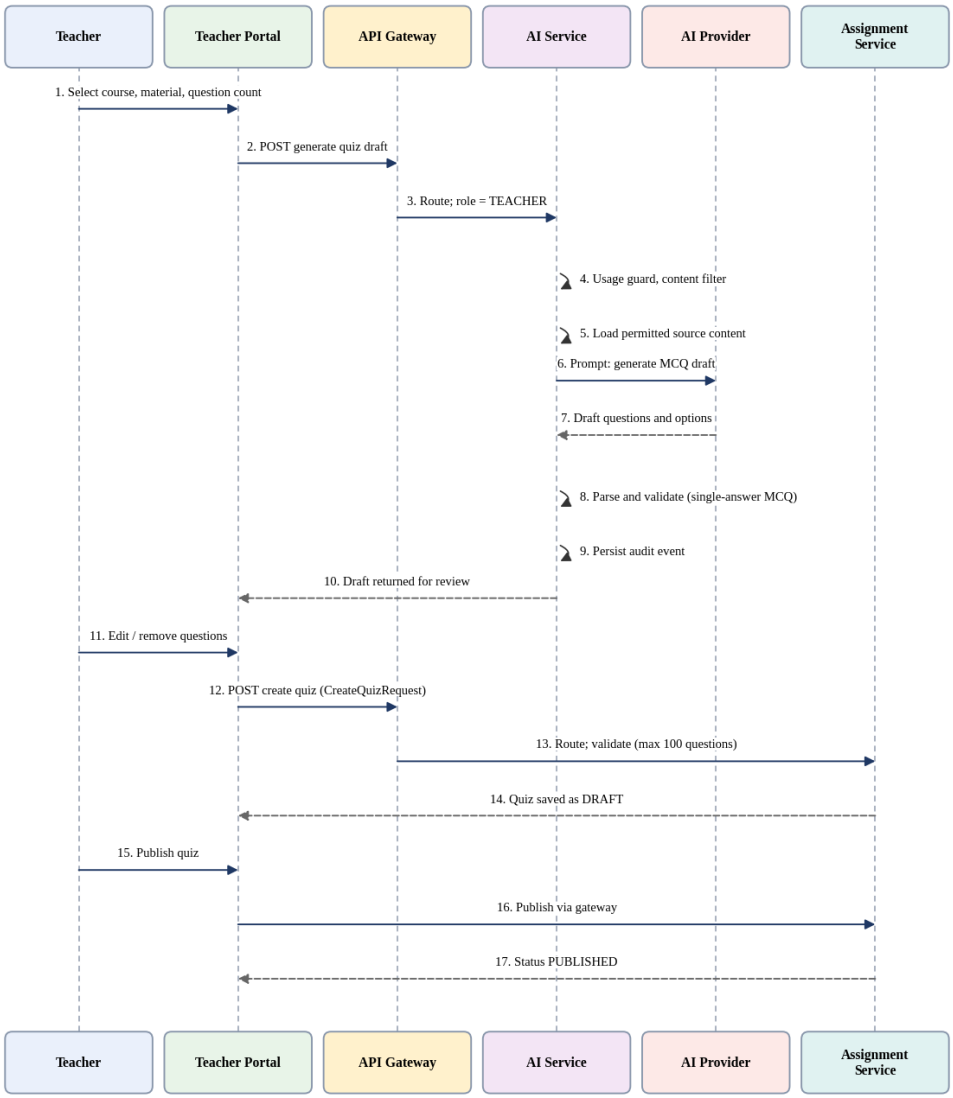


Figure 4.4: Sequence Diagram – AI-Assisted Quiz Generation

4.5 Student Attempts Online Quiz with Auto-Grading

This use case describes how an enrolled student attempts a published quiz. The Assignment Service checks access through the Enrollment Service, the availability window, and the remaining attempts, then scores the submitted single-answer multiple-choice answers by comparing them with the stored correct options. Auto-grading here is deterministic scoring; it does not use an AI model.

4.5.0.1 Use Case Details

Table 4.10: Use Case Details – Student Attempts Online Quiz with Auto-Grading

Use Case ID	UC-05
Use Case Name	Student Attempts Online Quiz with Auto-Grading
Primary Actor	Student
Supporting Actors	Student Portal, API Gateway, Assignment Service, Enrollment Service, Notification Service (Kafka)
Trigger	The student opens a published quiz and starts an attempt.

4.5.0.2 Actors and System Components

Table 4.11: Actors and Components – Student Attempts Online Quiz with Auto-Grading

Participant	Type	Responsibility
Student	Primary Actor	Attempts the quiz and views the result.
Student Portal	Frontend Component	Shows the questions, timer, and result.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Assignment Service	Application Server	Owns assignments, submissions, grading, quizzes, attempts, and results.

Enrollment Service	Application Server	Owns enrollment status; validates payment, applies approval rules, and confirms enrollment for access decisions.
Notification (Kafka)	Message Broker	Carries the result event for notifications.

4.5.0.3 Pre-conditions

- The student is authenticated and has an APPROVED enrollment in the course.
- The quiz is published and within its start and end times.

4.5.0.4 Main Success Flow

- The student opens the quiz; the portal requests its details.
- The Assignment Service confirms the enrollment with the Enrollment Service.
- It checks the availability window and the remaining attempts and returns questions without correct answers.
- The student starts the attempt and the service creates an attempt and timer.
- The student selects one option per question and submits.
- The service compares each selected option with the stored correct option.
- It calculates the score, percentage, and pass/fail result against the pass mark.
- A result event is published and the result is shown according to the quiz result-visibility setting.

4.5.0.5 Alternative and Exception Flows

If no attempts remain or the quiz is outside its window the service returns a conflict or validation error. If the timer expires the portal submits the recorded answers. Unanswered questions score zero.

4.5.0.6 Post-conditions

- The attempt, answers, score, percentage, and pass/fail result are persisted.
- A notification event is published for supported result notifications.

4.5.0.7 Sequence Diagram

Figure 37 below illustrates the complete interaction sequence for the use case described above.

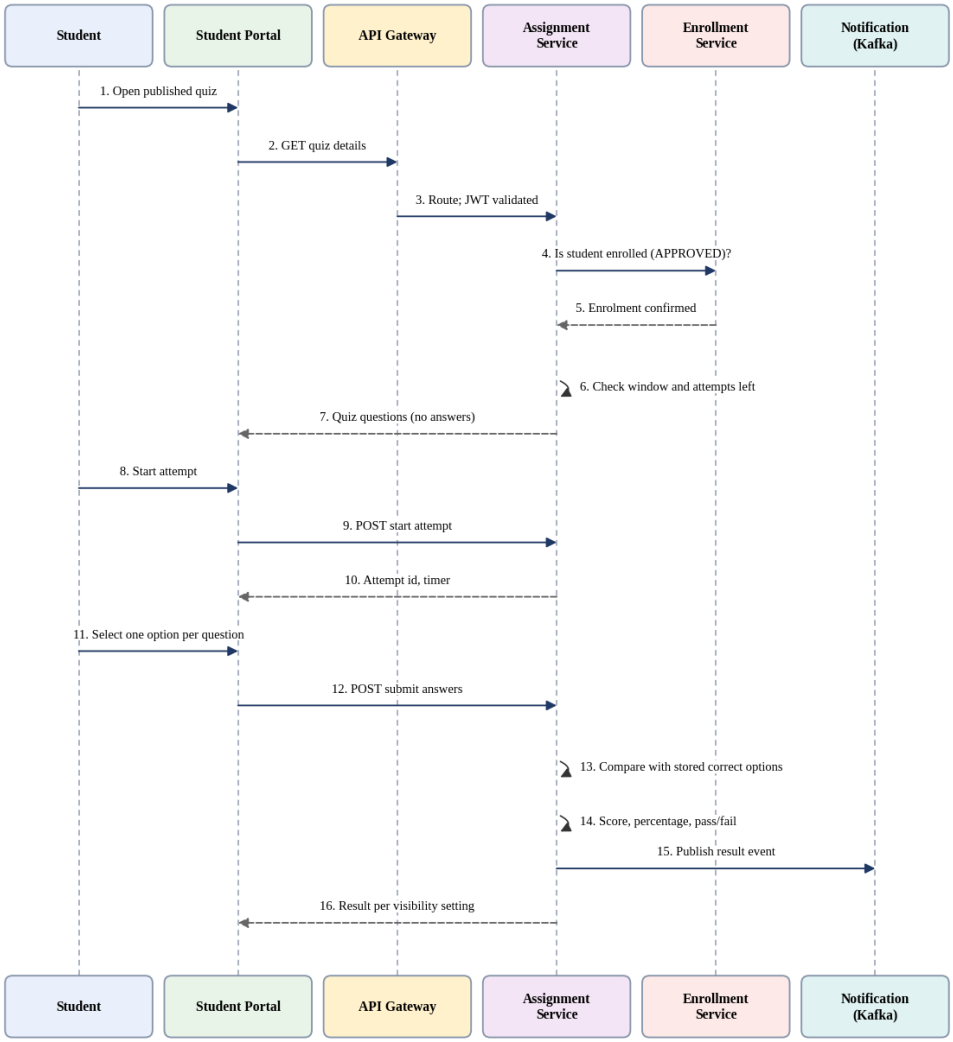


Figure 4.5: Sequence Diagram – Student Attempts Online Quiz with Auto-Grading

4.6 Teacher Grades Assignment

This use case describes teacher-driven grading. GradingServiceImpl verifies that the teacher is authorised for the assignment, records the marks and written feedback supplied in the grading request, updates the submission status, and triggers a student notification. Version 1.0 described AI-assisted grading with suggested scores; no AI grading of submissions is implemented, so that flow is removed and recorded as deferred.

4.6.0.1 Use Case Details

Table 4.12: Use Case Details – Teacher Grades Assignment

Use Case ID	UC-06
Use Case Name	Teacher Grades Assignment
Primary Actor	Teacher
Supporting Actors	Teacher Portal, API Gateway, Assignment Service, Material Service, Notification Service (Kafka)
Trigger	The teacher opens the submissions of an assignment in an assigned course.

4.6.0.2 Actors and System Components

Table 4.13: Actors and Components – Teacher Grades Assignment

Participant	Type	Responsibility
Teacher	Primary Actor	Reviews submissions, enters marks and feedback, and returns work.
Teacher Portal	Frontend Component	Shows submissions, rubric, and attachments and submits the grade.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Assignment Service	Application Server	Owns assignments, submissions, grading, quizzes, attempts, and results.

Material Service	Application Server	Owns resource metadata, access authorisation, pre-signed download URLs, and the marketplace catalogue.
Notification (Kafka)	Message Broker	Carries the grading event.

4.6.0.3 Pre-conditions

- The teacher is authenticated and assigned to the course of the assignment.
- At least one submission exists.

4.6.0.4 Main Success Flow

- The teacher opens the submissions list; the Assignment Service checks that the teacher is assigned to the course.
- The teacher opens a submission and downloads attachments through authorised presigned URLs.
- The teacher reviews the work against the rubric items.
- The teacher enters marks, feedback, and optional rubric points.
- The portal submits the grading request.
- The service validates that marks do not exceed the maximum marks.
- The grade is saved and the submission status becomes GRADED or RETURNED.
- A grading event notifies the student.

4.6.0.5 Alternative and Exception Flows

A teacher who is not authorised for the assignment receives 403 Forbidden. Marks above the maximum are rejected. Late submissions and exhausted attempts are rejected at submission time with HTTP 409.

4.6.0.6 Post-conditions

- Marks and feedback are persisted and the submission status is updated.
- The student is notified of the grading result.

4.6.0.7 Sequence Diagram

Figure 38 below illustrates the complete interaction sequence for the use case described above.

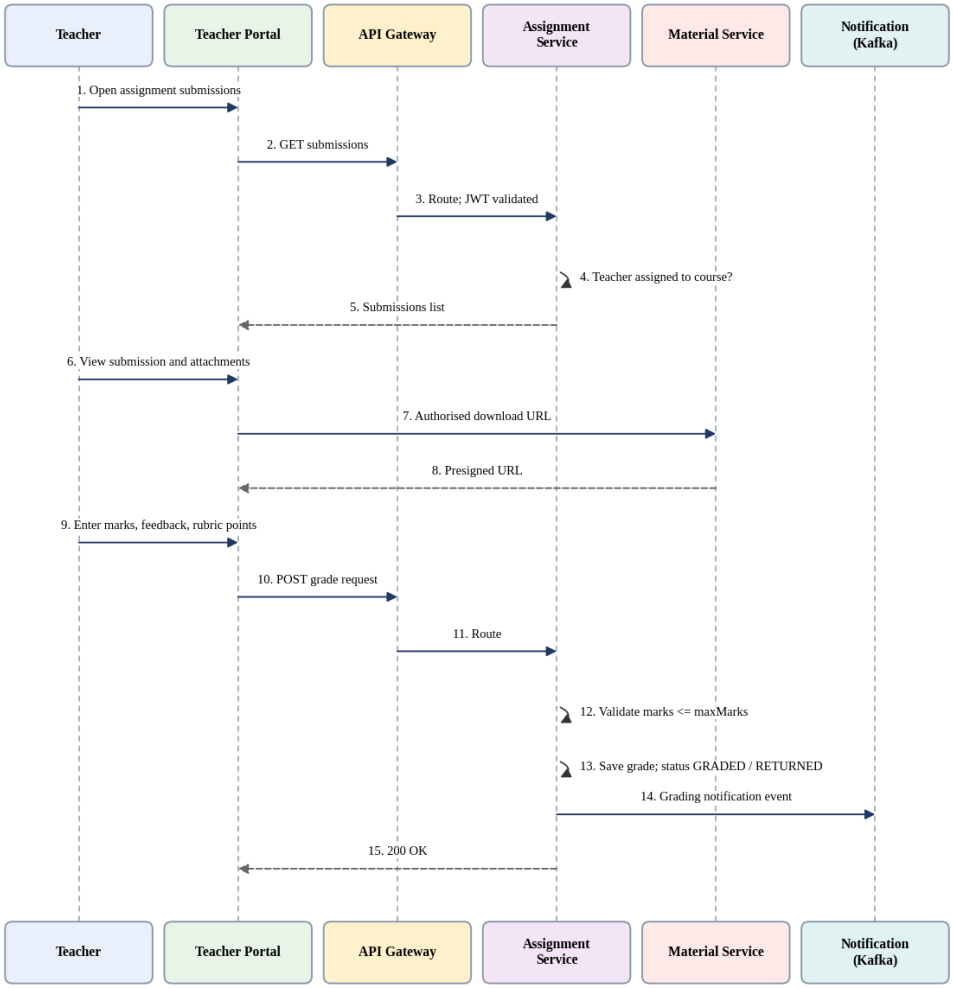


Figure 4.6: Sequence Diagram – Teacher Grades Assignment

4.7 Real-time Messaging between Student and Teacher

This use case describes group messaging between enrolled students and course teachers. Messaging is group-based: membership of a course group follows approved enrollment. The portal connects with a JWT over WebSocket/STOMP, the Communication Service binds the session to the authenticated user, persists messages, and broadcasts them to subscribed members. Private one-to-one messaging, as described in Version 1.0, is not part of the reviewed implementation.

4.7.0.1 Use Case Details

Table 4.14: Use Case Details – Real-time Messaging between Student and Teacher

Use Case ID	UC-07
Use Case Name	Real-time Messaging between Student and Teacher
Primary Actor	Student and Teacher
Supporting Actors	Student Portal, Teacher Portal, API Gateway, Communication Service, Enrollment Service
Trigger	A student or teacher opens a course communication group and sends a message.

4.7.0.2 Actors and System Components

Table 4.15: Actors and Components – Real-time Messaging between Student and Teacher

Participant	Type	Responsibility
Student	Primary Actor	Sends and reads messages in a course group.
Student Portal	Frontend Component	Maintains the WebSocket connection and renders messages.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Communication Service	Application Server	Owns groups, membership, messages, announcements, and WebSocket delivery.

Enrollment Service	Application Server	Owns enrollment status; validates payment, applies approval rules, and confirms enrollment for access decisions.
Teacher Portal	Frontend Component	Receives and replies to messages in real time.

4.7.0.3 Pre-conditions

- The student has an APPROVED enrollment and the teacher is assigned to the course.
- The course communication group exists and the WebSocket endpoint is reachable.

4.7.0.4 Main Success Flow

- The student opens the course group.
- The portal opens a STOMP connection and sends the JWT.
- The gateway routes the WebSocket and the Communication Service validates the token and binds the session to the user.
- The service verifies membership against the Enrollment Service.
- The student sends a message to the group destination.
- The service checks membership, persists the message, and broadcasts it to members.
- The teacher receives it and replies; the reply is broadcast in the same way.

4.7.0.5 Alternative and Exception Flows

A non-member is refused access to the group. If the WebSocket disconnects the portal reconnects and loads history through the REST API. Group synchronisation on enrollment uses strict and non-strict paths, so membership may lag briefly after approval.

4.7.0.6 Post-conditions

- Messages are persisted and delivered to connected members.
- Only members of the group see its messages.

4.7.0.7 Sequence Diagram

Figure 39 below illustrates the complete interaction sequence for the use case described above.

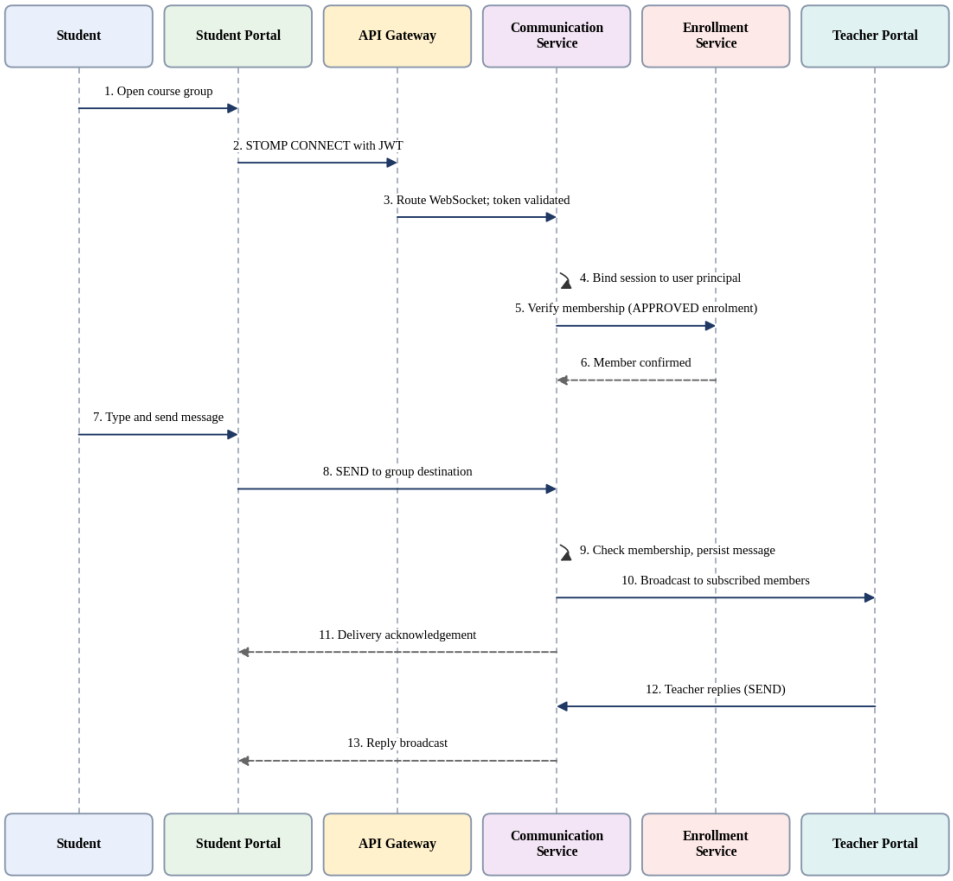


Figure 4.7: Sequence Diagram – Real-time Messaging between Student and Teacher

4.8 Teacher Posts Class-wide Announcement

This use case describes announcements to the students of a course. The Communication Service verifies that the teacher owns the course, persists the announcement, and publishes an event; the Notification Service consumes it to create notifications and deliver them through supported channels. Delivery is not guaranteed beyond what the Kafka and SMTP configuration provides.

4.8.0.1 Use Case Details

Table 4.16: Use Case Details – Teacher Posts Class-wide Announcement

Use Case ID	UC-08
Use Case Name	Teacher Posts Class-wide Announcement
Primary Actor	Teacher
Supporting Actors	Teacher Portal, API Gateway, Communication Service, Kafka, Notification Service
Trigger	The teacher composes an announcement for a course.

4.8.0.2 Actors and System Components

Table 4.17: Actors and Components – Teacher Posts Class-wide Announcement

Participant	Type	Responsibility
Teacher	Primary Actor	Authors and publishes the announcement.
Portal (Next.js)	Frontend Component	Renders the role-specific screens, calls the REST or WebSocket APIs with the Bearer token, and presents validation and error feedback.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.

Communication Service	Application Server	Owns groups, membership, messages, announcements, and WebSocket delivery.
Kafka	Message Broker	Carries asynchronous events between services.
Notification Service	Application Server	Consumes events, creates notifications, and delivers in-app and e-mail alerts.

4.8.0.3 Pre-conditions

- The teacher is authenticated and assigned to the course.
- Kafka and the Notification Service are operational.

4.8.0.4 Main Success Flow

- The teacher composes the announcement.
- The portal posts it through the gateway.
- The Communication Service verifies the teacher's ownership of the course.
- The announcement is persisted.
- An announcement event is published to Kafka and the teacher receives confirmation.
- The Notification Service consumes the event and creates a notification for each recipient.
- Notifications are delivered in-app and by e-mail where configured.
- Students read announcements through the communication API.

4.8.0.5 Alternative and Exception Flows

If Kafka is unavailable the announcement is stored but notifications are delayed or missing. E-mail failures are recorded as a failed delivery status. A teacher outside the course receives 403 Forbidden.

4.8.0.6 Post-conditions

- The announcement is persisted and visible to the course.
- Notification records and delivery statuses are created for recipients.

4.8.0.7 Sequence Diagram

Figure 40 below illustrates the complete interaction sequence for the use case described above.

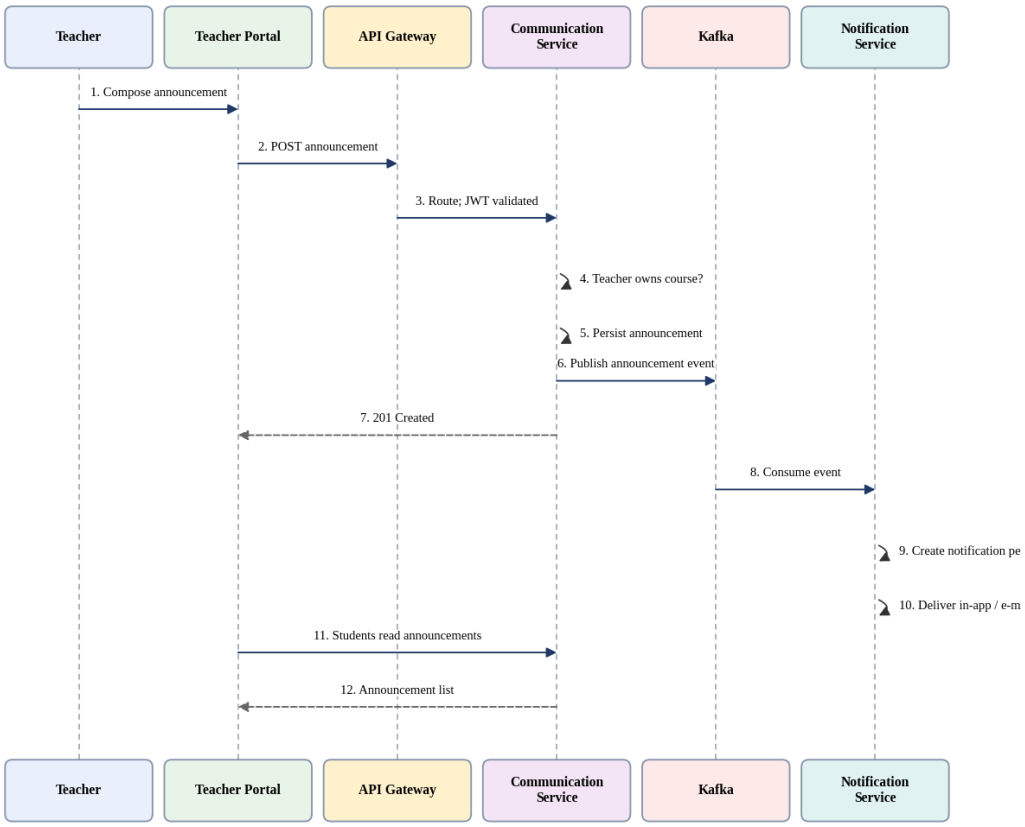


Figure 4.8: Sequence Diagram – Teacher Posts Class-wide Announcement

4.9 Teacher or Administrator Interacts with the AI Assistant

This use case describes the role-authorised AI conversation. The REST chat endpoint and the WebSocket controller permit only the Teacher, Admin, and Super Admin roles. The AI Service applies usage limits and content filtering, may call authorised tools that read quiz, attendance, and assignment performance, student progress, registration overview, and upcoming calendar events, calls the configured provider, and persists the conversation, messages, and an audit event. Lesson and material summarisation is a separate endpoint that also permits students. The Version 1.0 student help chatbot is not operational: the student frontend client calls /api/ai/chat, which denies the Student role (Appendix C).

4.9.0.1 Use Case Details

Table 4.18: Use Case Details – Teacher or Administrator Interacts with the AI Assistant

Use Case ID	UC-09
Use Case Name	Teacher or Administrator Interacts with the AI Assistant

Primary Actor	Teacher / Administrator / Super Administrator
Supporting Actors	Portal, API Gateway, AI Service, Domain Services, AI Provider
Trigger	A staff user opens the AI workspace and asks a question.

4.9.0.2 Actors and System Components

Table 4.19: Actors and Components – Teacher or Administrator Interacts with the AI Assistant

Participant	Type	Responsibility
Teacher / Admin	Primary Actor	Asks questions and continues conversations.
Portal	Frontend Component	Sends chat requests over REST or STOMP and shows answers.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
AI Service	Application Server	Applies role, usage, and content guards; calls authorised tools and the AI provider; persists conversations and audit events.
Domain Services	Application Servers	Provide scoped academic data to the AI tools.
AI Provider	External Service	Generates completions or draft content for the configured provider adapter.

4.9.0.3 Pre-conditions

- The user is authenticated with the TEACHER, ADMIN, or SUPER_ADMIN role.
- At least one AI provider is configured and reachable.

4.9.0.4 Main Success Flow

- The user asks a question in the AI workspace.
- The portal posts it to /api/ai/chat (or sends it over STOMP).
- The gateway routes the request and the AI Service verifies the role.
- The usage guard and content filter validate the request.
- The AI Service loads or creates the conversation.
- It calls authorised tools to retrieve scoped data from domain services.
- It sends the prompt with the context to the provider and receives the completion.
- The message and an audit event are persisted and the answer and conversation identifier are returned.

4.9.0.5 Alternative and Exception Flows

A Student calling the chat route receives 403 Forbidden. Provider timeouts or failures return an error and are audited; the conversation remains available. Tool calls never return data beyond the requester's authorisation.

4.9.0.6 Post-conditions

- The conversation and messages are persisted and can be continued by identifier.
- An audit event records the request outcome.

4.9.0.7 Sequence Diagram

Figure 41 below illustrates the complete interaction sequence for the use case described above.

4.10 Student Views Performance Dashboard with Analytics

This use case describes how the student dashboard is assembled. The Analytics Service resolves the student from the token so that only the student's own data is returned, fetches enrollments, grades, quiz and examination results, and attendance from the source services, and aggregates them into a summary. Learning-time or engagement tracking and AI analytics engines described in Version 1.0 are not part of the reviewed implementation.

4.10.0.1 Use Case Details

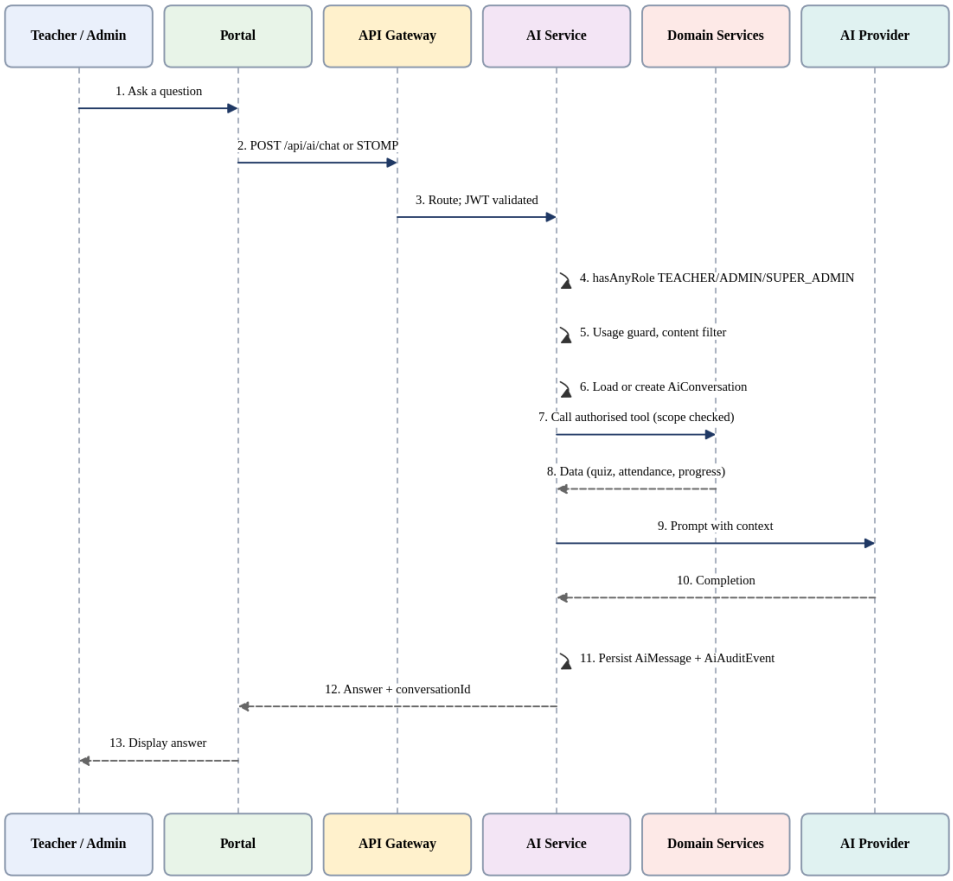


Figure 4.9: Sequence Diagram – Teacher or Administrator Interacts with the AI Assistant

Table 4.20: Use Case Details – Student Views Performance Dashboard with Analytics

Use Case ID	UC-10
Use Case Name	Student Views Performance Dashboard with Analytics
Primary Actor	Student
Supporting Actors	Student Portal, API Gateway, Analytics Service, Source Services, Enrollment Service
Trigger	The student opens the dashboard.

4.10.0.2 Actors and System Components

Table 4.21: Actors and Components – Student Views Performance Dashboard with Analytics

Participant	Type	Responsibility
Student	Primary Actor	Views progress, results, and attendance.
Student Portal	Frontend Component	Renders the summaries.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Analytics Service	Application Server	Aggregates source-service data into dashboard summaries with role and scope checks.
Source Services	Application Servers	Provide grades, results, and attendance.
Enrollment Service	Application Server	Owns enrollment status; validates payment, applies approval rules, and confirms enrollment for access decisions.

4.10.0.3 Pre-conditions

- The student is authenticated and has at least one enrollment.

4.10.0.4 Main Success Flow

- The student opens the dashboard.
- The portal requests the student summary.
- The Analytics Service validates the token and resolves the student identity.
- It fetches enrolled courses from the Enrollment Service.
- It fetches grades, quiz and exam results, and attendance from the source services.
- It aggregates the data into a summary DTO.
- The portal renders the progress and results.

4.10.0.5 Alternative and Exception Flows

If a source service is unavailable the summary is returned with the affected section omitted or an error indicated. A request for another student's data is rejected.

4.10.0.6 Post-conditions

- The student sees only own data; no source data is modified.

4.10.0.7 Sequence Diagram

Figure 42 below illustrates the complete interaction sequence for the use case described above.

4.11 Student Pays a Course Fee through Stripe

This use case describes course payment. The Payment Service resolves the course and grade fee, creates or reuses a pending payment, and creates a Stripe Payment Intent whose client secret is returned to the portal. The portal completes the payment with Stripe. A payment is considered successful only when Stripe's signed webhook event is verified and processed; a client redirect alone is not proof of capture. The configured monthly grade fee is a fee value and does not create recurring billing.

4.11.0.1 Use Case Details

Table 4.22: Use Case Details – Student Pays a Course Fee through Stripe

Use Case ID	UC-11
Use Case Name	Student Pays a Course Fee through Stripe
Primary Actor	Student

Supporting Actors	Student Portal, API Gateway, Payment Service, Stripe, Course Service
Trigger	The student selects a paid course and chooses to pay.

4.11.0.2 Actors and System Components

Table 4.23: Actors and Components – Student Pays a Course Fee through Stripe

Participant	Type	Responsibility
Student	Primary Actor	Chooses to pay and completes the payment.
Portal (Next.js)	Frontend Component	Renders the role-specific screens, calls the REST or WebSocket APIs with the Bearer token, and presents validation and error feedback.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Payment Service	Application Server	Owns grade fees and payments, creates Stripe Payment Intents, verifies webhooks, and processes refunds.
Stripe	External Service	Creates Payment Intents, confirms card payments, and sends signed webhook events.
Course Service	Application Server	Owns grades, courses, modules, and lessons and enforces teacher ownership of course content.

4.11.0.3 Pre-conditions

- The student is authenticated and the course requires payment.

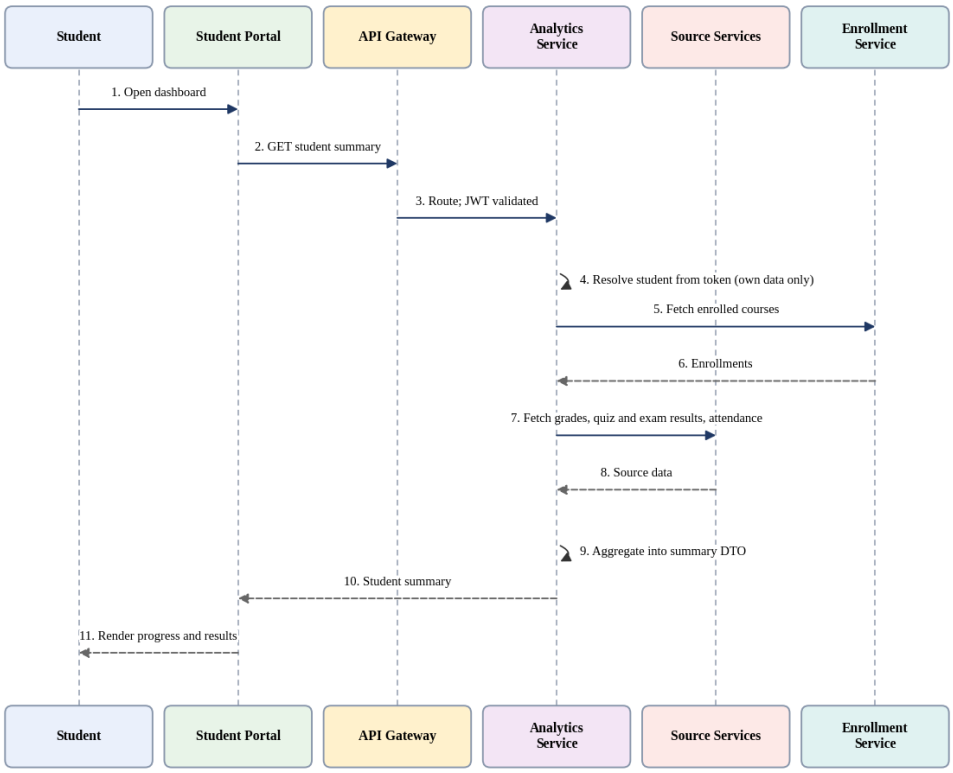


Figure 4.10: Sequence Diagram – Student Views Performance Dashboard with Analytics

• Stripe is reachable and webhook secrets are configured.

4.11.0.4 Main Success Flow

• The student selects a paid course and chooses to pay.
• The portal requests payment initiation through the gateway.
• The Payment Service resolves the course and grade fee from the Course Service.
• It creates or reuses a Payment record with status PENDING.
• It creates a Stripe Payment Intent and returns the client secret.
• The portal confirms the payment with Stripe.
• Stripe sends a signed webhook; the Payment Service verifies the signature and checks for a duplicate event.
• The payment status becomes SUCCESS or FAILED and the event is recorded.
• The student can read the status and payment history for own payments.

4.11.0.5 Alternative and Exception Flows

If the signature is invalid the webhook is rejected without any status change. A duplicate event is ignored. Failed payments allow a retry. If Stripe is unavailable, initiation fails with an error and no enrollment is possible. Refunds are performed by administrators and set the status to REFUNDED.

4.11.0.6 Post-conditions

- A payment record with its Stripe identifier and final status exists.
- Only a SUCCESS payment satisfies the enrollment payment check.

4.11.0.7 Sequence Diagram

Figure 43 below illustrates the complete interaction sequence for the use case described above.

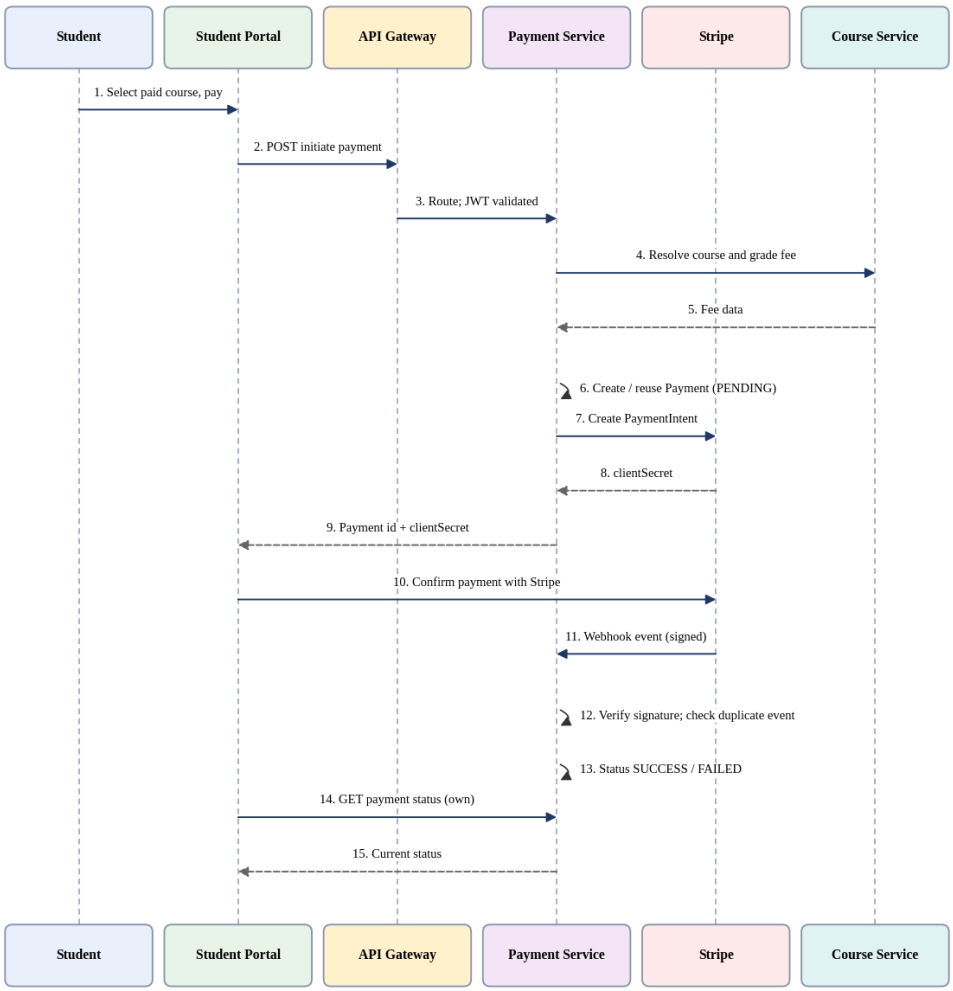


Figure 4.11: Sequence Diagram – Student Pays a Course Fee through Stripe

4.12 Student Enrolment and Approval

This use case describes student self-enrollment. The Enrollment Service validates the course and duplicates, verifies a successful payment for paid courses, and saves the enrollment with status PENDING. A teacher assigned to the course or an administrator then approves or rejects it. Staff-created enrollment is a separate path with its own authorisation and initial

status. Group membership synchronisation uses strict and non-strict handling paths and is not an all-or-nothing guarantee.

4.12.0.1 Use Case Details

Table 4.24: Use Case Details – Student Enrolment and Approval

Use Case ID	UC-12
Use Case Name	Student Enrolment and Approval
Primary Actor	Student
Supporting Actors	Student Portal, Enrollment Service, Payment Service, Course Service, Staff (Teacher or Admin)
Trigger	The student requests enrollment in a published course.

4.12.0.2 Actors and System Components

Table 4.25: Actors and Components – Student Enrolment and Approval

Participant	Type	Responsibility
Student	Primary Actor	Requests enrollment.
Portal (Next.js)	Frontend Component	Renders the role-specific screens, calls the REST or WebSocket APIs with the Bearer token, and presents validation and error feedback.
Enrollment Service	Application Server	Owns enrollment status; validates payment, applies approval rules, and confirms enrollment for access decisions.
Payment Service	Application Server	Owns grade fees and payments, creates Stripe Payment Intents, verifies webhooks, and processes refunds.

Course Service	Application Server	Owns grades, courses, modules, and lessons and enforces teacher ownership of course content.
Staff (Teacher / Admin)	Actor	Approves or rejects enrollment requests.

4.12.0.3 Pre-conditions

- The student is authenticated and the course is published.
- For paid courses a successful payment exists.

4.12.0.4 Main Success Flow

- The student requests enrollment; the portal calls the self-enroll endpoint.
- The Enrollment Service verifies that the course exists.
- It checks for a duplicate enrollment.
- For a paid course it asks the Payment Service whether a successful payment exists.
- It saves the enrollment with status PENDING.
- An authorised teacher (assigned course) or administrator reviews the request.
- The status is set to APPROVED or REJECTED.
- On approval, group membership is synchronised and the student is notified.

4.12.0.5 Alternative and Exception Flows

A duplicate returns a conflict. A missing payment rejects the request. An unauthorised teacher receives 403. If group synchronisation fails in the non-strict path the enrollment is kept and synchronisation is retried or reconciled.

4.12.0.6 Post-conditions

- An enrollment exists with status PENDING, APPROVED, or REJECTED.
- Approved students gain access to course content and the course group.

4.12.0.7 Sequence Diagram

Figure 44 below illustrates the complete interaction sequence for the use case described above.

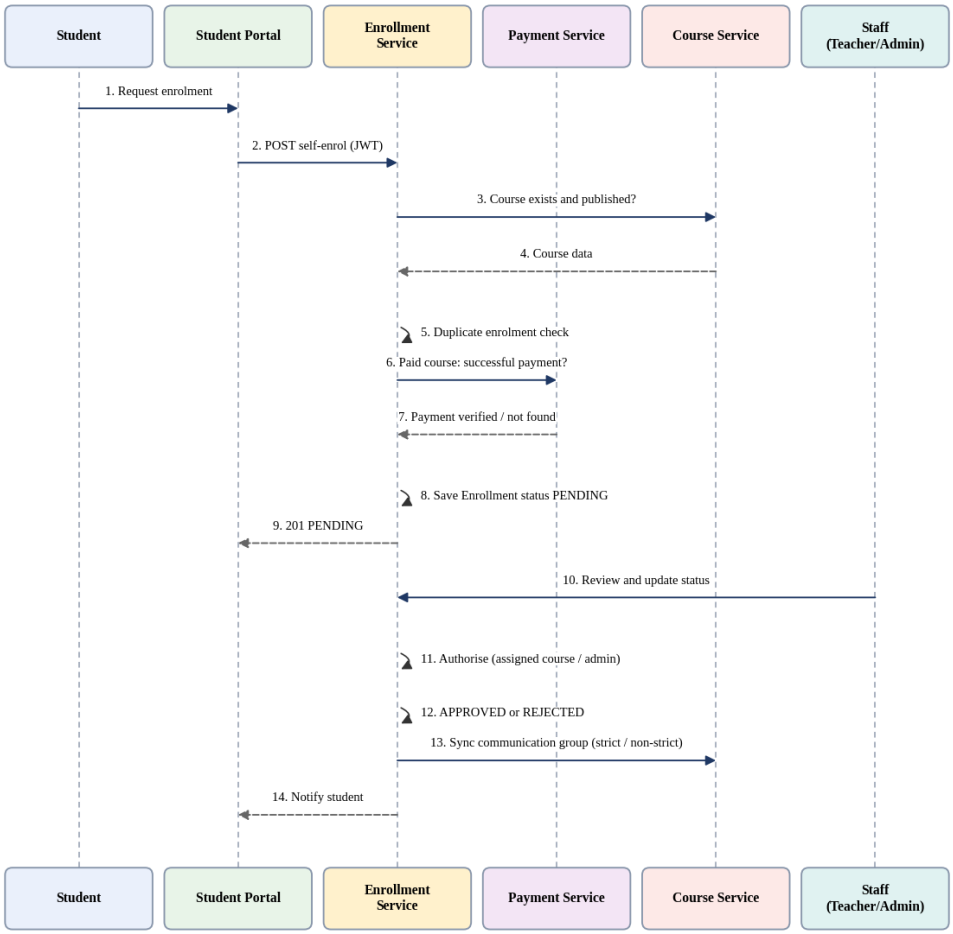


Figure 4.12: Sequence Diagram – Student Enrolment and Approval

4.13 Teacher Records Attendance

This use case describes attendance. A teacher creates an attendance session, marks each student, and completes the session. The Attendance Service maintains the records, updates summaries and analytics, and notifies the Calendar Service so that the session appears as a projected event. Students view only their own records and summaries. Automated methods such as QR, GPS, or biometric attendance are not implemented.

4.13.0.1 Use Case Details

Table 4.26: Use Case Details – Teacher Records Attendance

Use Case ID	UC-13
Use Case Name	Teacher Records Attendance
Primary Actor	Teacher
Supporting Actors	Teacher Portal, API Gateway, Attendance Service, Calendar Service, Student Portal
Trigger	The teacher creates an attendance session for a course.

4.13.0.2 Actors and System Components

Table 4.27: Actors and Components – Teacher Records Attendance

Participant	Type	Responsibility
Teacher	Primary Actor	Creates sessions and marks attendance.
Teacher Portal	Frontend Component	Renders the attendance workspace.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Attendance Service	Application Server	Owns attendance sessions, records, and summaries.
Calendar Service	Application Server	Owns calendar events and projections and applies role-based visibility.

Student Portal	Frontend Component	Shows the student's own attendance.

4.13.0.3 Pre-conditions

- The teacher is authenticated and assigned to the course.
- Students are enrolled in the course.

4.13.0.4 Main Success Flow

- The teacher creates an attendance session for the course.
- The Attendance Service checks authorisation and saves the session as OPEN.
- The session is projected to the Calendar Service.
- The teacher marks each student present, absent, or late.
- The service validates the students and stores the records.
- The teacher completes the session.
- The status becomes COMPLETED and summaries are updated.
- A student retrieves own records and summary.

4.13.0.5 Alternative and Exception Flows

Marking students who are not enrolled is rejected. If the calendar projection fails it is marked for reconciliation without blocking the session. A student can never read another student's records.

4.13.0.6 Post-conditions

- Attendance records and the session status are persisted.
- Summaries and analytics reflect the completed session.

4.13.0.7 Sequence Diagram

Figure 45 below illustrates the complete interaction sequence for the use case described above.

4.14 Examination Management and Results

This use case describes the examination lifecycle. An authorised user creates an examination of type MIDTERM, FINAL, PRACTICAL, or OTHER, attaches files, and the Exam Service synchronises the schedule with the calendar. After the exam, marks are entered and

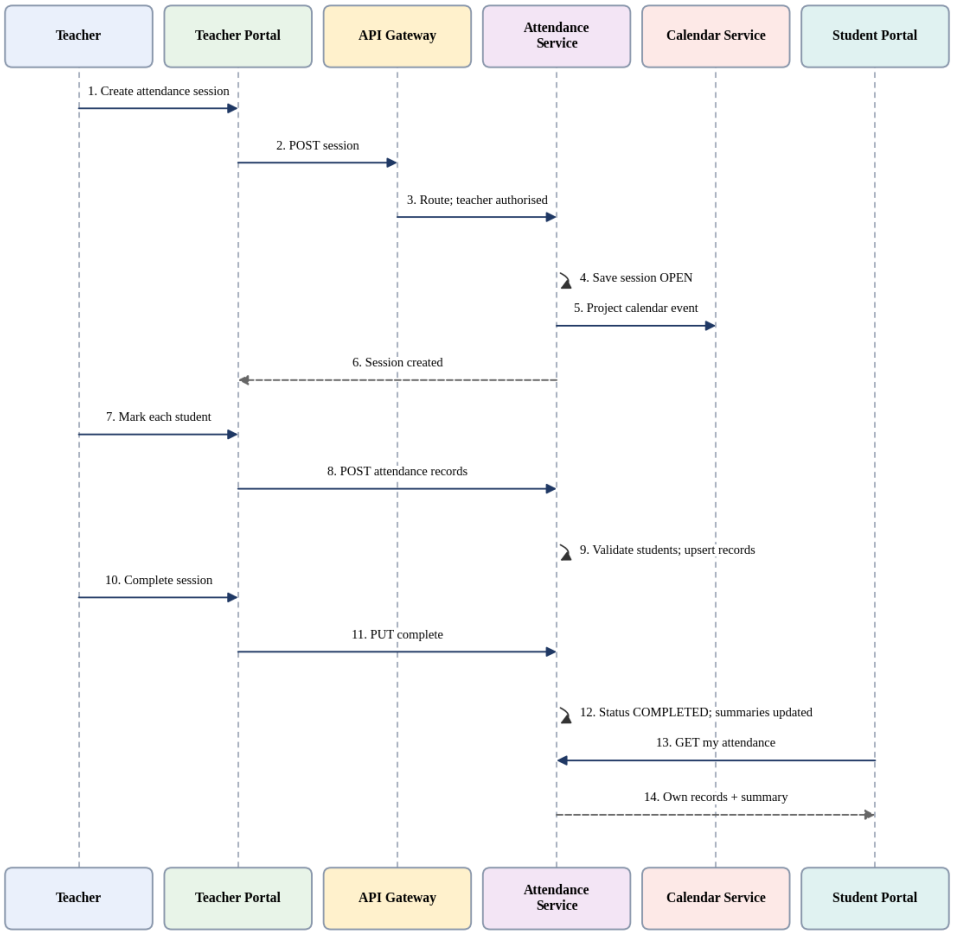


Figure 4.13: Sequence Diagram – Teacher Records Attendance

validated and a result sheet is maintained. Students see upcoming examinations and, once published, their own results.

4.14.0.1 Use Case Details

Table 4.28: Use Case Details – Examination Management and Results

Use Case ID	UC-14
Use Case Name	Examination Management and Results
Primary Actor	Teacher / Administrator
Supporting Actors	Portal, Exam Service, Material Service, Calendar Service, Student Portal
Trigger	An authorised user creates an examination for a course.

4.14.0.2 Actors and System Components

Table 4.29: Actors and Components – Examination Management and Results

Participant	Type	Responsibility
Teacher / Admin	Primary Actor	Creates exams, enters marks, and publishes results.
Portal	Frontend Component	Renders examination management and result entry.
Exam Service	Application Server	Owns examinations, attachments, marks, results, and result publication.
Material Service	Application Server	Owns resource metadata, access authorisation, pre-signed download URLs, and the marketplace catalogue.
Calendar Service	Application Server	Owns calendar events and projections and applies role-based visibility.
Student Portal	Frontend Component	Shows upcoming exams and published results.

4.14.0.3 Pre-conditions

- The user is authenticated and authorised for the course.
- The course exists.

4.14.0.4 Main Success Flow

- The user creates an exam with type, course, schedule, and instructions.
- The Exam Service authorises the request and validates the type.
- Exam attachments are linked after the Material Service authorises them.
- The schedule is synchronised to the Calendar Service.
- After the exam the user enters marks and completes the result sheet.
- The service validates that marks do not exceed the maximum.
- The user publishes the results.
- Students retrieve upcoming exams and only their own published results.

4.14.0.5 Alternative and Exception Flows

Invalid marks are rejected. Unpublished results are hidden from students. A failed calendar synchronisation is marked for reconciliation.

4.14.0.6 Post-conditions

- Examination, attachment, and result data are persisted.
- Results are visible to students only after publication.

4.14.0.7 Sequence Diagram

Figure 46 below illustrates the complete interaction sequence for the use case described above.

4.15 Administrator Publishes a Marketplace Product

This use case describes the administrator-managed marketplace catalogue. The administrator uploads a material with the marketplace purpose, creates a product that references it with category, subject, grade, price, discount, and currency, and publishes it. Public users can list and search published products only. Cart, checkout, purchase history, and purchaser entitlement are not implemented; the public website resource catalogue is sample data and its payment module is a placeholder.

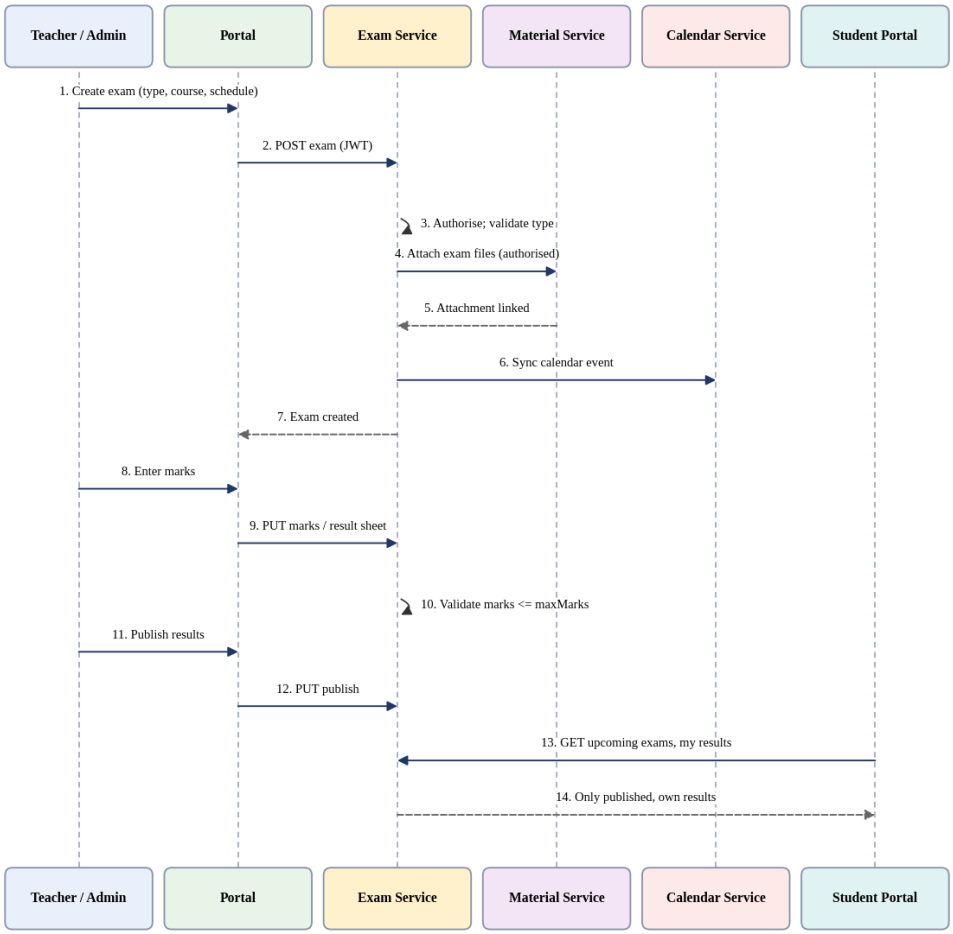


Figure 4.14: Sequence Diagram – Examination Management and Results

4.15.0.1 Use Case Details

Table 4.30: Use Case Details – Administrator Publishes a Marketplace Product

Use Case ID	UC-15
Use Case Name	Administrator Publishes a Marketplace Product
Primary Actor	Administrator
Supporting Actors	Administration Portal, API Gateway, Material Service, S3 Storage, Public Website
Trigger	The administrator creates and publishes an educational material product.

4.15.0.2 Actors and System Components

Table 4.31: Actors and Components – Administrator Publishes a Marketplace Product

Participant	Type	Responsibility
Administrator	Primary Actor	Uploads materials and manages products.
Admin Portal	Frontend Component	Renders product management.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Material Service	Application Server	Owns resource metadata, access authorisation, pre-signed download URLs, and the marketplace catalogue.
S3 Storage	Object Storage	Stores file content and serves time-limited pre-signed downloads.
Public Website	Frontend Component	Lists published products (sample content in the reviewed website).

4.15.0.3 Pre-conditions

• The administrator is authenticated with the ADMIN or SUPER_ADMIN role.

4.15.0.4 Main Success Flow

• The administrator uploads the material with the marketplace purpose.
• The Material Service stores the file and returns the material identifier.
• The administrator enters title, descriptions, category, subject, grade, price, and discount.
• The service validates pricing: free products must have a zero price and paid products a positive price; the final price is calculated.
• The product is saved as DRAFT.
• The administrator changes the status to PUBLISHED.
• Public requests list published products only.

4.15.0.5 Alternative and Exception Flows

Invalid pricing is rejected. Non-administrators receive 403. Products that are not published are not returned by the public catalogue.

4.15.0.6 Post-conditions

• The product exists with its publication status.
• No purchase or entitlement is created.

4.15.0.7 Sequence Diagram

Figure 47 below illustrates the complete interaction sequence for the use case described above.

4.16 Authorised Material Download

This use case describes secure file access. The Material Service evaluates the material status, purpose, and the requester's relationship (owner, enrolled student, assigned teacher, or administrator) and only then issues a time-limited presigned URL. The file is then retrieved directly from object storage, and an audit event is written.

4.16.0.1 Use Case Details

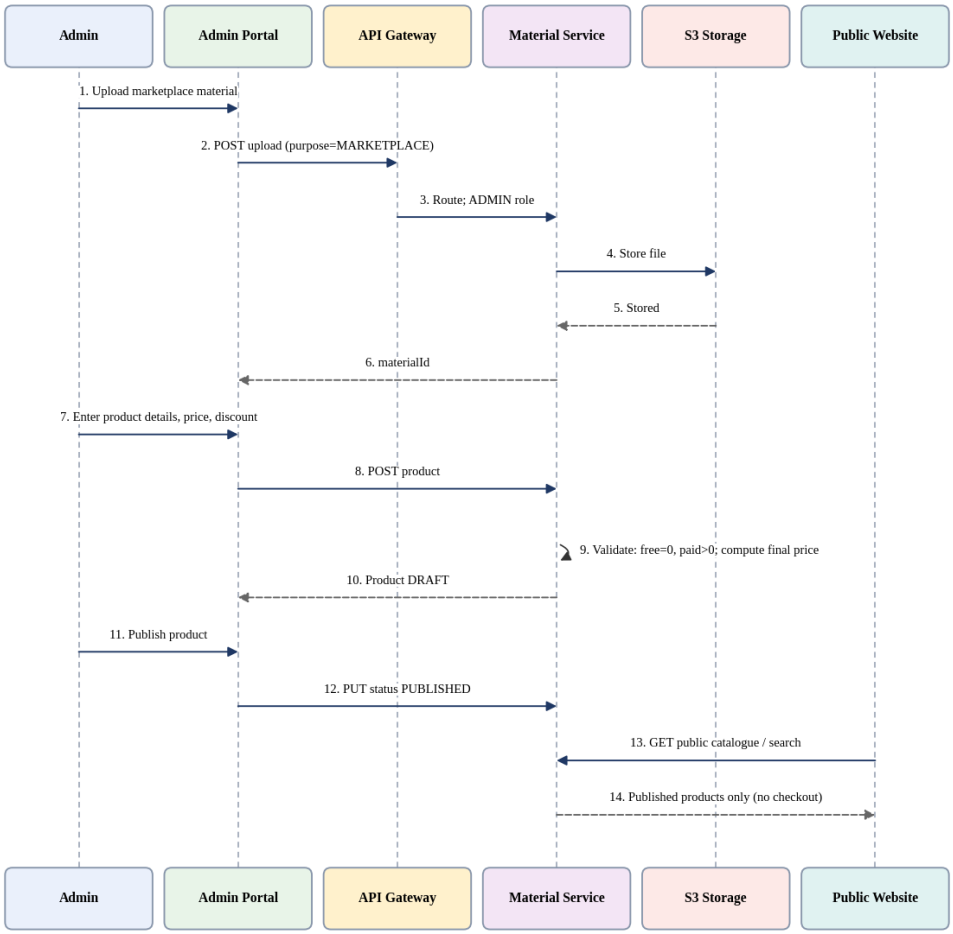


Figure 4.15: Sequence Diagram – Administrator Publishes a Marketplace Product

Table 4.32: Use Case Details – Authorised Material Download

Use Case ID	UC-16
Use Case Name	Authorised Material Download
Primary Actor	Student / Teacher / Administrator
Supporting Actors	Portal, API Gateway, Material Service, Enrollment Service, S3 Storage
Trigger	A user requests to download a learning resource.

4.16.0.2 Actors and System Components

Table 4.33: Actors and Components – Authorised Material Download

Participant	Type	Responsibility
User	Primary Actor	Requests the file.
Portal (Next.js)	Frontend Component	Renders the role-specific screens, calls the REST or WebSocket APIs with the Bearer token, and presents validation and error feedback.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Material Service	Application Server	Owns resource metadata, access authorisation, pre-signed download URLs, and the marketplace catalogue.
Enrollment Service	Application Server	Owns enrollment status; validates payment, applies approval rules, and confirms enrollment for access decisions.
S3 Storage	Object Storage	Stores file content and serves time-limited pre-signed downloads.

4.16.0.3 Pre-conditions

- The user is authenticated.
- The material exists and is active.

4.16.0.4 Main Success Flow

- The user selects a file; the portal requests a download URL.
- The gateway validates the token and routes to the Material Service.
- The service loads the metadata, status, and purpose.
- It determines the user's relationship to the resource, consulting the Enrollment Service for students.
- If access is denied it returns 403.
- If allowed it presigns a time-limited GET.
- An audit event is written.
- The portal retrieves the file using the presigned URL.

4.16.0.5 Alternative and Exception Flows

Archived or deleted materials return an error. Expired URLs must be re-requested. Students without approved enrollment cannot access course materials.

4.16.0.6 Post-conditions

- Only authorised users obtain a URL, and the access is audited.

4.16.0.7 Sequence Diagram

Figure 48 below illustrates the complete interaction sequence for the use case described above.

4.17 Password Recovery and First Login

This use case describes recovery and first login. A user who forgot a password receives a one-time password by e-mail and sets a new password. A user provisioned by an administrator signs in with temporary credentials and is required to change the password before using the portal. The OTP lifecycle (expiry, single use, delivery) must be confirmed against the auth-service implementation before specific values are quoted.

4.17.0.1 Use Case Details

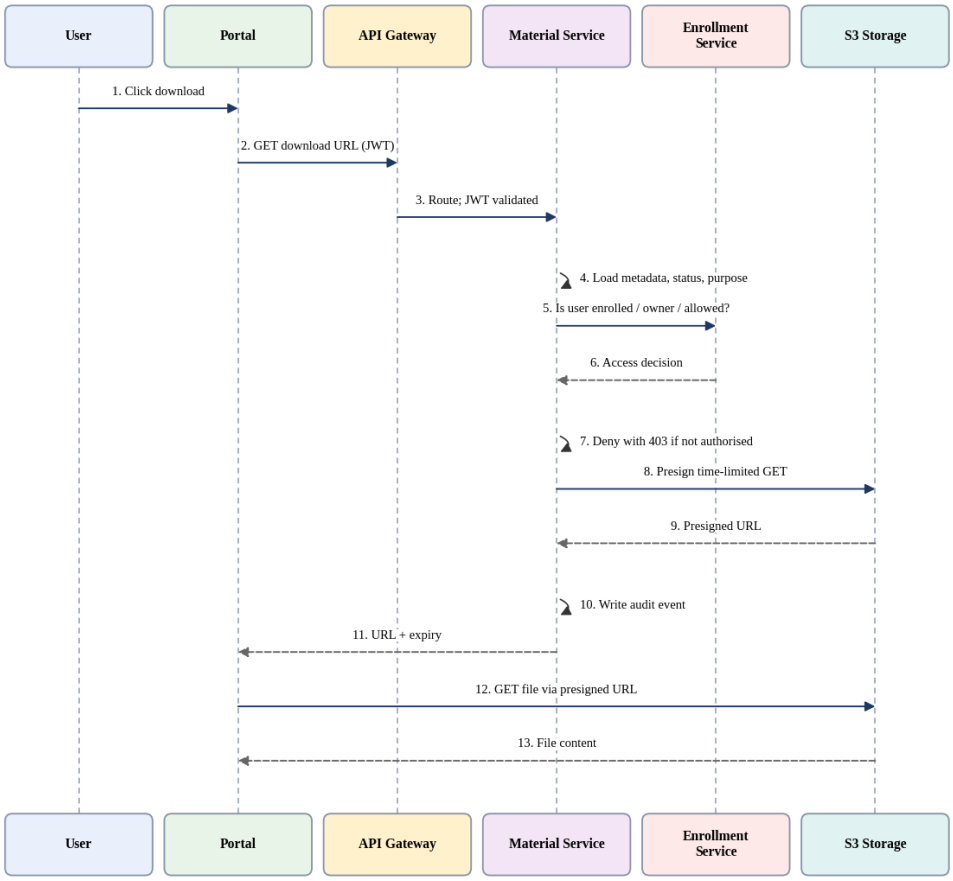


Figure 4.16: Sequence Diagram – Authorised Material Download

Table 4.34: Use Case Details – Password Recovery and First Login

Use Case ID	UC-17
Use Case Name	Password Recovery and First Login
Primary Actor	Any registered user
Supporting Actors	Portal, API Gateway, Auth Service, Mail (SMTP), Auth DB
Trigger	A user selects Forgot password, or signs in for the first time with temporary credentials.

4.17.0.2 Actors and System Components

Table 4.35: Actors and Components – Password Recovery and First Login

Participant	Type	Responsibility
User	Primary Actor	Requests recovery and sets a new password.
Portal (Next.js)	Frontend Component	Renders the role-specific screens, calls the REST or WebSocket APIs with the Bearer token, and presents validation and error feedback.
API Gateway	Infrastructure	Routes the request to the responsible service and applies the first-level CORS and JWT handling.
Auth Service	Application Server	Verifies credentials, issues and refreshes JWT access tokens, manages one-time passwords, and publishes the signing keys through JWKS.
Mail (SMTP)	External Service	Delivers invitations, one-time passwords, and notification e-mails.

PostgreSQL	Relational Database	Persists the owning service's data in its logical database.

4.17.0.3 Pre-conditions

● The user has a registered account and access to the registered e-mail inbox.

4.17.0.4 Main Success Flow

● The user requests recovery with the registered email.
● The Auth Service creates an OTP record and sends the OTP by e-mail; the response is generic.
● The user enters the OTP and the new password.
● The Auth Service validates the OTP for expiry and single use.
● It updates the password hash and revokes existing refresh tokens.
● For a first login the user signs in with the temporary password.
● The Auth Service returns mustChangePassword = true.
● The user changes the password and a normal session is issued.

4.17.0.5 Alternative and Exception Flows

An invalid, expired, or used OTP is rejected and a new one can be requested. An unknown email receives the same generic response to prevent enumeration.

4.17.0.6 Post-conditions

● The password hash is updated and previous sessions are revoked.
● The user can sign in with the new password.

4.17.0.7 Sequence Diagram

Figure 49 below illustrates the complete interaction sequence for the use case described above.

4.18 Calendar Projection and Reconciliation

This use case describes how academic events appear in the calendar. A source service publishes an event or calls the projection API; the Calendar Service maps it to a CalendarEvent keyed by the source type and identifier and upserts it. Failed projections are marked for

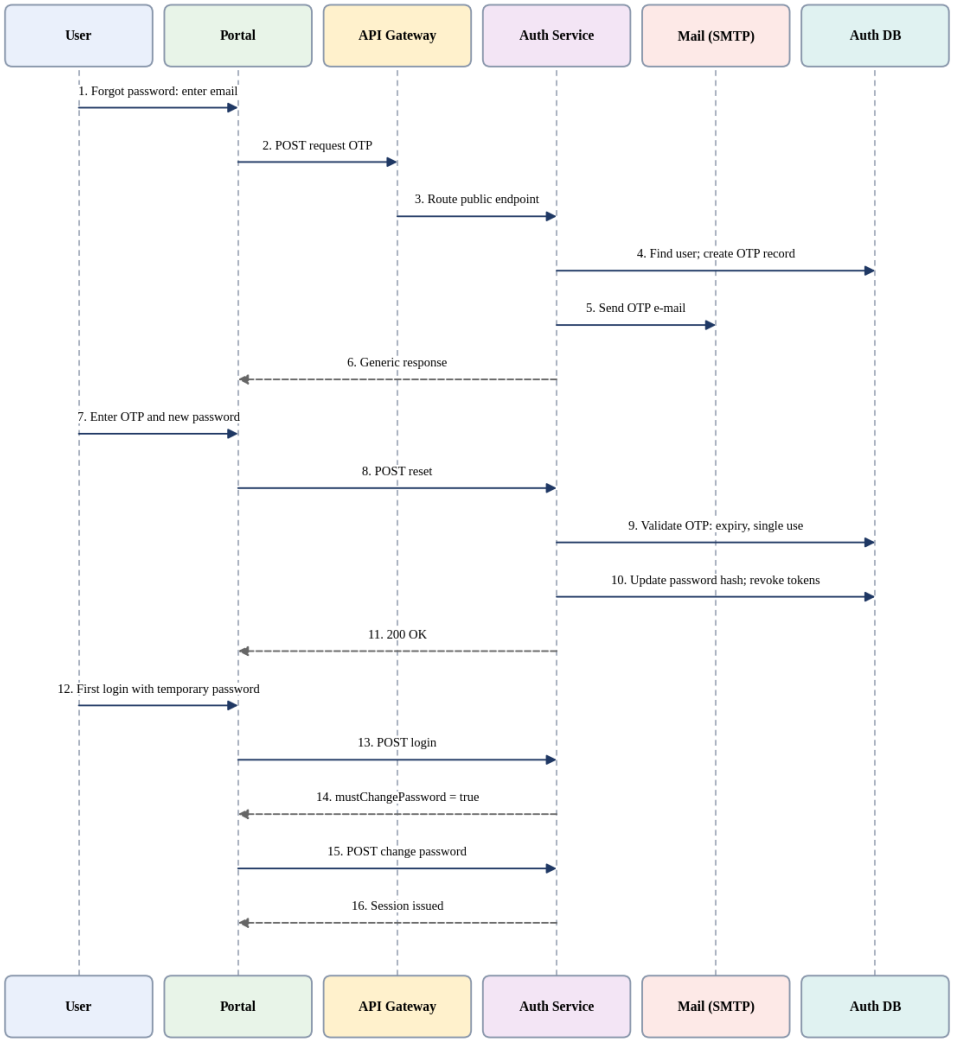


Figure 4.17: Sequence Diagram – Password Recovery and First Login

reconciliation and re-applied. Users query the calendar and see only the events permitted for their role.

4.18.0.1 Use Case Details

Table 4.36: Use Case Details – Calendar Projection and Reconciliation

Use Case ID	UC-18
Use Case Name	Calendar Projection and Reconciliation
Primary Actor	System (Calendar Service); users view the calendar
Supporting Actors	Source Services, Kafka / API, Calendar Service, Calendar DB, Student and Teacher Portals
Trigger	An exam, assignment, quiz, or attendance session is created or changed.

4.18.0.2 Actors and System Components

Table 4.37: Actors and Components – Calendar Projection and Reconciliation

Participant	Type	Responsibility
Source Service	Primary Actor	Publishes or calls the projection.
Kafka / API	Message Broker	Delivers the event.
Calendar Service	Application Server	Owns calendar events and projections and applies role-based visibility.
Calendar DB	Relational Database	Stores events and projections.
Student Portal	Frontend Component	Shows student-visible events.
Teacher Portal	Frontend Component	Shows course and personal events.

4.18.0.3 Pre-conditions

- The Calendar Service and database are operational.

4.18.0.4 Main Success Flow

- A source service records an exam, assignment, quiz, or attendance change.
- It publishes an event or calls the projection API.
- The Calendar Service validates and maps the event.
- It upserts the CalendarEvent by (sourceType, sourceId).
- On failure the projection is marked for reconciliation and later re-applied.
- Users request the calendar for a range.
- The service applies role-based visibility and returns the events.

4.18.0.5 Alternative and Exception Flows

Duplicate events upsert the same record. Cancelled sources set the projection to cancelled.

4.18.0.6 Post-conditions

- The calendar reflects source events and is eventually consistent with them.

4.18.0.7 Sequence Diagram

Figure 50 below illustrates the complete interaction sequence for the use case described above.

4.19 Event-Driven Notification Delivery

This use case describes notification delivery. After a successful operation the producer publishes a success event that avoids sensitive payload data. The Notification Service consumes it, skips already processed events where duplicate handling is implemented, creates notification records, attempts delivery on the supported channels, and records the delivery status. Users read and mark their own notifications. Push and SMS notifications, read receipts, and guaranteed delivery are not claimed.

4.19.0.1 Use Case Details

Table 4.38: Use Case Details – Event-Driven Notification Delivery

Use Case ID	UC-19
Use Case Name	Event-Driven Notification Delivery
Primary Actor	System (Notification Service); users read notifications

Supporting Actors	Producer Service, Kafka, Notification Service, Notification DB, Mail (SMTP), User Portal
Trigger	A producer service completes a business operation and publishes a success event.

4.19.0.2 Actors and System Components

Table 4.39: Actors and Components – Event-Driven Notification Delivery

Participant	Type	Responsibility
Producer Service	Primary Actor	Publishes success events.
Kafka	Message Broker	Carries asynchronous events between services.
Notification Service	Application Server	Consumes events, creates notifications, and delivers in-app and e-mail alerts.
Notification DB	Relational Database	Stores notifications and delivery status.
Mail (SMTP)	External Service	Delivers invitations, one-time passwords, and notification e-mails.
User Portal	Frontend Component	Shows and marks notifications.

4.19.0.3 Pre-conditions

- Kafka and the Notification Service are operational.

4.19.0.4 Main Success Flow

- A business operation succeeds in a producer service.
- The producer publishes an event to Kafka.
- The Notification Service consumes the event.
- It skips the event if already processed.
- It creates notification records for the recipients.
- It sends e-mail where configured and records the delivery result.
- A user requests own notifications and receives the list.
- The user marks notifications as read.

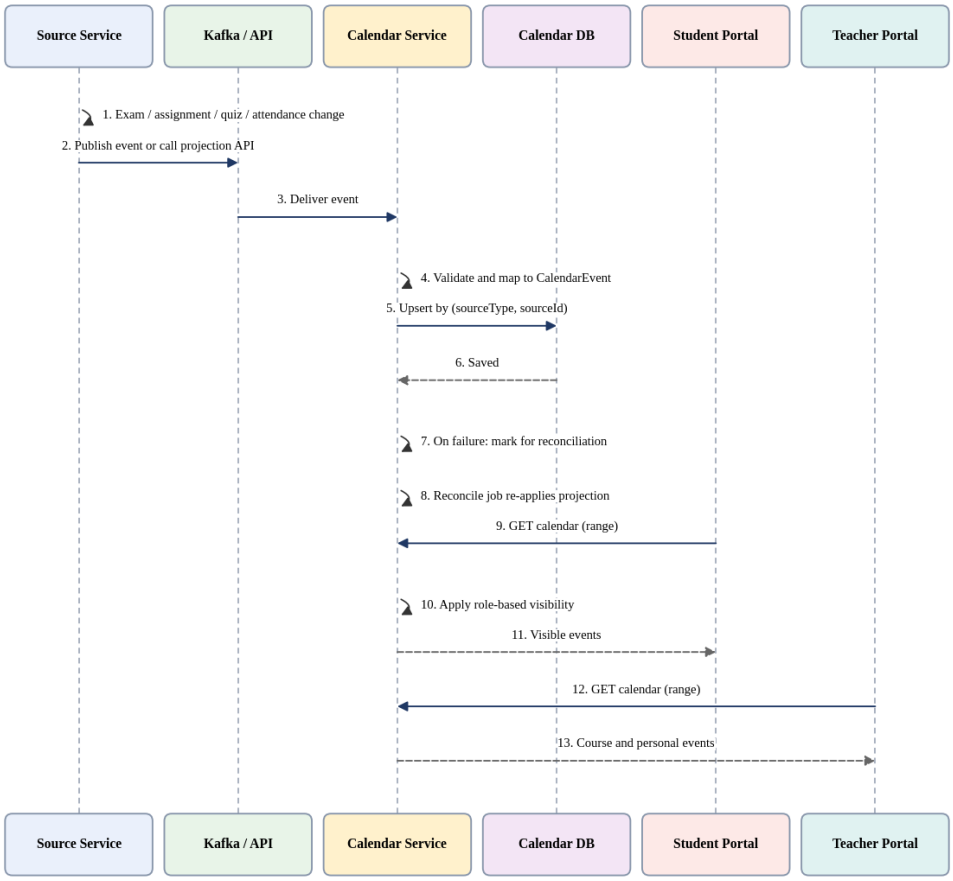


Figure 4.18: Sequence Diagram – Calendar Projection and Reconciliation

4.19.0.5 Alternative and Exception Flows

Consumer failures follow the configured retry behaviour. E-mail failures set a failed delivery status. Notification queries return only the requesting user's data.

4.19.0.6 Post-conditions

- Notifications and delivery statuses are persisted.

4.19.0.7 Sequence Diagram

Figure 51 below illustrates the complete interaction sequence for the use case described above.

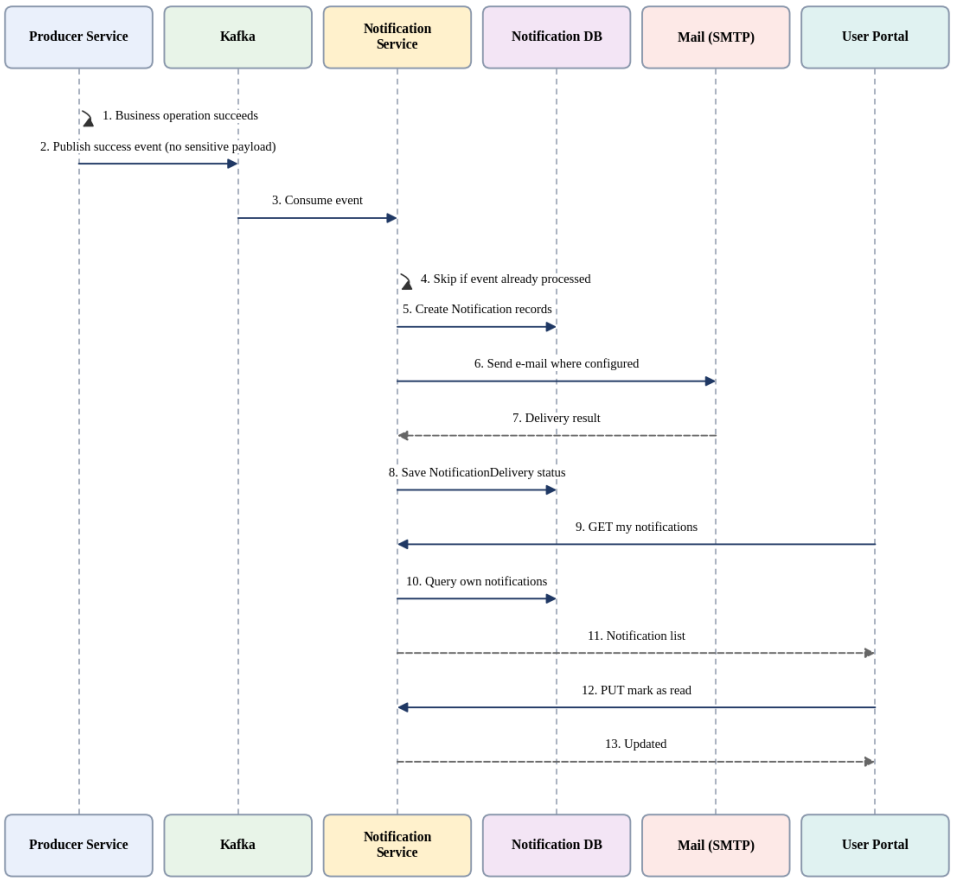


Figure 4.19: Sequence Diagram – Event-Driven Notification Delivery

Chapter 5

UI/UX Design

This chapter presents the user interface and user experience (UI/UX) design of the Mentify platform as implemented in the frontend repository. The design artefacts documented herein include screen wireframes for the key interfaces of each portal, user flow and navigation diagrams for primary system interactions, and an accessibility strategy aligned with the Web Content Accessibility Guidelines (WCAG) 2.1. Version 1.0 documented a single AI-themed student experience (Mentify AI and Cognitive Gallery); those mock-ups are replaced by the current portal structure.

5.1 Design Overview

The Mentify frontend consists of four separate web experiences built with React and Next.js: the Administration portal, the Teacher portal, the Student portal, and the public institute website. The portals share a role-based layout with a persistent navigation sidebar and header, feature-separated API-client modules, session restoration with protected access, form validation, loading states, and error feedback. The student portal switches between dashboard, courses, course workspace, enrollments, materials, assignments, quiz taking, examinations, grades, calendar, messages, notifications, profile, and settings views. Layouts are responsive for desktop and mobile.

The public website provides Home, About, Services, Contact, Pre-registration, Resources, Privacy Policy, and Terms of Use pages and carries English, Sinhala, and Tamil translation assets. This is website localisation only and is not evidence of complete translation of the portals. The website resource catalogue declares its entries to be samples and the public payment module states that no payment provider is connected, so no paid-material sales workflow is presented.

The following table summarises the screens documented in this chapter. The images are wireframes derived from the component structure of the frontend repository and use placeholder values; they must be replaced with screenshots of the running application before

final submission.

Table 5.1: Summary of Key User Interface Screens

No.	Screen Name	Description	Frontend Evidence
1	Administration Dashboard	Summary cards for users, courses, enrollments, and payments, pending approvals, and attendance and exam indicators.	admin/src/admin/dashboard/Admi
2	User Management (Admin)	Register users and administrators, resend invitations, and manage account status under the fixed role model.	admin: users/, students/, teachers/, administrators/
3	Enrollment Management (Admin)	Review enrollment requests with payment state and approve, reject, or create enrollments.	admin: enrollments/
4	Curriculum and Lesson Builder (Teacher)	Modules, lessons, material upload, and lesson publication.	teacher portal: curriculum views
5	Quiz Generator (Teacher)	Configure and generate an AI draft, edit it, save as draft, and publish.	teacher portal: quiz generator
6	Attendance Workspace (Teacher)	Create sessions, mark students, and complete sessions with a summary.	teacher portal: attendance workspace
7	Student Dashboard	Courses, deadlines, results, and attendance.	student/src/App.tsx (dashboard view)

8	Course Workspace and Materials (Student)	Course structure, lessons, and authorised material downloads.	student portal: course workspace, materials
9	Quiz Attempt (Student)	Timed single-answer multiple-choice quiz with attempt information.	student portal: quiz taking
10	Public Website – Pre-registration and Resources	Pre-registration form and the sample resource catalogue.	website: pre-registration, resources

5.2 Screen Wireframes for Key User Interfaces

The following subsections present wireframes for key interfaces of each portal. Each wireframe is accompanied by a descriptive annotation identifying its purpose, constituent components, and intended interactions.

5.2.0.1 Administration Dashboard

The Administration Dashboard is the landing screen for administrators. A left sidebar gives access to Users, Grades and Courses, Enrollments, Assignments, Quizzes, Examinations, Attendance, Materials, Marketplace, Analytics, the AI Assistant, and Calendar. Summary cards show users, courses, enrollments, and payments, a table lists pending enrollment requests with a Review action, and a panel shows upcoming examinations and attendance. Data is retrieved from the Analytics Service and domain services.

5.2.0.2 User Management – Administration

The user management screen lists accounts with name, email, role, status, and actions. Administrators register students and teachers and resend invitations; the control to register an administrator is available to the Super Admin only. Roles are limited to STUDENT, TEACHER, ADMIN, and SUPER_ADMIN, replacing the role-permission editor and AI role auditor of Version 1.0.

5.2.0.3 Enrollment Management – Administration

Administrators review enrollment requests with the student, course, source (self-enrollment or staff-created), payment state, and status (PENDING, APPROVED, REJECTED), and can

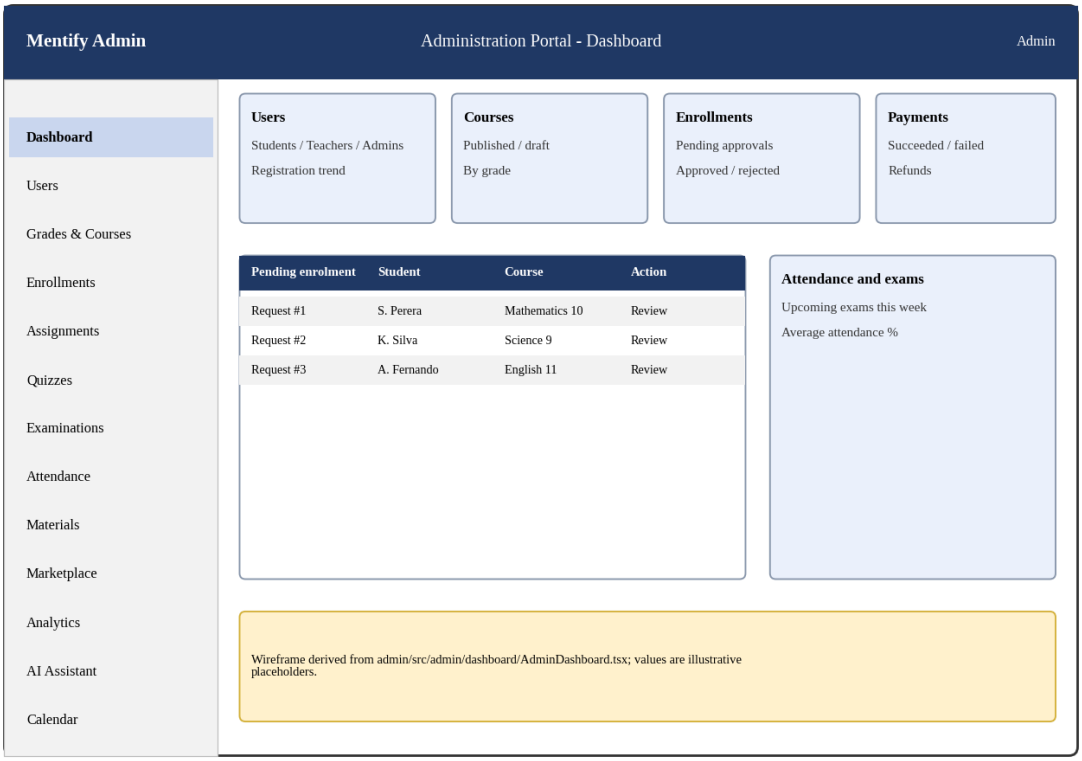


Figure 5.1: Administration Dashboard – Wireframe

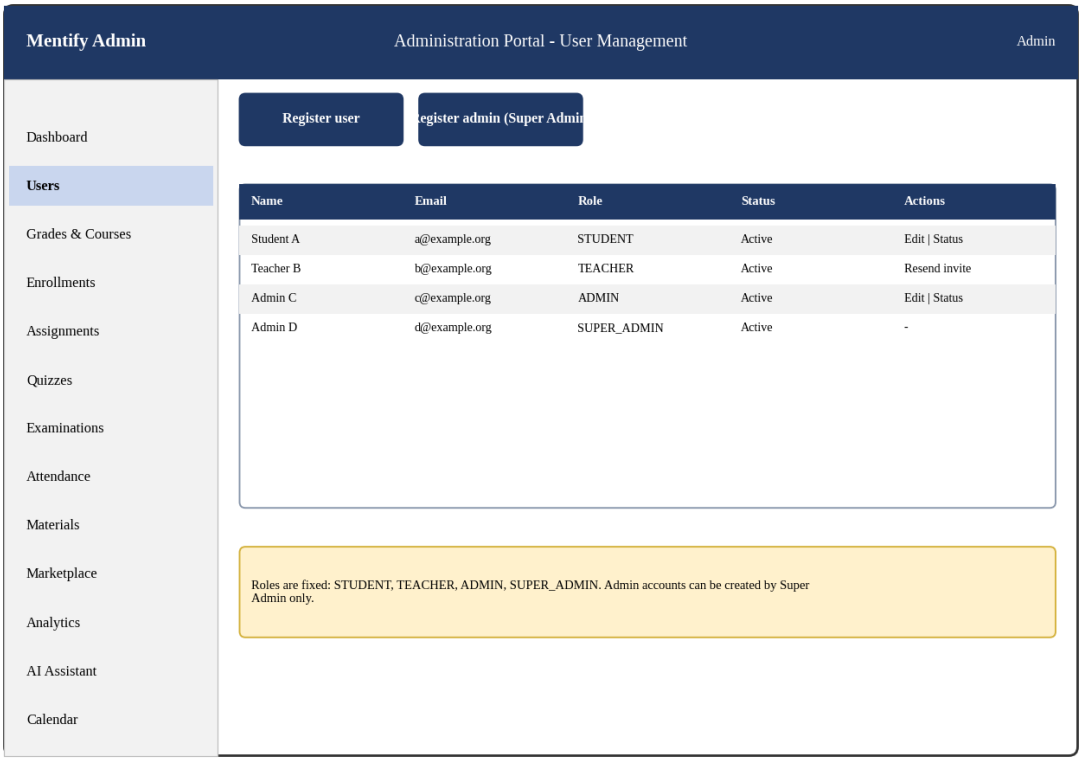


Figure 5.2: User Management – Wireframe

approve, reject, or create enrollments.

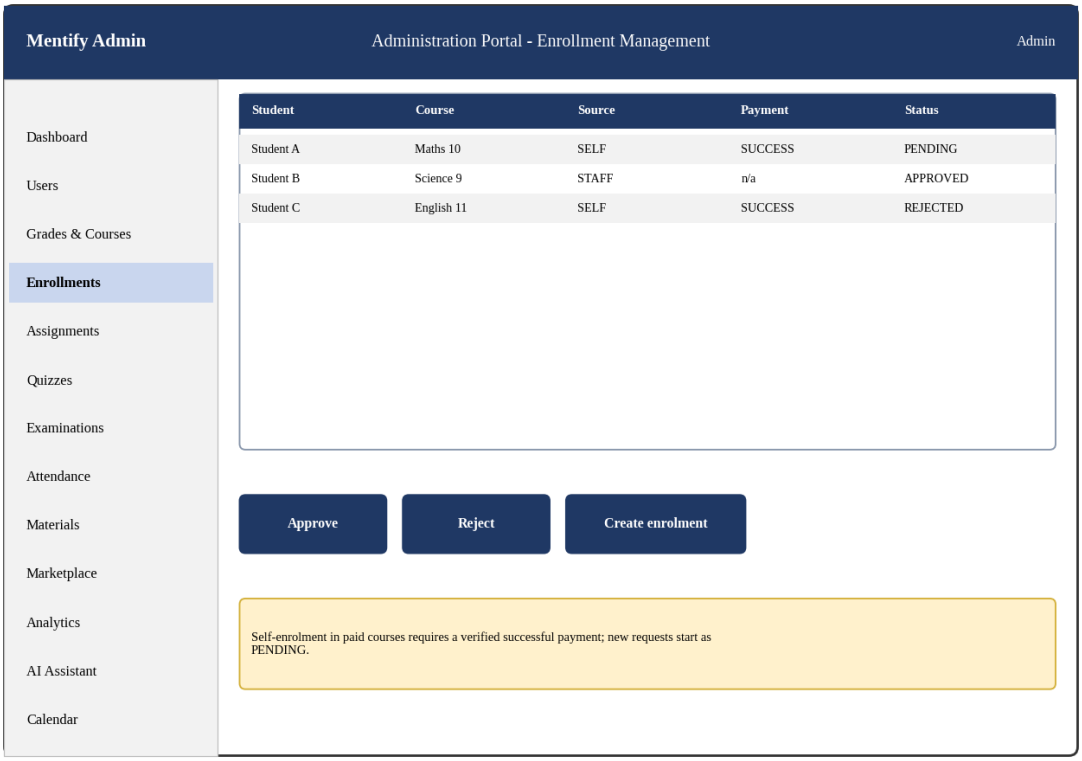


Figure 5.3: Enrollment Management – Wireframe

5.2.0.4 Curriculum and Lesson Builder – Teacher

Teachers organise a course into modules and lessons, edit lesson content, upload materials, and publish lessons. Uploads are sent to the Material Service and stored in object storage.

5.2.0.5 Quiz Generator – Teacher

The quiz generator lets a teacher choose a course, source material, number of questions, duration, and pass mark, and request an AI draft. The AI draft is clearly distinguished from a published assessment: the teacher reviews, edits, and saves it as a draft and then publishes it. Only single-answer multiple-choice questions are supported.

5.2.0.6 Attendance Workspace – Teacher

Teachers create attendance sessions, mark each student present, absent, or late, and complete the session. A summary panel presents the session attendance rate.

5.2.0.7 Student Dashboard

The Student Dashboard follows the views defined in the student application shell. Cards show enrolled courses (including pending enrollments), upcoming quiz and assignment

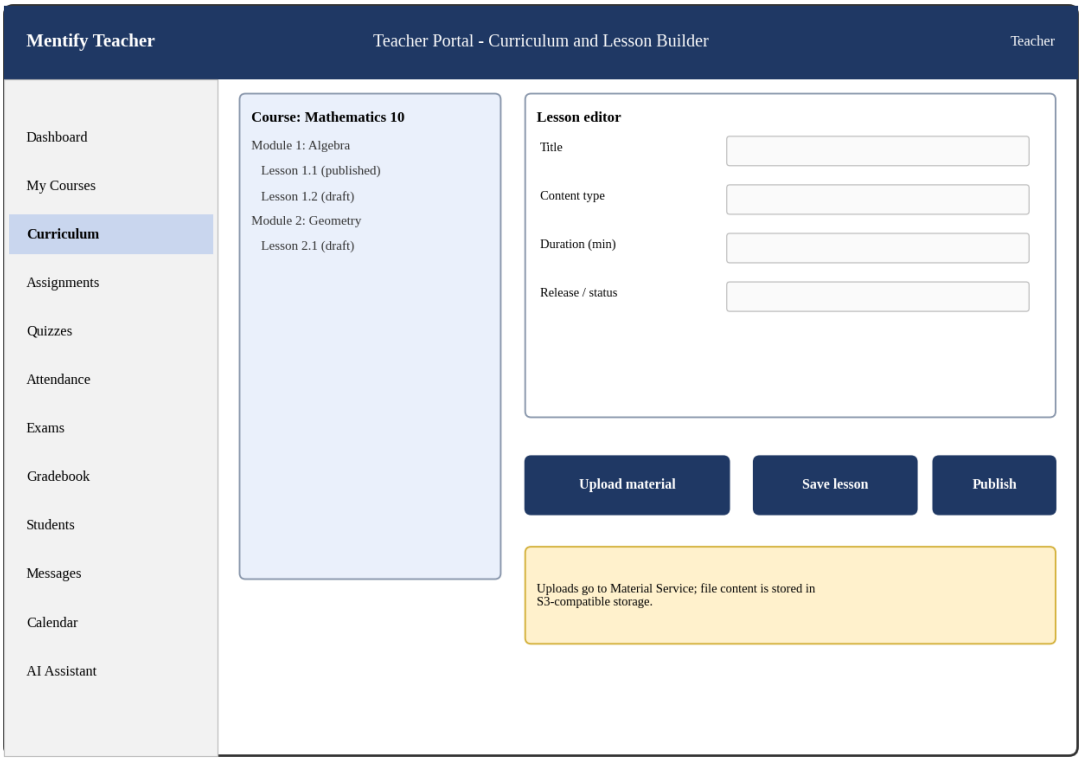


Figure 5.4: Curriculum and Lesson Builder – Wireframe

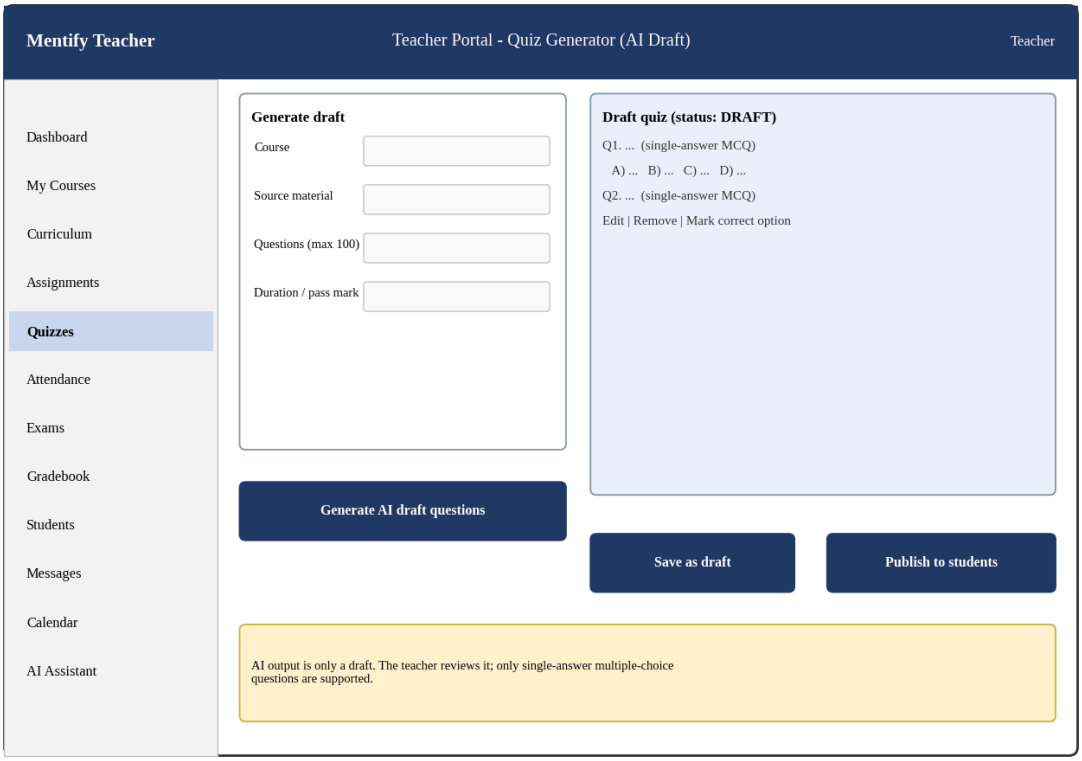


Figure 5.5: Quiz Generator (AI Draft) – Wireframe

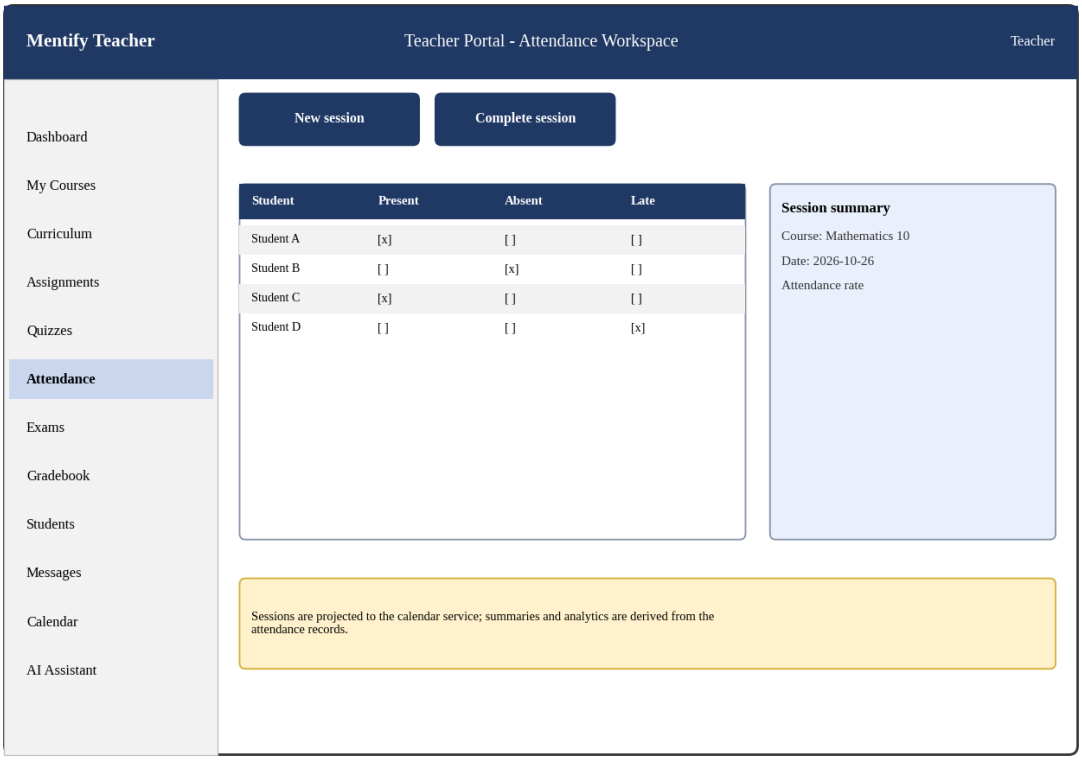


Figure 5.6: Attendance Workspace – Wireframe

deadlines, and latest results; a table lists assignments and an attendance panel shows overall attendance. Navigation covers courses, enrollments, materials, assignments, quizzes, examinations, grades, calendar, messages, notifications, and profile.

5.2.0.8 Course Workspace and Materials – Student

The course workspace shows the course structure, the selected lesson, and a materials table. Download actions request time-limited presigned URLs that are issued only after the Material Service authorises the user.

5.2.0.9 Quiz Attempt – Student

The quiz screen shows the question counter, remaining time, the question with single-answer options, and attempt information. Navigation controls and the Submit action are at the bottom. Hint panels described in Version 1.0 are not part of the reviewed implementation.

5.2.0.10 Public Website – Pre-registration and Resources

The public website offers a pre-registration form, language selection (English, Sinhala, Tamil), and a resources page. The resources entries are sample data and no payment provider is connected, so the page must not be read as a completed paid-resource storefront.

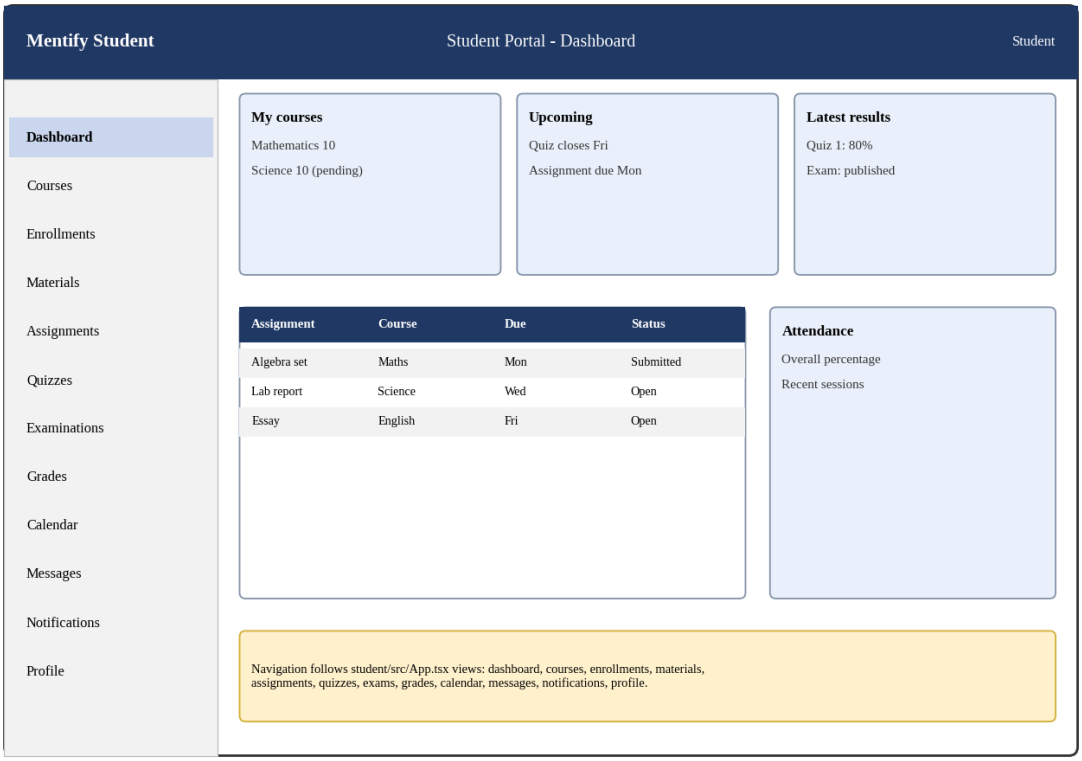


Figure 5.7: Student Dashboard – Wireframe

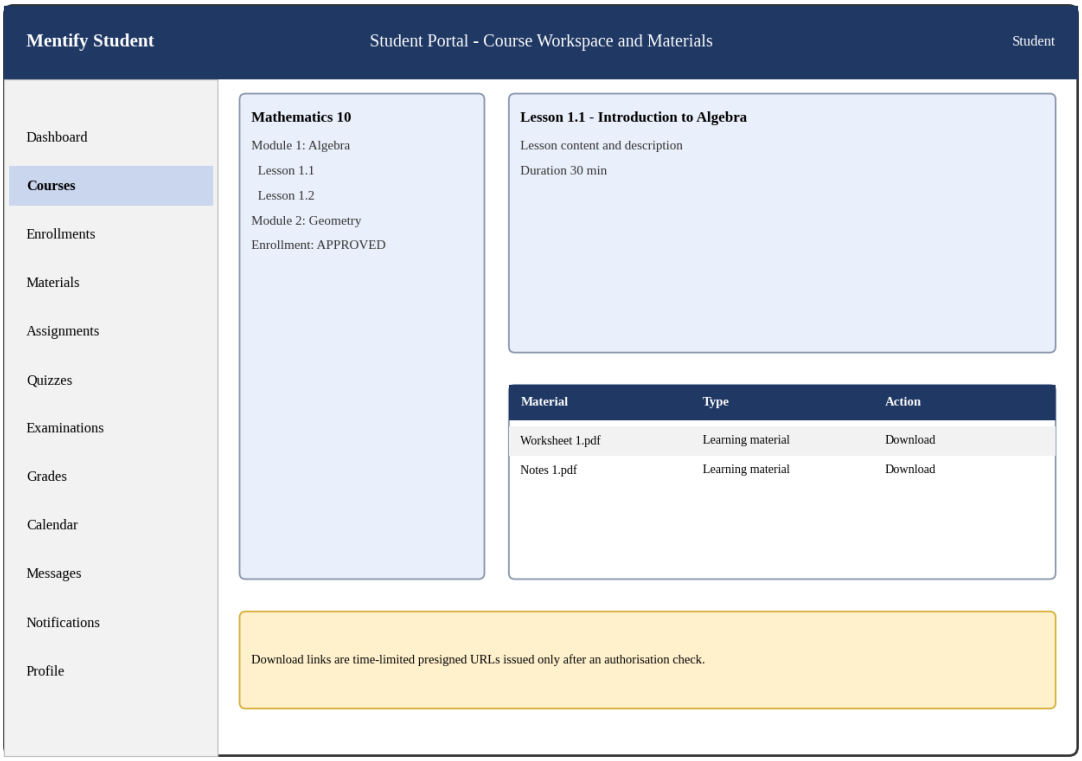


Figure 5.8: Course Workspace and Materials – Wireframe

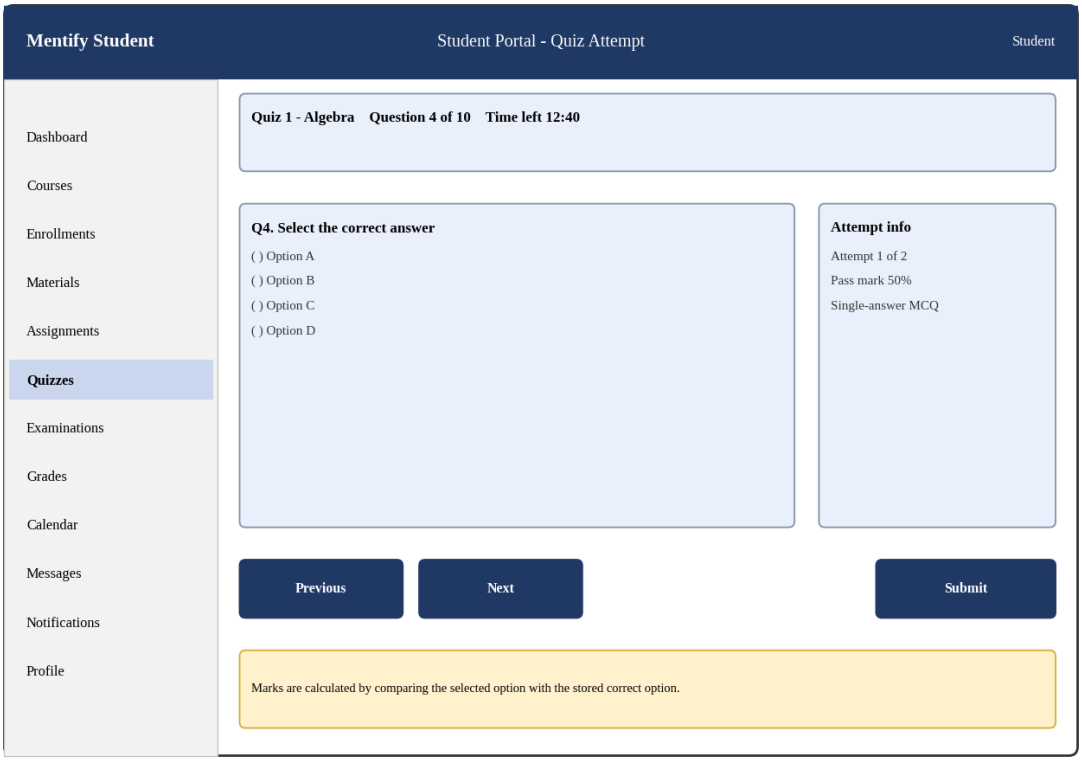


Figure 5.9: Quiz Attempt – Wireframe

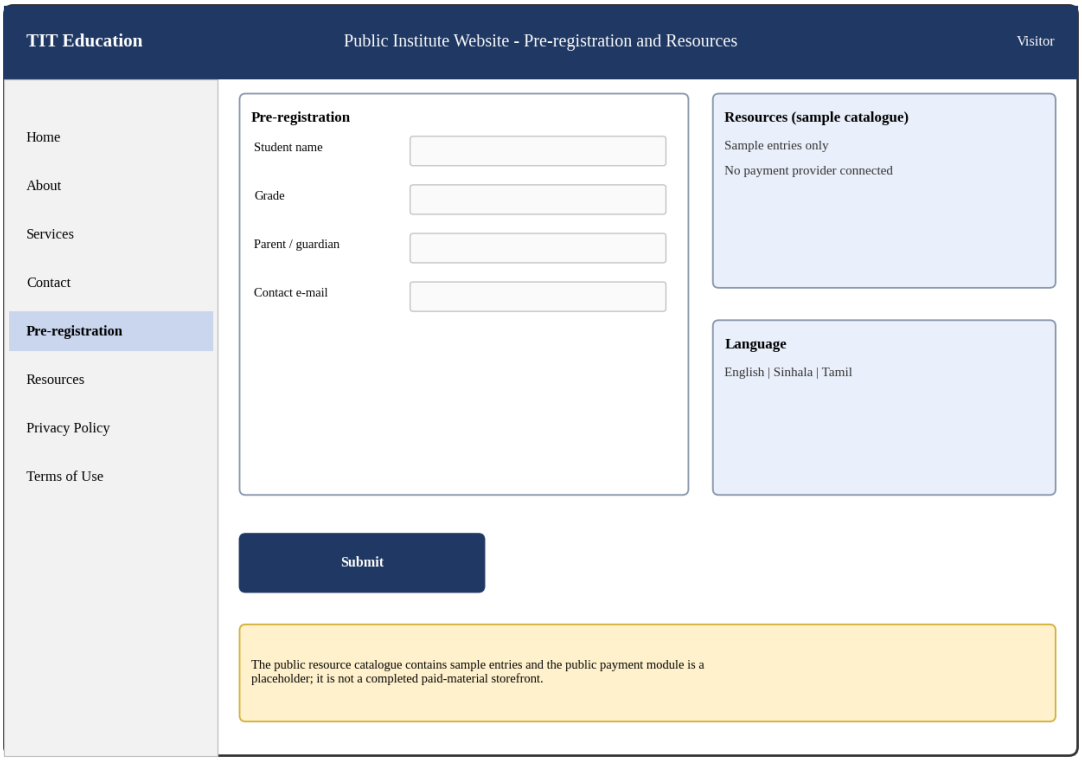


Figure 5.10: Public Website – Wireframe

5.3 User Flow and Navigation Diagrams

This section presents user flow diagrams for five primary interactions: login, enrollment, course management, group messaging access, and public pre-registration. Each diagram models the steps, decision points, and system responses that govern a user's path and informs both the frontend navigation and the backend rules.

Table 5.2: Summary of User Flow Diagrams

User Flow	Flow Description	Reference
User Login	The user enters credentials; on failure the forgot-password flow sends a one-time password by e-mail. A user with temporary credentials must change the password at first login before reaching the role portal.	Figure 62
Course Enrolment	The student explores courses, requests enrollment, and pays for a paid course first. The enrollment is created as PENDING and approved or rejected by authorised staff.	Figure 63
Course Management	An administrator or assigned teacher creates or edits a course, adds modules and lessons, and publishes; deletion is restricted to administrators.	Figure 64
Group Messaging Access	Authenticated members of a course group (approved enrollment or assigned staff) read and send messages in real time.	Figure 65

Public Pre-registration	A visitor browses the public website and submits a pre-registration form; staff follow up.	Figure 66

5.3.0.1 User Login Flow

The login flow governs authentication for all roles. The user submits an email and password. If they are invalid, the user may retry or start the forgot-password process, in which a one-time password is e-mailed and validated for expiry and single use before a new password is set. If the credentials are valid and the account requires a password change, the portal forces the change before opening the role-specific portal.

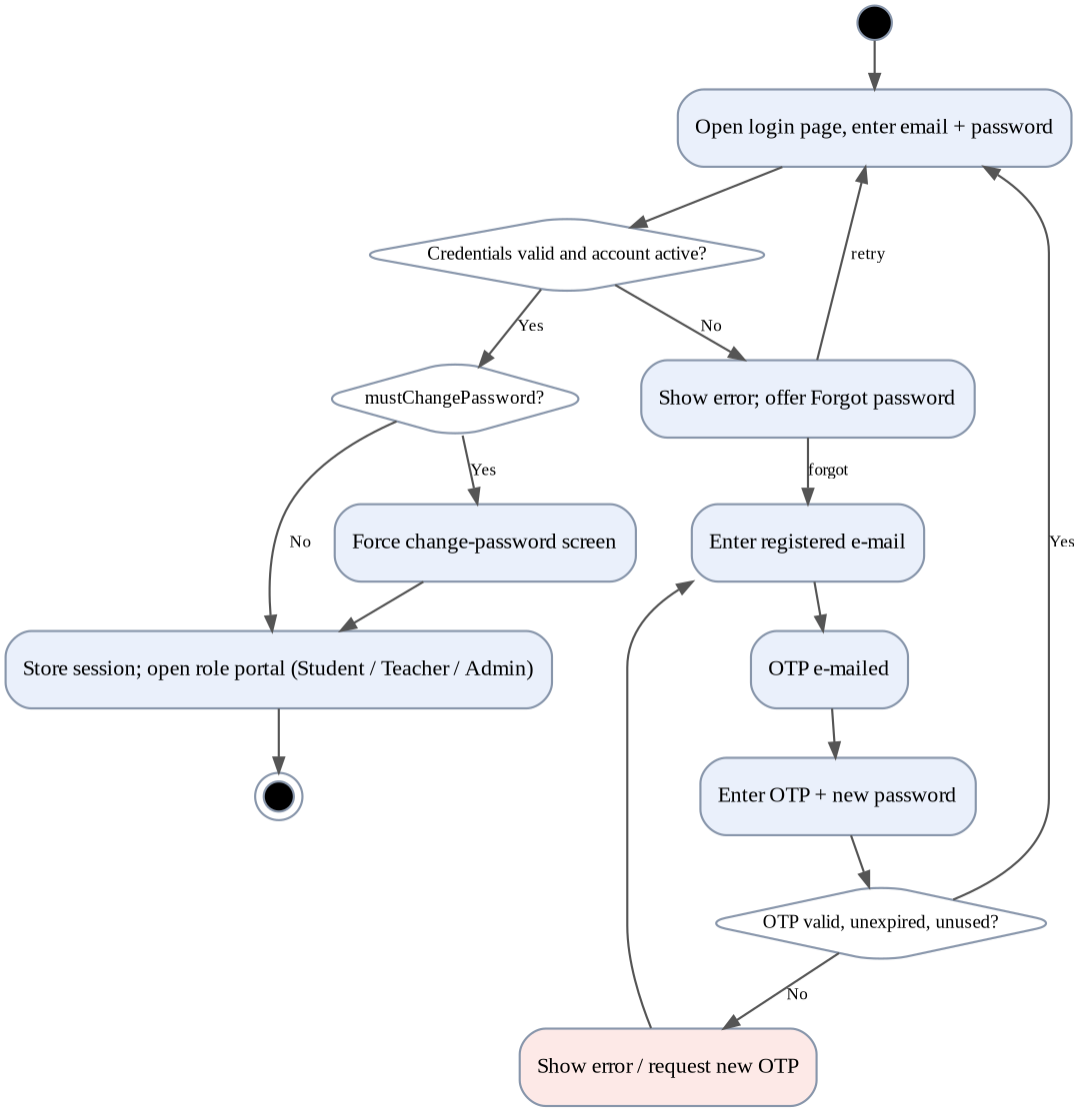


Figure 5.11: User Login Flow Diagram

5.3.0.2 Course Enrolment Flow

The enrollment flow begins when the student opens the course list and selects a course. The system checks for an existing enrollment. For a paid course the student first pays through Stripe; only a successful payment allows the request to proceed. The enrollment is created with status PENDING, and a teacher assigned to the course or an administrator approves or rejects it. Version 1.0 opened a payment portal after teacher approval; the implementation verifies payment before the PENDING enrollment is created.

5.3.0.3 Course Management Flow

An administrator or assigned teacher opens Courses and chooses to add a new course or edit an existing one. The form requires the mandatory attributes and is validated and authorised; modules, lessons, and materials are added and the course is published. Version 1.0 required an administrator review (Send for Review) before publication; the implementation applies role-based authorisation instead, and only Admin and Super Admin may delete a course.

5.3.0.4 Group Messaging Access Flow

Access to group messaging follows enrollment. An authenticated user opens a course group; membership is established by approved enrollment or staff assignment, and non-members cannot see the group. Members connect over WebSocket with their JWT, read history, and exchange messages and notifications. Version 1.0 forums with participation requests are not implemented.

5.3.0.5 Public Pre-registration Flow

Visitors browse the public website in their chosen language and submit the pre-registration form. After client-side validation the submission is handled by the website workflow, and staff follow up to provision a student account.

5.4 Accessibility Considerations

The platform design aims to be inclusive and accessible to users with visual, auditory, motor, and cognitive impairments. The accessibility strategy is aligned with WCAG 2.1 and targets Level AA. This section describes design intent; it does not assert verified conformance. The product must not be labelled WCAG 2.1 AA compliant until audit and assistive-technology test evidence exists.

The WCAG guidelines are organised under the principles Perceivable, Operable, Understandable, and Robust (POUR). The following subsections describe how each principle is addressed by the design intent.

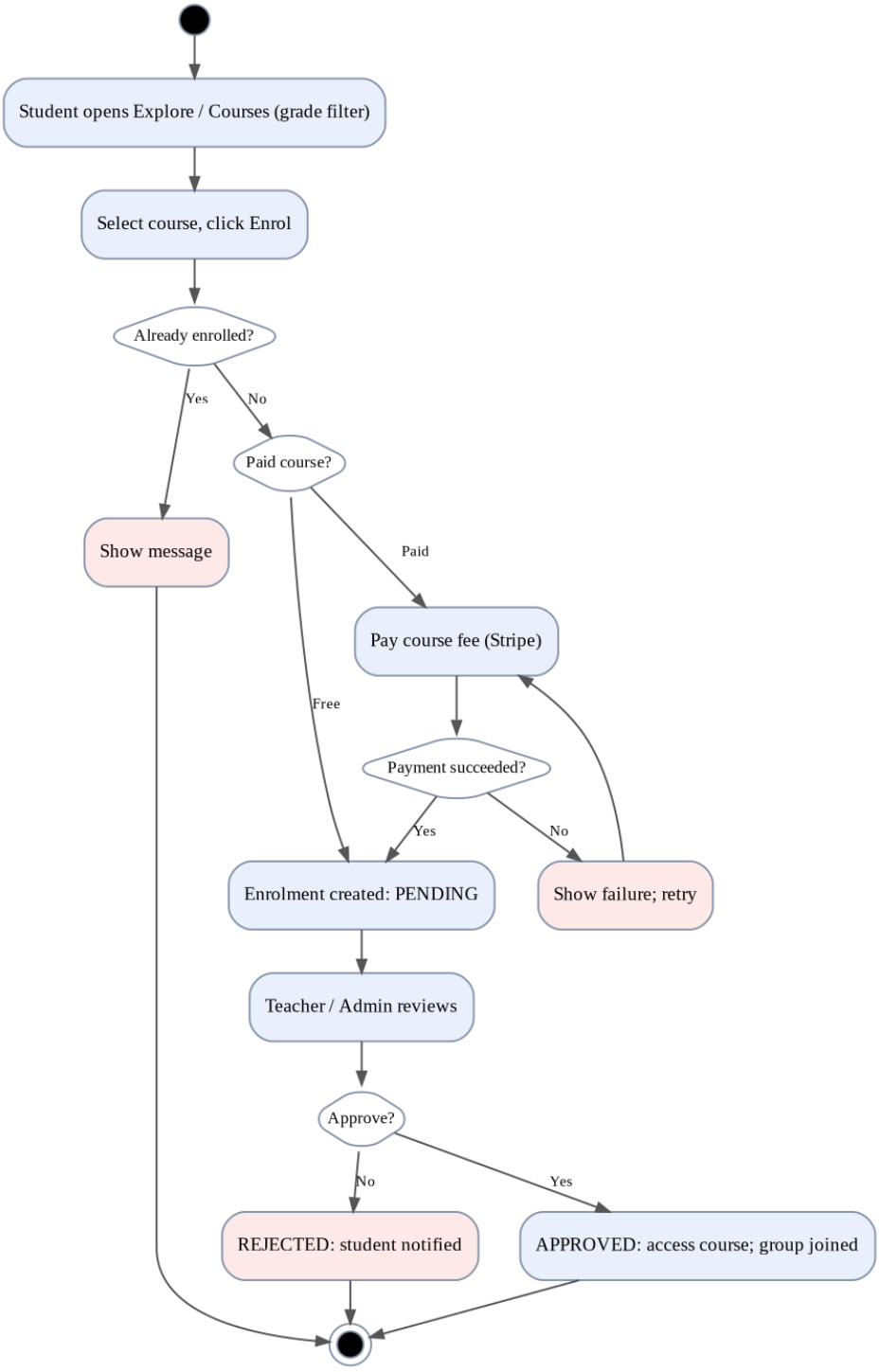


Figure 5.12: Course Enrolment Flow Diagram

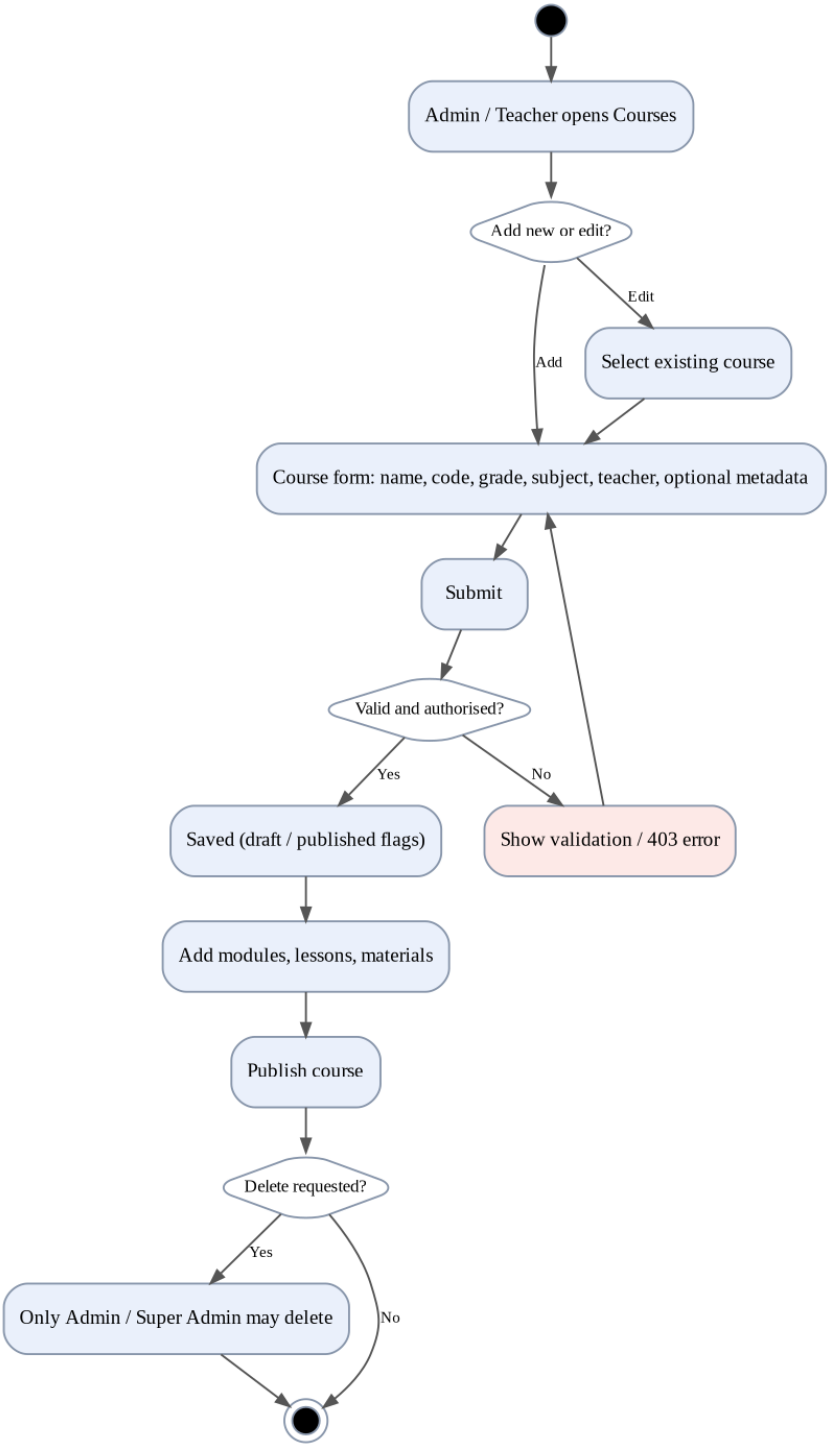


Figure 5.13: Course Management Flow Diagram

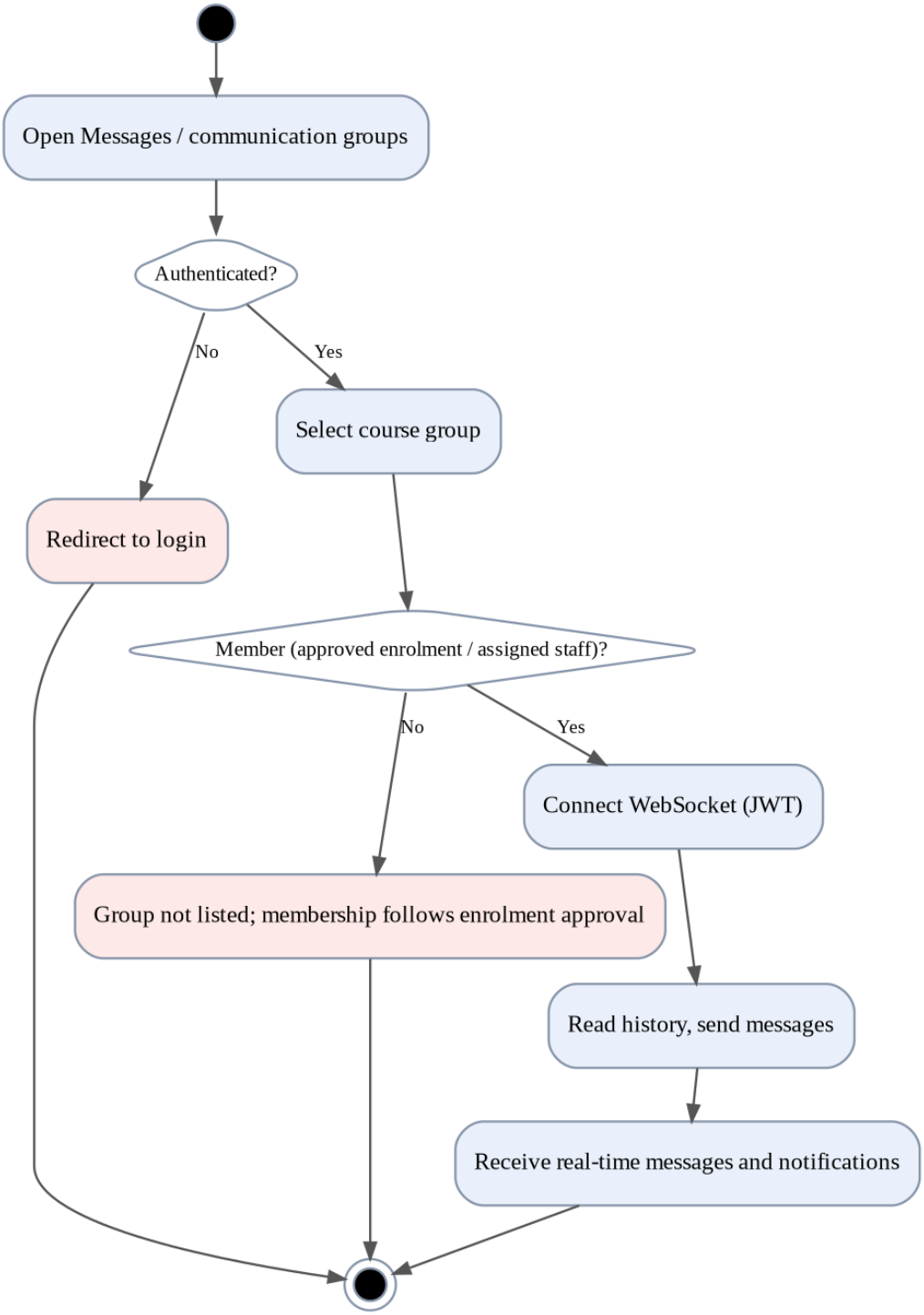


Figure 5.14: Group Messaging Access Flow Diagram

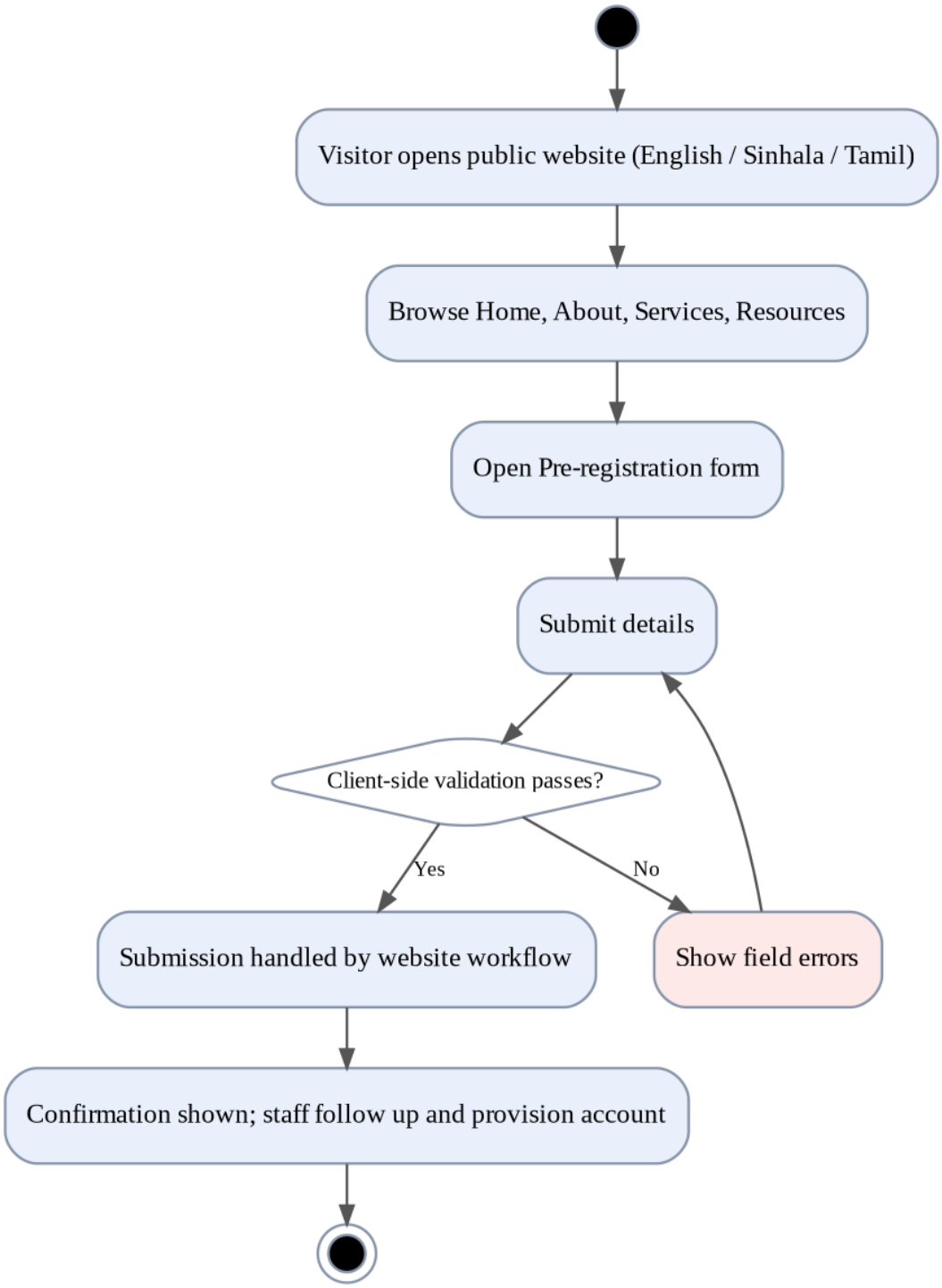


Figure 5.15: Public Pre-registration Flow Diagram

5.4.0.1 Perceivable

Design intent: non-text content such as icons and course imagery carries descriptive alternative text, and the colour palette targets the minimum contrast ratios of WCAG 2.1 criterion 1.4.3 (4.5:1 for normal text, 3:1 for large text). Status information such as enrollment status, payment status, and grading state is conveyed through both colour and text labels. Text uses relative units and is intended to remain legible at 200% zoom.

5.4.0.2 Operable

Design intent: interactive components, including navigation, form controls, dialogs, and action buttons, are operable by keyboard, with a logical tab order and visible focus indicators. The quiz timer and complex widgets are to be keyboard accessible. Content must not flash at frequencies known to induce photosensitive responses.

5.4.0.3 Understandable

Design intent: the page language is declared, form inputs use explicit labels, and inline validation messages are shown next to the relevant field in plain language with recovery guidance. Navigation is consistent across screens within each portal.

5.4.0.4 Robust

Design intent: components use valid semantic HTML and ARIA roles, states, and properties (for example aria-expanded, aria-selected, aria-live). Automated accessibility audits and manual screen-reader checks are planned test activities; the test repository does not currently record accessibility results, so they are not claimed as completed.

The following table summarises the WCAG 2.1 criteria targeted by the design and the intended implementation approach for each. Each entry is design intent pending verification.

Table 5.3: WCAG 2.1 Accessibility Design Intent Summary

WCAG 2.1 Criterion	Conformance Level	Implementation Approach
1.1.1 Non-text Content	Level A	Design intent: Images include descriptive alt text; icons and buttons carry accessible labels.
1.3.1 Info and Relationships	Level A	Design intent: Semantic HTML; tables use headers and ARIA roles.

1.4.1 Use of Colour	Level A	Design intent: Colour is not the only means of conveying information; status also uses text.
1.4.3 Contrast (Minimum)	Level AA	Design intent: 4.5:1 for normal text and 3:1 for large text.
1.4.4 Resize Text	Level AA	Design intent: Text resizable to 200% without loss of content or function.
1.4.10 Reflow	Level AA	Design intent: Content reflows without horizontal scrolling at 320px width.
2.1.1 Keyboard	Level A	Design intent: All interactive elements operable by keyboard.
2.4.3 Focus Order	Level A	Design intent: Focus follows a meaningful sequence consistent with visual layout.
2.4.4 Link Purpose	Level A	Design intent: Link text is descriptive out of context.
2.4.7 Focus Visible	Level AA	Design intent: A visible focus indicator for all keyboard-navigable elements.
3.1.1 Language of Page	Level A	Design intent: Primary document language is programmatically identified.
3.3.1 Error Identification	Level A	Design intent: Input errors are identified and described in text.
3.3.2 Labels or Instructions	Level A	Design intent: Labels are provided for all input fields.

4.1.2 Name, Role, Value	Level A	Design intent: ARIA attributes expose name, role, and state of custom components.

Verification of these criteria through automated audits, assistive-technology testing, and user testing remains to be performed and recorded. Until then, the table above records design intent only.

Chapter 6

Security Design

This chapter presents the security design of the system as implemented, encompassing the authentication mechanism, the authorisation framework, the data protection strategy, input validation and sanitisation, and encryption. The design is aligned with the OWASP Top 10 and with TLS and AES-256 as targets. Where a control is a deployment target rather than evidenced in the reviewed configuration, it is stated as such so that the document does not overstate the implemented security posture.

The overall security architecture is illustrated in Figure 67, which depicts authentication and authorisation for client requests, the API Gateway as the first security boundary, the first-party Auth Service as identity provider, and the second authorisation layer inside each business service.

6.1 Authentication Mechanism

Authentication verifies the identity of a user before access to protected resources is granted. The system uses a first-party Authentication Service that issues and validates JSON Web Tokens (JWT). Google OAuth 2.0 social login, described in Version 1.0, is not part of the implemented architecture and is recorded as a future enhancement only.

6.1.0.1 JSON Web Token (JWT) Authentication

Upon successful credential verification, the Auth Service issues a signed JWT access token and a refresh token to the client. The following lifecycle governs JWT usage:

• The client submits credentials to the API Gateway, which routes the public authentication endpoint to the Auth Service.

• The Auth Service verifies the credentials and account status against its own credential records.

• It issues a signed access token encoding the user identity and role, and a refresh token, according to the signing configuration (algorithm, issuer, audience, expiry, and claims are

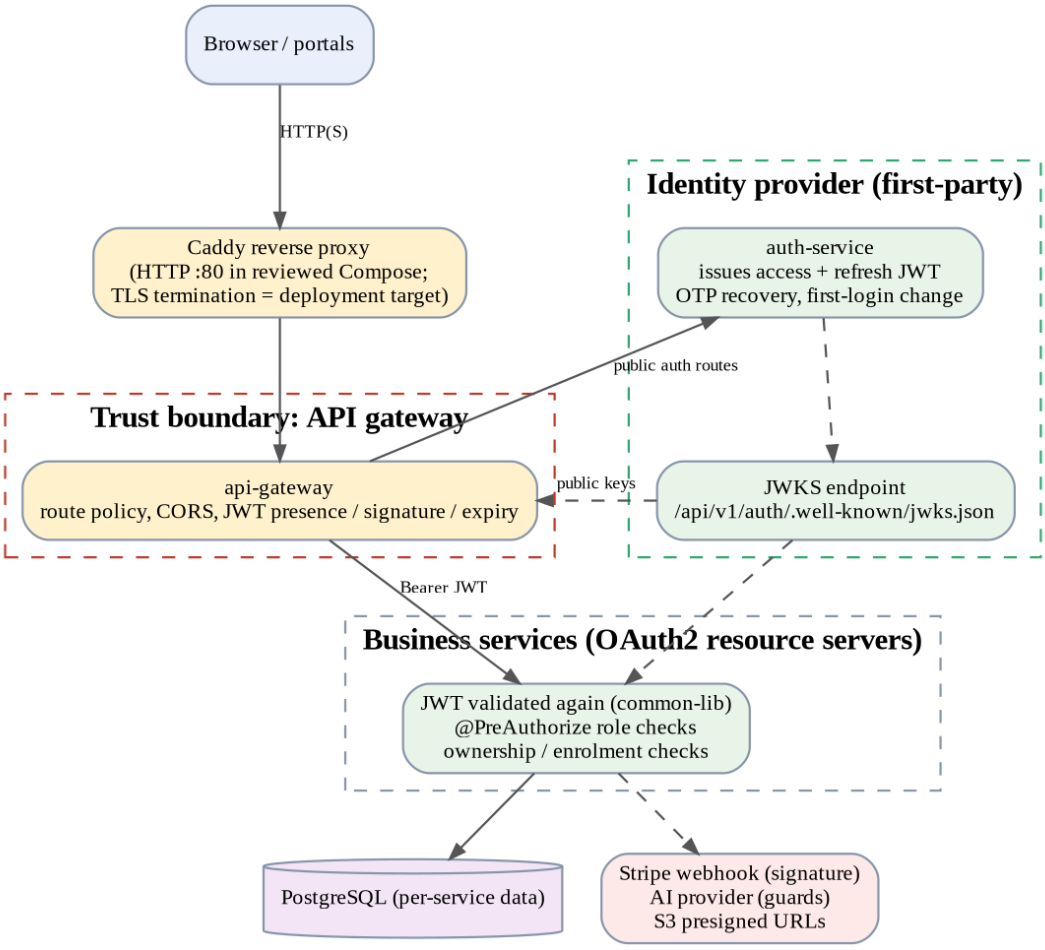


Figure 6.1: Security Architecture Diagram

defined in the auth-service configuration).

• The public verification keys are published at /api/v1/auth/.well-known/jwks.json (JWKS).

• The client sends the access token in the Authorization header on subsequent requests.

• The API Gateway validates the token, and each business service validates it again as an OAuth2 resource server before applying role and ownership checks.

Access tokens are short-lived to limit the exposure window; the exact expiry values are configuration parameters and must be quoted from the deployed configuration. The refresh token is used to obtain a new access token without re-entering credentials and is validated on the server, with revocation behaviour limited to what the auth-service implements.

6.1.0.2 Credential Lifecycle: First Login, Recovery, and Logout

Accounts are provisioned by administrators with temporary credentials delivered by e-mail invitation. At first login the Auth Service returns a mustChangePassword indicator and the portal forces a password change. Forgotten passwords are recovered through a one-time password sent by e-mail; the OTP is validated for expiry and single use before the new password hash is stored. The OTP lifetime and attempt rules are to be confirmed against the implementation. Logout clears the client session and ends the use of the refresh token according to the auth-service behaviour.

Table 6.1: Authentication Token Summary

Token / Credential	Purpose	Storage and Handling
JWT access token	Authorises API requests	Held by the portal session; sent as Bearer token; validated at gateway and service; expiry per configuration
Refresh token	Obtains new access tokens	Issued by the Auth Service and validated server-side; revocation per implementation
JWKS public keys	Verification of token signatures	Published at /api/v1/auth/.well-known/jwks.json
One-time password (OTP)	Password recovery	Sent by e-mail; stored hashed with expiry and single-use state (to be confirmed)

Temporary password	First login	Provisioned by administrators; must be changed at first login

6.2 Authorisation Levels

Authorisation determines the actions an authenticated user may perform. The system implements Role-Based Access Control (RBAC) combined with resource-ownership checks. The role is carried in the validated token, and enforcement does not rely on frontend route visibility.

6.2.0.1 Role Definitions

The role enumeration defines four roles. The Instructor and Guest roles and the SupportAgent role of Version 1.0 are replaced by the roles below; unauthenticated visitors can only reach public routes such as the public website and the published marketplace catalogue.

Table 6.2: Role-Based Access Control Definitions

Role	Description	Permitted Actions (Domain of Access)
SUPER_ADMIN	Highest-level administrative authority.	Administrator operations plus privileged operations such as registering administrator accounts, subject to endpoint restrictions.
ADMIN	Institution administrator.	User management, grades and fees, course management and deletion, enrollment oversight, payment monitoring and refunds, marketplace products, and administrative analytics.

TEACHER	Academic staff.	Assigned courses and students, curriculum and materials, assignments and grading, quizzes, attendance, exams, communication, and AI chat.
STUDENT	Enrolled learner.	Own profile, authorised enrolled course content, own submissions, quiz attempts, grades, payments, attendance, and results.

6.2.0.2 Authorisation Enforcement

Authorisation is enforced at three levels:

• Gateway Level: The API Gateway applies routing rules and CORS and rejects requests without a valid token on protected routes with HTTP 401.

• Service Level: Each business service validates the JWT and applies method-level security (for example @PreAuthorize with hasAnyRole), providing a second layer even if the gateway is bypassed.

• Ownership Level: Services additionally check relationships: a teacher is limited to assigned courses, a student to approved enrollments and own records and payments, and file downloads to authorised owners, students, teachers, and administrators.

Representative restrictions are summarised below. A complete authorisation matrix for every endpoint must be generated from the controller annotations.

Table 6.3: Representative Authorisation Matrix

Operation	SUPER_ADMIN	ADMIN	TEACHER	STUDENT
Register administrator account	Yes	No	No	No
Register student / teacher	Yes	Yes	No	No
Delete course	Yes	Yes	No	No
Create / update course	Yes	Yes	Assigned / relevant	No
Update enrollment status	Yes	Yes	Assigned courses	No

Self-enrol in course	No	No	No	Yes
View own payments / initiate payment	No	No	No	Yes
View and filter all payments; refund	Yes	Yes	No	No
Grade assignment submissions	No	No	Assigned courses	No
Conversational AI chat (REST / WebSocket)	Yes	Yes	Yes	No
AI lesson / material summarisation	Yes	Yes	Yes	Yes
Manage marketplace products	Yes	Yes	No	No

The principle of least privilege is applied throughout; each role is granted only the permissions needed to perform its functions.

6.3 Data Protection Approach

Data protection encompasses the measures that safeguard the confidentiality, integrity, and availability of data. The following strategies are applied at each tier.

6.3.0.1 Password and Credential Management

• Passwords are never stored in plaintext; they are hashed (BCrypt) by the Auth Service, which is the only owner of credential data. The work factor must be confirmed against the implementation; the project target is a minimum of 12.
• Credentials are held by the Auth Service and profile data by the User Service; passwords are not stored in profile tables.
• Password recovery uses one-time passwords with expiry and single-use state, and administrators provision temporary passwords that must be changed at first login.
• Rate limiting and account locking on repeated failed logins are targets that must be verified in the implementation before being claimed.

6.3.0.2 Session Management

• Sessions are represented by short-lived access tokens and server-validated refresh tokens.
• Logout clears the client session and ends refresh-token use according to the implementation.
• WebSocket connections authenticate with the JWT and are bound to the user principal for user-specific message delivery.
• Storage of tokens in Redis, concurrent session limits, and anomaly-based invalidation described in Version 1.0 are not established and are not claimed.

6.3.0.3 Sensitive Data Classification

Data is classified into three sensitivity tiers with corresponding controls.

Table 6.4: Data Sensitivity Classification and Controls

Classification	Examples	Protection Controls
Highly Sensitive	Password hashes, OTPs, refresh tokens, signing keys, Stripe and AI provider secrets	Hashing or secret storage; never logged or returned; supplied through runtime configuration; log redaction.
Sensitive	User PII, grades, results, attendance, payment records, AI conversations	Role and ownership checks; TLS in transit (deployment target); encryption at rest where the environment provides it.
General	Course metadata, published marketplace products, public website content	Integrity checks; audit trail where implemented.

6.4 Input Validation and Sanitisation Strategy

All data from external sources is treated as untrusted and validated before processing or persistence, mitigating injection, cross-site scripting, and other input-borne vulnerabilities from the OWASP Top 10.

6.4.0.1 Validation Layer

Validation is applied in the service layer using Bean Validation constraints on request DTOs and in the frontend forms. Examples evidenced in the DTOs include: course name, descrip-

tion, grade, assigned teacher, and subject being required; assignment titles limited to 150 characters, maximum marks and allowed attempts of at least 1, late penalty between 0 and 100 percent, and at most 50 rubric items; and quiz creation requests limited to 100 questions. Further rules cover type and format validation, whitelists for enumerations (such as roles, enrollment statuses, exam types, and question types), and rejection of invalid values with HTTP 400. Business-rule violations such as late submission when not allowed or exhausted attempts return HTTP 409.

6.4.0.2 Sanitisation Layer

• SQL Injection Prevention: Database access uses Spring Data JPA and parameterised queries.
• File Upload Controls: Uploads are validated for configured types and size, stored under generated object keys, and downloaded only through authorised presigned URLs. Inline malware scanning described in Version 1.0 is not established.
• Payment Webhook Authenticity: Stripe webhook payloads are accepted only after signature verification, and processed event identifiers prevent replay.
• AI Guardrails: AI requests pass usage limits, input validation, content filtering, and prompt-injection checks; tools enforce the requester's data scope.
• Output and Browser Controls: Output encoding, CORS configuration, and security headers are applied; the exact header set must be confirmed in the gateway configuration.

Table 6.5: Input Validation and Sanitisation Mechanisms

Attack Vector	Mitigation Mechanism	Implementation
SQL Injection	Parameterised queries	Spring Data JPA / Hibernate
Broken access control	Role, ownership, and enrollment checks	Spring Security method security; service-level checks
Forged payment events	Webhook signature verification and idempotency	Payment Service webhook handler
Unauthorised file access	Authorisation before presigned URL	Material Service
Prompt injection / abuse	Guards, filters, and scoped tools	AI Service
Mass assignment / invalid input	DTO validation and enumeration whitelists	Bean Validation on request DTOs
Excessive payload / abuse	Size limits and usage limits	Gateway and upload configuration; AI usage guards

6.5 Encryption Standards for Data at Rest and in Transit

This section distinguishes the evidenced configuration from deployment targets. It does not claim universal encryption at rest, automated key rotation, secret-manager integration, or complete TLS coverage, none of which is demonstrated by the reviewed configuration.

6.5.0.1 Encryption in Transit

• The reviewed Caddyfile listens on HTTP port 80 and forwards to api-gateway:8080. That configuration alone is not evidence of end-to-end HTTPS. TLS termination (for example automatic certificates at Caddy or a managed load balancer) is the deployment target for production, with TLS 1.2 as the minimum version.
• Inter-service traffic is carried on the internal Docker network. Mutual TLS through a service mesh, described in Version 1.0, is not part of the deployment.
• External calls to Stripe, AI providers, S3-compatible storage, and SMTP use TLS as provided by those services and must use TLS 1.2 or later.

6.5.0.2 Encryption at Rest

• PostgreSQL data resides on a named Docker volume. Encryption at rest depends on the host or volume configuration of the chosen environment and is a target, not a verified property.
• Uploaded files are stored in S3-compatible object storage; server-side encryption depends on the storage provider configuration and is to be enabled and verified.
• Secrets such as database credentials, JWT keys, Stripe keys, and AI provider keys are supplied through runtime environment configuration and must not be stored in images or source control.

6.5.0.3 Key Management

• JWT signing keys are managed by the Auth Service configuration, and the public keys are published as a JWKS document. Key-rotation procedures must be documented once defined; automatic rotation is not claimed.
• Application secrets are injected at runtime. AWS KMS and AWS Secrets Manager described in Version 1.0 are future options and are not part of the reviewed deployment.

Table 6.6: Encryption Standards Summary

Data Category	Storage Location	Encryption Standard	Evidence Status

Relational data (all services)	PostgreSQL 14 container volume	AES-256 at rest (target)	Not verified; depends on host or volume
Files and media	S3-compatible object storage	Provider server-side encryption (target)	Not verified; provider configuration
Passwords	Auth Service database	BCrypt hash	Implemented per auth-service; cost factor to confirm
Secrets and API keys	Runtime environment configuration	Not committed to images or source	Configured; secret manager is a future option
Data in transit (client to gateway)	Caddy on HTTP port 80 in reviewed configuration	TLS 1.2+ (target)	Not evidenced in reviewed Caddyfile
Data in transit (inter-service)	Internal Docker network	No mutual TLS	Not claimed

6.6 Security Flows for Payments and Files

Two security-critical flows are documented in detail because they combine authentication, authorisation, and external providers.

6.6.0.1 Payment Webhook Verification

A payment is final only after Stripe's signed webhook is verified and processed. The raw body and signature header are validated, duplicate event identifiers are ignored, and only then is the payment status updated (Figure 68).

6.6.0.2 Authorised File Download

File access is decided by the Material Service before any storage URL is issued, and the decision is audited (Figure 69).

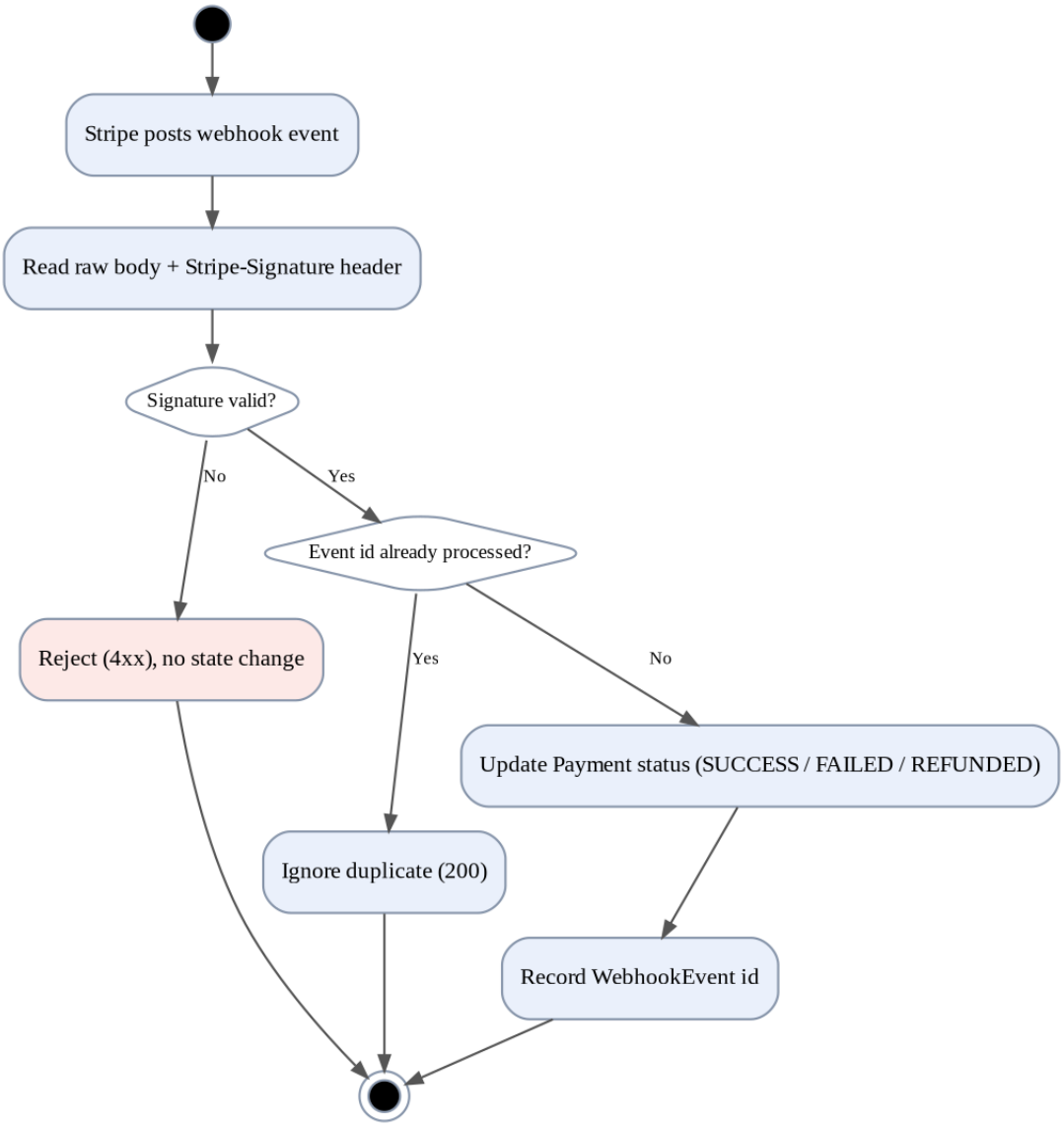


Figure 6.2: Payment Webhook Verification Flow

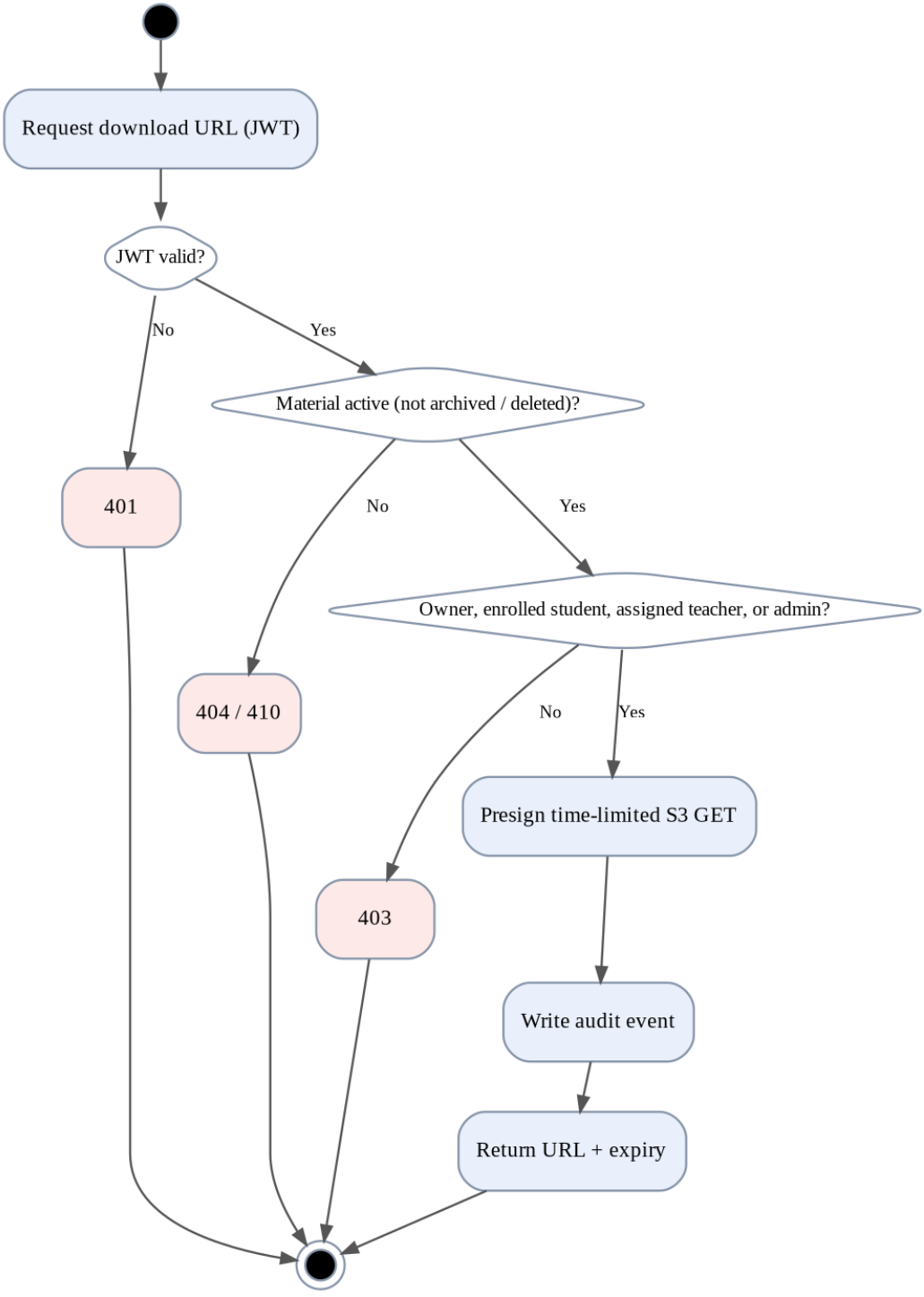


Figure 6.3: Authorised File Download Flow

Chapter 7

Deployment Design

This chapter presents the deployment design of the system, detailing the hosting strategy, capacity planning, network configuration, environment setup, and the continuous integration and delivery automation. The design reflects the Docker Compose configuration found in the backend and frontend repositories. A configuration file is evidence of deployment preparation; it is not evidence of a running production environment, and the chapter marks planned elements accordingly. Version 1.0 described an AWS EKS, ECR, RDS, MongoDB Atlas, and ALB deployment; that architecture is retained only as a future option in Section 7.5.4.

7.1 Hosting Plan

The system is defined for deployment as Docker containers orchestrated by Docker Compose on a Docker host, planned on an Amazon Web Services EC2 instance. The backend production Compose file includes the Spring Cloud infrastructure services, the business microservices, PostgreSQL 14, Redis 7.4, Apache Kafka 3.9.1, and a Caddy reverse proxy, with named persistent volumes, container health checks, and an internal Docker network. The API Gateway is exposed on loopback port 8080 and Caddy listens on port 80, forwarding to api-gateway:8080. The frontend production Compose file defines three portal containers (Administration, Teacher, Student) that join the backend's external Docker network, which creates a deployment dependency between the two stacks. The public website is present in the repository and is deployed separately. The following table summarises the hosting components.

Table 7.1: Hosting Plan Summary

Component	Service / Technology	Status	Purpose

Compute Host	Docker host (planned AWS EC2) running Docker Compose	Configured, Not Deployment-Verified	Hosts backend and frontend containers
Reverse Proxy	Caddy (port 80)	Configured	Forwards external requests to api-gateway:8080
API Gateway	Spring Cloud Gateway (loopback :8080)	Configured	Routing and first-level security
Service Discovery	Netflix Eureka	Configured	Dynamic service registration and discovery
Configuration	Spring Cloud Config Server	Configured	Centralised configuration
Business Services	14 Spring Boot containers	Configured	Domain functionality
Relational Database	PostgreSQL 14 container with named volume	Configured	Persistence for all services (single instance)
Cache	Redis 7.4 container	Configured	Caching and coordination
Message Broker	Apache Kafka 3.9.1 (single broker)	Configured	Asynchronous events
Object Storage	S3-compatible storage	Integration implemented	File content storage
Portals	Administration, Teacher, Student containers	Configured	Frontend experiences
Public Website	Next.js application	Deployed separately	Public information and pre-registration
Managed AWS Services	EKS, ALB, RDS, WAF, CloudWatch	Future Design	Possible production upgrades

Docker images are referenced through the configurable DOCKERHUB_IMAGE setting. The earlier description of AWS ECR, AWS API Gateway, CloudFront with S3 static hosting, and Nginx as the load balancer is superseded. Production deployment on AWS EC2 is intended and has not been evidenced by the reviewed repositories.

7.2 Hardware Requirements

Hardware capacity is not proven by the choice of an instance type. The reviewed deployment runs seventeen Java service containers, PostgreSQL, Kafka, Redis, Caddy, and up to three portal containers on one Docker host. The original plan of T3 Micro and T3 Small instances (1 to 2 GB of memory each) predates this decomposition and must be re-validated by measurement. The table below records the resource profile of each group so that the host can be sized; the figures are planning estimates, not measured values.

Table 7.2: Deployment Resource Planning Profile

Service Group	Resource Profile	Planning Note (Estimate)
Spring Cloud infrastructure (gateway, registry, config)	Three JVM containers; low CPU, moderate memory	Start first; gateway latency affects all requests
Business microservices (14)	Fourteen JVM containers; memory is the main constraint	Budget memory per JVM and measure at idle and under load
PostgreSQL 14	Single container; disk and memory for all service databases	Persistent volume, backups, and disk growth to be planned
Apache Kafka 3.9.1	Single broker; disk and memory	No broker redundancy; retention and disk growth to be monitored
Redis 7.4	Small memory footprint	Persistence and eviction policy to be configured as needed
Portals (3) and Caddy	Next.js containers and a lightweight proxy	Low CPU; memory per container
Object storage	External S3-compatible service	Capacity managed by the provider

The final host size (CPU, memory, disk, and network) must be recorded once confirmed by the deployment plan and measured under realistic load. The deployment must not be described as highly available simply because Docker restarts crashed containers. PostgreSQL and Kafka persistence, backups, and disk growth are operational risks that require explicit procedures.

7.3 Network Diagram

The network diagram below shows the external users, the reverse proxy, the frontend containers, the API Gateway, the backend network with the business services, the data and messaging infrastructure, and calls to external providers (Figure 70). It reflects the reviewed Compose configuration; AWS ALB, EKS ingress, and managed TLS termination are not drawn because they are not deployed.

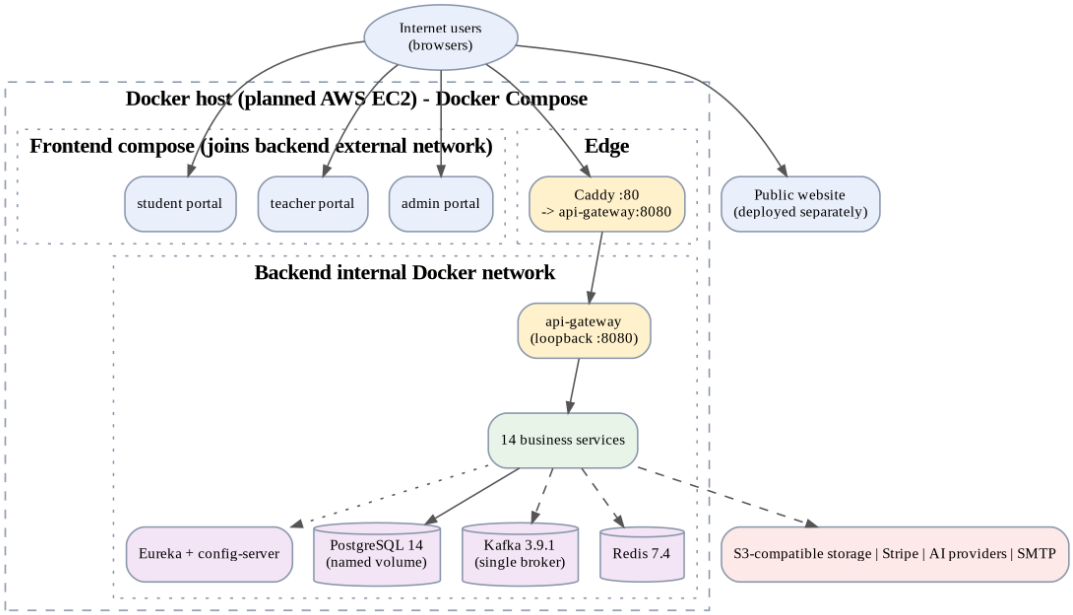


Figure 7.1: System Network and Deployment Architecture Diagram

7.3.0.1 Network Layer Description

External clients reach the Caddy reverse proxy on port 80, which forwards supported requests to the API Gateway on loopback port 8080. The portal containers, which attach to the backend's external Docker network, are accessed by browsers and call the API Gateway. Within the backend Docker network, services discover each other through Eureka, obtain configuration from the Config Server, and communicate through HTTP/OpenFeign and Kafka. PostgreSQL, Redis, and Kafka are reachable on the internal network only. Calls to Stripe, AI providers, SMTP, and S3-compatible storage leave the host over the public network.

The security boundary separates public routes (the proxy, the public website, public authentication routes, and the published marketplace catalogue) from internal services, which are not published externally. The dependency of the portal Compose file on the backend's named external network means the backend stack must be started before the portal stack.

7.4 Environment Specification

Distinct environments are described below. A staging environment is not claimed as deployed merely because a staging workflow exists.

Table 7.3: Environment Specification Matrix

Attribute	Local Development	CI	Controlled Integration	Staging	Production
Purpose	Developers run the application and dependencies locally	Static and unit verification, builds, and contract validation	Live backend services tested with synthetic identities and approved fixtures	Separately approved non-production deployment for restricted release smoke verification	Intended deployed configuration for real users
Infrastructure	Docker Compose on a developer machine	GitHub Actions runners	Backend stack reachable by the test suite	Approved non-production deployment	Docker host (planned AWS EC2) running Docker Compose
Data	Synthetic data	No live data	Synthetic test identities and fixtures	Non-production data	Live data
Secrets	Local environment files	Repository or workflow secrets	Test credentials, non-production	Staging configuration	Runtime environment configuration
Access	Developers	Maintainers	Authorised testers	Approved staff	Operations team
Status	In use	Configured	Opt-in; requires enabled profile	Smoke checks are manual-only	Configured, Not Deployment-Verified

7.5 CI/CD Pipeline Overview

Automation is implemented with GitHub Actions in the three repositories: frontend, backend (Mentify-backend), and tests. Each repository uses its development branch as the integration baseline. The workflows have different roles, and a configured workflow is not the same as a successful execution or a completed deployment.

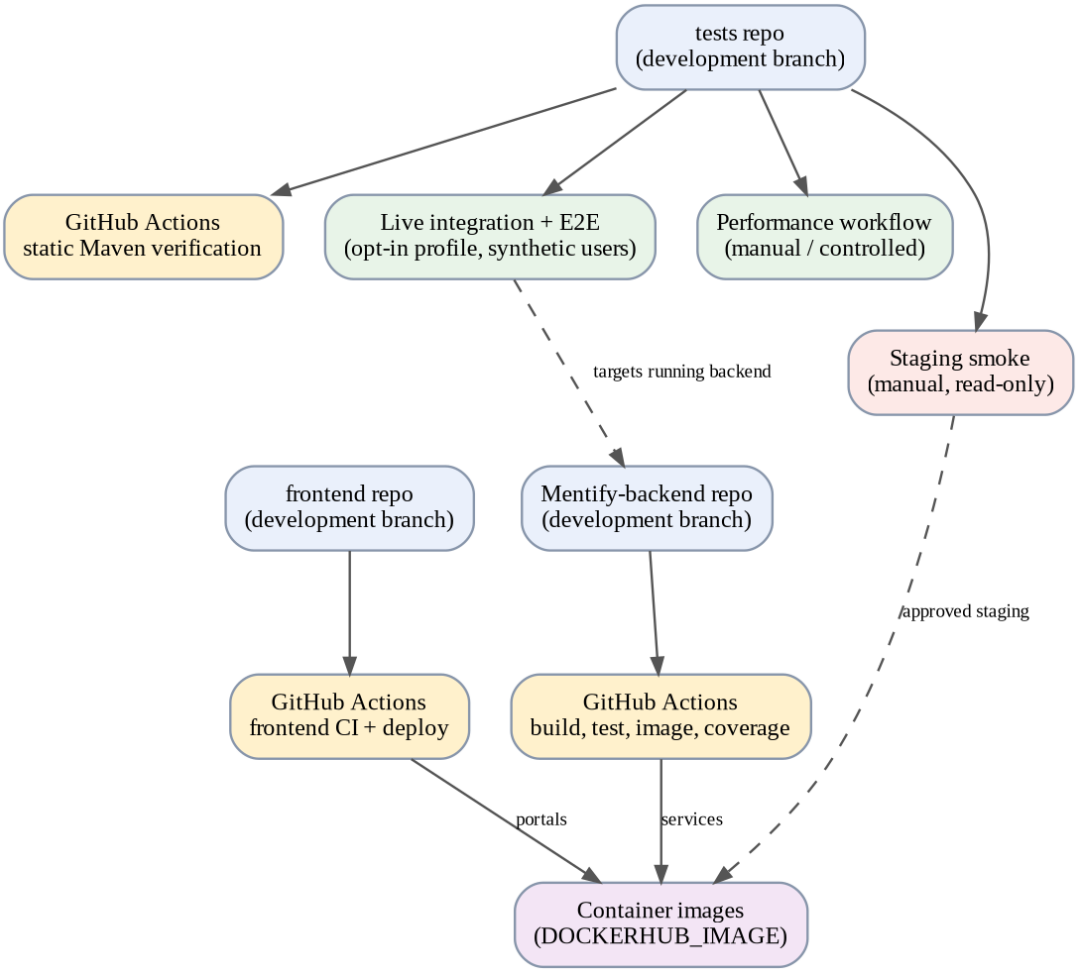


Figure 7.2: CI/CD and Test Architecture Diagram

7.5.0.1 Source Control and Branching Strategy

Source code is managed in Git repositories hosted on GitHub: the frontend repository, the Mentify-backend repository, and the tests repository. The development branch of each repository is the baseline documented in this version. The main branch holds the stable line, and feature work is integrated through the development branch with peer review. The detailed branch naming rules of Version 1.0 (release and hotfix branches) are to be applied as the team finalises its release process.

7.5.0.2 Pipeline Stages

The stages below describe the automation as designed across the repositories. Stages that depend on a live environment are opt-in or manual.

Table 7.4: CI/CD Pipeline Stages

Stage	Name	Description	Tooling
1	Code Commit and Trigger	A push or pull request to a repository triggers its GitHub Actions workflow.	GitHub Actions
2	Frontend Build and CI	The frontend workflow installs dependencies, lints and builds the Next.js applications; a frontend deployment workflow exists separately.	Node.js, Next.js, GitHub Actions
3	Backend Build and Test	The backend workflow compiles the services, runs unit tests, and produces coverage artefacts.	Maven, JUnit, GitHub Actions
4	Container Image Creation	Service and portal images are built and referenced through DOCKERHUB_IMAGE.	Docker
5	Static Test Verification	The test repository runs static Maven verification without a live backend.	Maven, JUnit (Java 17)

6	Live Integration and E2E Tests	Opt-in suites run against a running backend with synthetic identities and fixtures through an explicit integration profile.	Maven, JUnit, REST clients
7	Performance Checks	Lightweight performance workflows run manually or in controlled conditions; they are not production capacity guarantees.	Test repository workflows
8	Staging Smoke	A manual workflow performs bounded read-only checks against an approved staging environment; it does not deploy or certify a release.	Manual workflow
9	Deployment	Images are pulled and started with Docker Compose on the host; deployment automation exists where configured and must be evidenced per release.	Docker Compose

7.5.0.3 External Integrations in Deployment

Table 7.5: External Integration Summary

Integration	Provider	Role in Deployment

Container Registry	Docker Hub (image name via DOCKER-HUB_IMAGE)	Stores and versions service and portal images
AI Model API	Gemini / OpenAI (configured adapters)	Summarisation, quiz drafts, and AI conversation
Payments	Stripe	Course payments, webhooks, refunds
Object Storage	S3-compatible storage	Learning resources
Version Control Hosting	GitHub	Source repositories, workflows, and reviews
Email / SMTP	SMTP provider	Invitations, OTP, and notification e-mails

7.5.0.4 Future Production Upgrade Path

The following upgrades are future design options. None is part of the reviewed deployment and no cost estimate is asserted here.

- Kubernetes (for example AWS EKS) with Horizontal Pod Autoscaling and self-healing in place of Docker Compose.
- A managed load balancer (AWS ALB) across multiple hosts or availability zones with managed TLS certificates.
- A multi-broker Kafka cluster for fault-tolerant event streaming.
- A managed or replicated PostgreSQL deployment (for example RDS Multi-AZ) with automated failover and backups.
- AWS WAF, AWS Secrets Manager, and AWS KMS for edge protection and key and secret management.
- Monitoring and alerting (for example CloudWatch) for infrastructure metrics and health.

Bibliography

[1] VMware, “What Is Microservices Architecture?,” VMware, 2023. [Online]. Available: https://www.vmware.com/topics/glossary/content/microservices-architecture.html. [Accessed: Apr. 2026].

[2] Pivotal Software, “Spring Boot Reference Documentation,” Spring, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/. [Accessed: Apr. 2026].

[3] Spring, “Spring Cloud Gateway,” Spring Cloud Documentation. [Online]. Available: https://spring.io/projects/spring-cloud-gateway. [Accessed: Oct. 2026].

[4] Spring, “Spring Cloud OpenFeign,” Spring Cloud Documentation. [Online]. Available: https://spring.io/projects/spring-cloud-openfeign. [Accessed: Oct. 2026].

[5] Pivotal Software, “Spring Cloud Config Reference Documentation,” Spring, 2024. [Online]. Available: https://docs.spring.io/spring-cloud-config/docs/current/reference/html/. [Accessed: Apr. 2026].

[6] Netflix OSS, “Netflix Eureka – Service Discovery,” GitHub, 2023. [Online]. Available: https://github.com/Netflix/eureka. [Accessed: Apr. 2026].

[7] Apache Software Foundation, “Apache Kafka Documentation,” Apache Kafka, 2024. [Online]. Available: https://kafka.apache.org/documentation/. [Accessed: Apr. 2026].

[8] Redis Ltd., “Redis Documentation,” Redis, 2024. [Online]. Available: https://redis.io/docs/. [Accessed: Apr. 2026].

[9] Docker Inc., “Docker Documentation,” Docker, 2024. [Online]. Available: https://docs.docker.com/. [Accessed: Apr. 2026].

[10] Caddy, “Caddy Web Server Documentation,” 2024. [Online]. Available: https://caddyserver.com/docs/ [Accessed: Oct. 2026].

[11] GitHub, “GitHub Actions Documentation,” GitHub, 2024. [Online]. Available: https://docs.github.com [Accessed: Apr. 2026].

[12] Amazon Web Services, “Amazon Simple Storage Service (S3),” AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html. [Accessed: Apr. 2026].

[13] Amazon Web Services, “Amazon Elastic Kubernetes Service (EKS),” AWS Documentation, 2024 (future option). [Online]. Available: https://docs.aws.amazon.com/eks/latest/userguide/wl is-eks.html. [Accessed: Apr. 2026].

[14] Stripe, “Stripe API Documentation: Payment Intents and Webhooks.” [Online]. Available: https://stripe.com/docs. [Accessed: Oct. 2026].

[15] OpenAI, “OpenAI API Reference,” OpenAI, 2024. [Online]. Available: https://platform.openai.com/d reference. [Accessed: Apr. 2026].

[16] Google LLC, “Gemini API Documentation,” Google AI for Developers, 2024. [Online]. Available: https://ai.google.dev/gemini-api/docs. [Accessed: Apr. 2026].

[17] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA: O’Reilly Media, 2021.

[18] M. Fowler and J. Lewis, “Microservices,” martinfowler.com, 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html. [Accessed: Apr. 2026].

[19] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Harlow: Pearson, 2016.

[20] C. J. Date, An Introduction to Database Systems, 8th ed. Reading, MA: Addison-Wesley, 2004.

[21] The PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2024. [Online]. Available: https://www.postgresql.org/docs/. [Accessed: Oct. 2026].

[22] Redgate, “Flyway Documentation.” [Online]. Available: https://documentation.redgate.com/flyway. [Accessed: Oct. 2026].

[23] E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[24] R. Ramakrishnan and J. Gehrke, Database Management Systems, 3rd ed. New York, NY: McGraw-Hill, 2003.

[25] Object Management Group, “Unified Modeling Language (UML) Specification, Version 2.5.1,” OMG, Dec. 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/. [Accessed: Apr. 2026].

[26] G. Booch, J. Rumbaugh, and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Reading, MA: Addison-Wesley, 2005.

[27] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Reading, MA: Addison-Wesley, 2003.

[28] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Upper Saddle River, NJ: Prentice Hall, 2017.

[29] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA: Addison-Wesley, 2003.

[30] Internet Engineering Task Force, "RFC 7519: JSON Web Token (JWT)," IETF, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519. [Accessed: Apr. 2026].

[31] Internet Engineering Task Force, "RFC 7517: JSON Web Key (JWK)," IETF, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7517. [Accessed: Apr. 2026].

[32] Internet Engineering Task Force, "RFC 6455: The WebSocket Protocol," IETF, Dec. 2011. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6455. [Accessed: Apr. 2026].

[33] Pivotal Software, "Spring Security Reference – Method Security," Spring, 2024. [Online]. Available: https://docs.spring.io/spring-security/reference/servlet/authorization/method-security.html. [Accessed: Apr. 2026].

[34] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/. [Accessed: Apr. 2026].

[35] World Wide Web Consortium, "Accessible Rich Internet Applications (WAI-ARIA) 1.2," W3C Recommendation, Jun. 2023. [Online]. Available: https://www.w3.org/TR/wai-aria-1.2/. [Accessed: Apr. 2026].

[36] Vercel, "Next.js Documentation." [Online]. Available: https://nextjs.org/docs. [Accessed: Oct. 2026].

[37] React Team, "React Documentation," Meta Open Source, 2024. [Online]. Available: https://react.dev/. [Accessed: Oct. 2026].

[38] S. Krug, Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability, 3rd ed. San Francisco, CA: New Riders, 2014.

[39] OWASP Foundation, “OWASP Top Ten,” OWASP, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 2026].

[40] Internet Engineering Task Force, “RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3,” IETF, Aug. 2018. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446 [Accessed: Apr. 2026].

[41] National Institute of Standards and Technology, “FIPS PUB 197: Advanced Encryption Standard (AES),” NIST, Nov. 2001. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/1 [Accessed: Apr. 2026].

[42] N. Provos and D. Mazieres, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annual Technical Conference, Jun. 1999. [Online]. Available: https://www.usenix.org/legacy/events/u [Accessed: Apr. 2026].

[43] Stripe, “Check the webhook signatures,” Stripe Documentation. [Online]. Available: https://stripe.com/docs/webhooks. [Accessed: Oct. 2026].

[44] OWASP Foundation, “OWASP SQL Injection Prevention Cheat Sheet,” OWASP, 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat [Accessed: Apr. 2026].

[45] Jakarta EE, “Bean Validation 3.0 (JSR 380) Specification,” Eclipse Foundation, 2020. [Online]. Available: https://beanvalidation.org/3.0/. [Accessed: Apr. 2026].

[46] Pivotal Software, “Spring Security Reference Documentation,” Spring, 2024. [Online]. Available: https://docs.spring.io/spring-security/reference/. [Accessed: Apr. 2026].

[47] National Institute of Standards and Technology, “NIST SP 800-63B: Digital Identity Guidelines – Authentication and Lifecycle Management,” NIST, Jun. 2017. [Online]. Available: https://pages.nist.gov/800-63-3/sp800-63b.html. [Accessed: Apr. 2026].

[48] Amazon Web Services, “Amazon EC2 Instance Types,” AWS Documentation, 2024. [Online]. Available: https://aws.amazon.com/ec2/instance-types/. [Accessed: Apr. 2026].

[49] Docker Inc., “Docker Compose Documentation,” Docker, 2024. [Online]. Available: https://docs.docker.com/compose/. [Accessed: Apr. 2026].

[50] JUnit Team, “JUnit 5 User Guide,” JUnit, 2024. [Online]. Available: https://junit.org/junit5/docs/curre guide/. [Accessed: Apr. 2026].

[51] Pivotal Software, “Spring Boot Actuator: Production-ready Features,” Spring, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/actuator.html.

[Accessed: Apr. 2026].

[52] Mentify, frontend repository (development branch), SENG-34213/frontend, GitHub. [Accessed: Oct. 2026].

[53] Mentify, backend repository (development branch), SENG-34213/Mentify-backend, GitHub. [Accessed: Oct. 2026].

[54] Mentify, tests repository (development branch), SENG-34213/tests, GitHub. [Accessed: Oct. 2026].

Appendix A

Interface Design and Distributed Workflows

Version 1.0 gave little detail about service interfaces. This appendix defines the interface documentation template and a catalogue of interface groups. Only routes that are evidenced in the repositories are stated explicitly; for all other interfaces the route is to be transcribed from the controller mappings and the API Gateway configuration, since endpoint routes must not be inferred from file names.

A.0.1 A.1 Interface Documentation Template

Table A.1: Interface Documentation Template

Field	**What to Document**
Interface identifier	A unique identifier such as INT-AUTH-01
Owning service	The service that implements the interface
Consumer	Student, Teacher, or Admin portal, public website, or another service
Purpose	What the interface achieves
Method / route	Exact controller mapping and gateway route
Authentication	Public, authenticated, role-restricted, or internal
Request	Exact request DTO fields and validation
Response	Exact response DTO and status codes
Success conditions	Verified application behaviour
Error conditions	Invalid credentials, conflicts (409), forbidden (403), validation (400), and other defined failures

Table A.1 continued

Field	What to Document
Source	Controller and service implementation
Tests	Matching test case and execution evidence

A.0.2 A.2 Interface Catalogue

Table A.2: Interface Catalogue

ID	Owning Service	Consumer	Purpose	Access	Route / Source
INT-AUTH-01	auth-service	All portals	Login, token refresh, and logout	Public / authenticated	AuthController; To be transcribed from the controller mapping
INT-AUTH-02	auth-service	Gateway, services	Publish public keys (JWKS)	Public	`/api/v1/auth/.well-known/jwks.json`
INT-AUTH-03	auth-service	All portals	OTP password recovery and password change	Public / authenticated	AuthController; To be transcribed from the controller mapping
INT-USER-01	user-service	Admin portal	Register users and administrators, resend invitation, update status, delete	ADMIN, SUPER_ADMIN (admin accounts: SUPER_ADMIN)	User controller; To be transcribed from the controller mapping
INT-CRS-01	course-service	Admin, Teacher, Student portals	Grades, courses, modules, lessons; deletion restricted	Role-restricted	CourseController, CourseRequest
INT-ENR-01	entrollment-service	Student, Admin, Teacher portals	Self-enrollment, staff enrollment, status update, statistics	Role and ownership	EntrollmentServiceImpl; To be transcribed from the controller mapping
INT-ASG-01	assignment-service	Teacher, Student portals	Assignments, submissions, grading, return	Role and ownership	GradingServiceImpl; To be transcribed from the controller mapping
INT-ASG-02	assignment-service	Teacher, Student portals	Quizzes, AI draft intake, attempts, scoring	Role and ownership	QuizGradingServiceImpl, CreateQuizRequest
INT-ATT-01	attendance-service	Teacher, Student, Admin portals	Sessions, records, summaries, analytics	Role and ownership	Attendance controller; To be transcribed from the controller mapping

Table A.2 continued

ID	Owning Service	Consumer	Purpose	Access	Route / Source
INT-EXM-01	exam-service	Teacher, Admin, Student portals	Exams, attachments, marks, results	Role and publication rules	Exam controller; To be transcribed from the controller mapping
INT-MAT-01	material-service	All portals	Upload, search, metadata, archive, authorised download URL	Authenticated and authorised	MaterialController; To be transcribed from the controller mapping
INT-MKT-01	material-service	Admin portal, public website	Marketplace products (admin) and published listing (public)	ADMIN / public	MarketplaceProductController; To be transcribed from the controller mapping
INT-PAY-01	payment-service	Student, Admin portals	Initiate payment, status, history, administrator search, refund	Student own data; ADMIN	PaymentController; To be transcribed from the controller mapping
INT-PAY-02	payment-service	Stripe	Signed webhook events	Signature verified	Payment webhook handler; To be transcribed from the controller mapping
INT-COM-01	communication-service	All portals	Groups, membership, messages, announcements (REST)	Membership-based	Communication controller; To be transcribed from the controller mapping
INT-COM-02	communication-service	All portals	Real-time messaging (WebSocket / STOMP)	JWT; membership	WebSocket configuration; To be transcribed from the controller mapping
INT-NOT-01	notification-service	All portals	List and mark notifications	Own data	Notification controller; To be transcribed from the controller mapping
INT-CAL-01	calender-service	All portals	Calendar events, venues, queries, projections	Role-based visibility	Calendar controller; To be transcribed from the controller mapping
INT-ANA-01	analytics-service	Admin, Teacher, Student portals	Dashboards and summaries	Role and scope	Analytics controller; To be transcribed from the controller mapping
INT-AI-01	ai-service	Teacher, Admin portals	Conversational AI chat	TEACHER, ADMIN, SUPER_ADMIN	`/api/ai/chat`

Table A.2 continued

ID	Owning Service	Consumer	Purpose	Access	Route / Source
INT-AI-02	ai-service	Teacher, Admin portals	AI chat over WebSocket	TEACHER, ADMIN, SUPER_ADMIN	AI WebSocket controller
INT-AI-03	ai-service	All roles	Lesson and material summarisation	STUDENT, TEACHER, ADMIN, SUPER_ADMIN	AI summarisation controller; To be transcribed from the controller mapping
INT-EVT-01	Kafka	Producers and notification-service	Success events for notifications	Internal	Topic names and contracts to be transcribed

A.0.3 A.3 Distributed Workflow Rules

- **Enrollment controls access:** Course materials, quizzes, and course groups are available only to students with APPROVED enrollment.
- **Payment gates enrollment:** A successful payment verified by the Payment Service is required for self-enrollment in a paid course.
- **Academic actions create secondary data:** Exam, attendance, assignment, and quiz events create calendar projections and notifications.
- **Failure handling:** External failures (Stripe, AI providers, SMTP, object storage) affect only the dependent features, and calendar and group synchronisation use reconciliation or non-strict paths.

Appendix B

Software Testing and Quality Evidence

The tests repository (development branch) is separate from the application code and provides the verification strategy for the design. This appendix records the testing architecture, the execution model, the limitations that must not be overlooked, and a traceability matrix from design features to test evidence.

B.0.1 B.1 Testing Architecture

The tests repository uses Java 17, Maven, JUnit-based suites, reusable API clients, authentication clients, integration tests, and gateway end-to-end workflows. Documented areas include authentication, user management, courses, enrollment, assignments and attachments, quizzes, exams, attendance, materials, payments, communications, notifications, AI, calendar, security, resilience, performance, and staging.

B.0.2 B.2 Test Execution Model

Table B.1: Test Execution Model

Test Type	**Status**	**Description**
Static / unit verification	CI configured	Runs independently of a live backend in the normal test repository workflow.
Live integration testing	Opt-in	Requires backend services, credentials, non-production fixtures, and an explicit integration profile.
End-to-end workflows	Environment-gated	Require explicit enablement and valid test data.
Performance testing	Manual / controlled	Lightweight regression checks; not production capacity guarantees.

Table B.1 continued

Test Type	Status	Description
Staging release smoke	Manual-only	Bounded read-only checks of an approved staging environment; it neither deploys nor certifies a release.

B.0.3 B.3 Testing Limitations

The tests repository contains an audit dated 21 October 2026. That audit states that it was based on a particular feature-branch worktree and an earlier backend snapshot, so its statistics are historical findings and not necessarily final figures for the current development branches. It reported 232 annotated integration test methods, 65 of which were not mapped in its case-mapping snapshot, and stated that service test coverage was substantial but not complete.

A documentation inconsistency also exists: `docs/known-limitations.md` and `docs/ci-test-execution.md` still describe the test-case mapping CSV as having no data rows, whereas the newer coverage-audit text reports populated mappings. This must be reconciled before mapping statistics are quoted in the final SDS. The following must never be equated:

- Test files existing with tests having passed.
- Passing static CI with live integration success.
- A defined test case with executed test evidence.
- The number of test methods with backend code coverage.
- A staging smoke workflow with a completed staging verification.
- A successful staging smoke run with full release certification.

B.0.4 B.4 Design-to-Test Traceability Matrix

Requirement identifiers refer to the revised SRS (Version 02). Test evidence refers to packages of the tests repository; execution results are to be recorded per run.

Table B.2: Design-to-Test Traceability Matrix

SDS Design Feature	Backend Source	Frontend Source	Test Evidence (Package)
JWT authentication (FR-AU001–AU008)	auth-service, api-gateway	Portal auth modules	`tests/src/test/java/com/mentify/tests/auth/`

Table B.2 continued

SDS Design Feature	Backend Source	Frontend Source	Test Evidence (Package)
User management (FR-A001–A003)	user-service	Admin users views	`tests/src/test/java/com/mentify/tests/user/`
Course management (FR-GC001–GC008)	course-service	Admin and teacher course views	`tests/src/test/java/com/mentify/tests/course/`
Course enrollment (FR-EN001–EN008)	entrollment-service	Student enrollments, admin enrollments	`tests/src/test/java/com/mentify/tests/enrollment/`
Assignment lifecycle (FR-T021–T029)	assignment-service	Student and teacher assignment components	`tests/src/test/java/com/mentify/tests/assignment/`
Quiz and scoring (FR-T024–T026)	assignment-service	Teacher quiz generator, student quiz taking	`tests/src/test/java/com/mentify/tests/assignment/` (quiz tests)
Examinations (FR-EX001–EX006)	exam-service	Admin and student exam views	`tests/src/test/java/com/mentify/tests/exam/`
Attendance (FR-AT001–AT005)	attendance-service	Teacher attendance workspace	`tests/src/test/java/com/mentify/tests/attendance/`
Materials and marketplace (FR-MT, FR-MK)	material-service	Materials and market-place views	`tests/src/test/java/com/mentify/tests/material/`
Payment processing (FR-PY001–PY009)	payment-service	Portal billing views; public website payments are placeholders	`tests/src/test/java/com/mentify/tests/payment/`
Communication and notifications (FR-CN)	communication-service, notification-service	Messages and notification views	`tests/src/test/java/com/mentify/tests/communication/` and `notification/`
Calendar (FR-CA001–CA004)	calender-service	Calendar views	`tests/src/test/java/com/mentify/tests/calendar/`
AI chat (FR-AI003, FR-AI008–AI012)	ai-service	Teacher / admin AI views; student API contract needs reconciliation	`tests/src/test/java/com/mentify/tests/ai/`
Security, resilience, performance	All services and gateway	Not applicable	`tests/src/test/java/com/mentify/tests/security/`, `resilience/`, `performance/`

Appendix C

Implementation Status And Known Deviations

A technically credible SDS discloses where the implementation deviates from its intended design. The following known deviations are recorded so that they are not hidden. Design elements follow the status convention in Section 1.1.6.

Table C.1: Known Deviations and Implementation Status

ID	Deviation	Evidence	Resolution Required
KD-01	Student AI access inconsistency: the student client calls a route that denies the Student role.	student/src/api/ai.ts calls /api/ai/chat; backend controller uses hasAnyRole TEACHER, ADMIN, SUPER_ADMIN; WebSocket controller has the same restriction.	Reconcile the contract; until then the student chatbot is not presented as operational (status: Partially Implemented).
KD-02	Public paid-resource storefront is incomplete.	Website resource catalogue declares sample resources and the public payment module states no provider is connected.	Do not caption screens as a completed paid-material workflow.

KD-03	Repository README files contain stale architecture descriptions.	The backend README still mentions MongoDB for AI while ai-service uses PostgreSQL, JPA, and Flyway.	Correct the README or rely on source and migrations.
KD-04	Test reports require version reconciliation.	The test repository holds historical mapping counts and documents with differing descriptions of mapping completion.	Reconcile before quoting statistics.
KD-05	Staging validation is conditional.	Staging smoke code exists but is manual-only and not proof of a completed staging run.	Record an executed run before claiming staging verification.
KD-06	Kubernetes is not established.	Reviewed production definitions are Docker Compose files; no HPA, EKS, or RDS configuration.	Keep Kubernetes as Future Design.
KD-07	Features differ in completion level.	A UI screen, API client, controller, test, and verified workflow are different levels of evidence.	Label each feature with the status convention.
KD-08	TLS is not evidenced in the reviewed edge configuration.	Caddyfile listens on port 80 and forwards to api-gateway:8080.	Document the production TLS termination once configured.

KD-09	Single-instance infrastructure.	One PostgreSQL service and one Kafka broker in the production Compose file.	Do not describe as fault-tolerant; plan replication as a future upgrade.
KD-10	Version 1.0 features not found in the implementation.	Personal notes, forums, private messaging, AI recommendations, AI assignment grading, support tickets, and Google OAuth are not established.	Recorded as removed or deferred in Appendix E.

Appendix D

Architectural Decision Records

The development guidelines require significant architectural changes to be recorded as Architectural Decision Records (ADRs) and reflected in the SDS. The following register summarises the decisions that distinguish this version from Version 1.0. Each entry should be expanded into a full ADR in the project repository, avoiding duplicates of any ADR already recorded by the team.

Table D.1: Architectural Decision Register

ADR	Decision	Reason for Change and Alternatives	Consequences
ADR-01	Adopt Spring Boot microservices with a gateway, registry, and config server	Domain separation and independent deployment; alternatives: monolith, layered MVC	Operational complexity and distributed consistency
ADR-02	First-party JWT authentication service with JWKS	Replaces Google OAuth; alternatives: Keycloak, external identity provider	Credential ownership and key management are the team's responsibility
ADR-03	PostgreSQL for all services, logical database per service	Replaces MongoDB for AI, Analytics, and Communication; alternatives: polyglot persistence	Single database technology; one container hosts all databases
ADR-04	Kafka for asynchronous integration	Decouples notification delivery; alternatives: RabbitMQ, direct calls	Single broker; at-least-once handling requires idempotency

ADR-05	S3-compatible object storage with metadata in material-service	Separates file content from authorisation; alternatives: database blobs	Presigned URLs and audit events needed
ADR-06	Stripe Payment Intents with signed webhooks	Payment verification before enrollment; alternatives: manual payment	Webhook idempotency and failure handling required
ADR-07	Grade-based fee configuration	Fees defined per grade; alternatives: per-course pricing	Not recurring billing; amounts snapshotted per payment
ADR-08	Role-restricted AI service with persisted conversations and guards	Control cost and data exposure; alternatives: open student chatbot	Student chat deferred; tool scopes must be maintained
ADR-09	WebSocket/STOMP for communication and AI chat	Real-time delivery; alternatives: polling	Authentication of WebSocket sessions required
ADR-10	Administrator-managed marketplace catalogue (no checkout)	Publish learning resources without payment flow; alternatives: full storefront	Checkout and entitlement deferred
ADR-11	Docker Compose deployment instead of EKS	Lower cost and complexity for the current scale; alternatives: Kubernetes	Limited scaling and availability; EKS is a future option

Appendix E

Sds Version 1.0 To 2.0 Change Register

This register records the principal changes made to the original SDS so that the document history remains traceable.

Table E.1: SDS Version 1.0 to 2.0 Change Register

SDS Section	Version 1.0 Content	Version 2.0 Action
Ch. 1 Table 1	Six microservices (User, Assessment, Course, Communication, Analytics, AI)	Replaced by 14 business services, three infrastructure services, and a shared library
Ch. 1.1.2	Mobile and web client	Replaced by four web experiences (Admin, Teacher, Student portals and public website)
Ch. 1.1.3 / 1.2	AWS ALB, EKS, ECR, RDS, MongoDB Atlas, HPA	Replaced by Caddy, Spring Cloud Gateway, Docker Compose, PostgreSQL container, Redis, and Kafka; EKS retained as future option
Ch. 1.2.4	Google OAuth, Gmail SMTP, OpenAI	Stripe, AI providers, SMTP, and S3-compatible storage; Google OAuth removed

Ch. 2 ER diagrams	Six ERDs; SupportAgent; passwordHash in User; Analytics owns attendance	Fourteen service ERDs; credentials separated; Attendance, Enrollment, Material, Payment, Notification, and Calendar added
Ch. 2.2 Normalisation	All relations in 3NF	Table-by-table analysis with documented denormalisation
Ch. 3 Class diagrams	Seven diagrams for six services	Fifteen diagrams per domain plus inter-service component diagram
Ch. 4 Sequence diagrams	11 use cases incl. AI grading, AI help chatbot, personal notes	19 use cases; AI grading replaced by teacher grading; chatbot replaced; personal notes withdrawn; nine added
Ch. 5 UI/UX	Ten Mentify AI / Cognitive Gallery mock-ups; four flows	Ten portal and website wireframes; five flows; accessibility restated as design intent
Ch. 6 Security	Google OAuth, Redis sessions, mTLS, KMS, Secrets Manager	First-party JWT and JWKS, ownership checks, evidenced versus target controls
Ch. 7 Deployment	EKS, ALB, Nginx, CloudFront, RDS, MongoDB, cost estimates	Docker Compose topology, planned EC2, environments, CI/CD by repository; upgrades as future options
Ch. 8 References	68 references including AWS EKS and MongoDB Atlas	Updated references for technologies in use
New appendices	None	Appendices A to E: interfaces, testing evidence, known deviations, ADRs, change register